ragfarm — on-prem RAG + infra-control agent

A living reference. Generated 2026-07-27 00:30 UTC. ewline ewline Source of truth is the repository; this PDF is a snapshot.

David Kubicek (kubicek@gmail.com)

July 27, 2026

Contents

\newpage

Chapter 1 Introduction

Introduction — what this document is and why it exists

This bundle is the single authoritative reference for the **ragfarm** proof-of-concept: a fully on-prem retrieval-augmented generation (RAG) assistant with infrastructure-control capabilities, running today on a small AMD Ryzen AI 9 HX 370 mini-PC with a Radeon 890M integrated GPU, and portable, unchanged, to production NVIDIA hardware.

It is generated on demand from the repository's live source-of-truth documents (README, deployment guide, architecture decision records, prompt library, model records, tracing tool docs, and the current progress ledger). No content in this PDF exists only here — every chapter has a stable home in the repo tree, and this bundle only concatenates and typesets them into one printable, offline-readable artifact. When any of those source documents change, the PDF is regenerated from `docs/pdf/build.sh` in a few seconds.

Why on-prem RAG matters right now

Two forces meet at this point in time. The first is the practical maturity of open-weight, apache-licensed transformer models in the 4–30 billion-parameter range: Qwen 2.5, Qwen 3, and Qwen 3-VL specifically. These models — quantized to Q4_K_M and served through llama.cpp — are competent enough for structured document retrieval, tabular data extraction, tool-driven infrastructure operations, and, in the vision variants, direct optical understanding of

scanned pages, screenshots, diagrams and photographs. They fit on hardware operators already own.

The second force is the regulatory and business tension around cloud AI. Corporate documentation, employee contacts, network topology, credentials, and operational runbooks either cannot legally leave the enterprise perimeter (energy, finance, healthcare, defense, NIS2) or represent competitive information whose loss to a third-party model provider is unacceptable. Cloud AI trades ease of setup for a permanent flow of internal data through vendors' servers, permanent per-token cost per query, and a permanent lock-in to whichever provider best matches this quarter's price/quality trade-off.

ragfarm is the concrete demonstration that the trade is optional. The same natural-language interface — the same OpenAPI-compatible chat, the same tool-calling, the same vision capabilities — can run entirely inside the enterprise perimeter on hardware bought once, with no per-token cost, no data egress, no vendor dependency for the working system.

What is genuinely custom about this build

Most open-source LLM stacks are collections of upstream projects glued together with wiring code. This one is honestly the same, and the wiring is deliberately thin so the pieces stay independently upgradeable. But there are five areas where genuinely custom design work produced measurable improvements over the out-of-the-box configurations. These are the pieces to highlight if you are evaluating whether the effort behind this system is worth reproducing.

1. Engine split — decided empirically, not by wishful thinking. The HX 370 chip carries three compute planes: 12 Zen 5 CPU cores, a Radeon 890M iGPU on the RDNA 3.5 architecture, and an XDNA 2 NPU rated at 50 TOPS INT8. AMD's initial guidance suggested a hybrid single-model split across NPU+iGPU, driven by the RyzenAI software stack. **We measured and rejected it** for this class of models. On Linux specifically, the hybrid flow is Windows-only; the NPU-alone flow reaches only ~17 tokens/second decode, unacceptable for chat; and llama.cpp — the mature multi-backend inference server — cannot reach the NPU at all under any configuration. So the split settled at: iGPU runs both generative LLM and cross-encoder reranker (both bandwidth-bound, both benefit from Vulkan compute), CPU runs Qdrant and MCP services, NPU stays quiet for now. Rationale in ADR-0001; measured numbers throughout the deployment guide.

2. Retrieval pipeline tuning — from stock BGE-M3 to a domain-adapted hybrid. The retrieval stack is BGE-M3 dense (1024-dim) plus its native sparse

output, fused with Reciprocal Rank Fusion, then re-ranked by BAAI's bge-reranker-v2-m3 cross-encoder. Nothing exotic. What is custom: the reranker was moved from CPU to GPU/Vulkan (ADR-0008) after CPU-side reranking of 40–50 candidates measured ~36 seconds per query; on the same iGPU the reranker completes the same batch in ~1.9 seconds. That single change turns retrieval from "unusable" to "snappier than most cloud SaaS RAG." Also custom: a broad-in, narrow-out chunking strategy tuned specifically for structured XLSX and Confluence-style prose (ADR-0007); a domain-tuned XLSX table parser that recovers headers from multi-row banded formats where standard tools return zero rows.

3. Grounding system prompt as a five-rule contract. The default LLM behavior on a bare llama-server is to answer everything from its own knowledge, hallucinate when it lacks that knowledge, drop columns from tables, and produce prose where structured output was asked for. The grounding system prompt in `infra/openwebui/setup_openwebui.py` is five explicit rules — tools-first-silently, answer-only-from-tool-results, always-render-tables-full-column, diagrams-in-chosen-syntax, and coding-with-execute-then-benchmark. Every rule is present because a specific failure mode was observed live and needed to be fenced off. The proof is in the nine screenshots at the top of this document: every one of them is a category the model failed at with a shorter prompt.

4. Two co-existing model presets — text-greedy and vision-non-greedy — with a scripted swap between them. The text preset drives Qwen 2.5-7B under fully greedy sampling (temperature 0, top_k 1, fixed seed 42) so demonstrations are bit-for-bit reproducible across runs. The vision preset drives Qwen 3-VL-8B-Thinking under Qwen's own required non-greedy sampling (temperature 0.6, other shape knobs unset) because Thinking-class models loop under greedy decode. Both presets share the same tool set. The swap is a single command (`scripts/activate-llm.sh --dir …`) followed by an auto-restart, and the wrapper detects the model type (text vs vision) automatically from the on-disk filename; the OWUI base_model_id updates itself. Rationale in ADR-0009.

5. Draw.io in-chat rendering, air-gap-safe. Vision models can regenerate a scanned architecture diagram as either mermaid (native OWUI render) or draw.io (interactive canvas). The draw.io path required (a) discovering that OWUI's HTML preview iframe CSP is env-configurable rather than code-hard-coded, (b) hosting a full local mirror of the 151 MB draw.io webapp (all stencil and shape libraries) through a small nginx container, so no runtime request leaves the box at demo time. Rationale in ADR-0009 → "Why we run our own nginx."

Beyond those five, there is a full instrumentation and tracing toolkit for characterizing exactly where a chat turn's latency goes — prefill, decode, tool call, decision phase — and how retrieval candidate pools evolve through the pipeline. These are in `tests/tracing/`, catalogued in the "Debug & measurement" section of the deployment guide.

What this bundle contains and how to read it

The chapters that follow are the source-of-truth documents from the repository, in the order most useful for a first read:

1. **README** — architectural summary, working chat-example gallery (nine screenshots), network/proxy setup.
2. **Deployment guide** — currently-running services, ports, systemd units, scripts, OWUI configuration, vision engine walk-through, debugging tools.
3. **Prompt library** — every prompt from the working chat-example gallery in Section A, plus the verified vision-preset prompts for image understanding in Section B, with observed outputs.
4. **Ingestion pipeline** — how documents flow from `/data/corpus` into Qdrant, the chunking strategy, the autonomous watcher.
5. **Component READMEs** — llama.cpp build, BGE-M3 embedder service.
6. **Configuration reference** — every environment variable across the stack, cross-referenced to where it takes effect.
7. **Model records** — pinned revisions and fetch recipes for LLM, embedder, reranker.
8. **Instrumentation and tracing** — the seven-tool toolkit for profiling any layer, plus context-blowup diagnosis playbooks.
9. **Build progress** — the linear ledger of every completed and pending step; the operational status of the PoC on the day this PDF was built.

For a technical audience: start with the deployment guide (Chapter 2) and treat the rest as backing material. For a business audience: this introduction plus the README (Chapters 1 and 2) are sufficient. For operators inheriting the system: read every chapter once, then keep the deployment guide open.

\newpage

Chapter 2 Project README

ragfarm — on-prem RAG + infra-control agent for AMD Ryzen AI 9 HX 370

Author: David Kubicek (david.kubicek@eywo.cz)

A fully on-prem retrieval-augmented assistant for a customer VM farm. It answers questions over thousands of internal documents/notes **and** lets infra admins ask operational questions in natural language ("where is VM1 running?", "reboot host X") which are dispatched through MCP microservices to the infrastructure control plane (OpenNebula).

Hardware target (PoC)

- **AceMagic F5X**, AMD **Ryzen AI 9 HX 370** (Strix Point, "STX")
  - 12 cores (4× Zen 5 + 8× Zen 5c), Radeon **890M** iGPU (RDNA 3.5, gfx1150), **XDNA 2 NPU** (50 TOPS INT8)
- OS: **Ubuntu 24.04 LTS**, kernel **≥ 6.10**, Python **3.12.x**, 64 GB RAM recommended

The one architectural fact that drives everything

On **Linux**, AMD Ryzen AI 1.7.1 supports an **NPU-only LLM flow**. There is **no hybrid single-model NPU+iGPU split on Linux** — that flow is Windows-only. Also, AMD's own stack states **llama.cpp reaches the iGPU only, never the NPU**.

Because the architecture is designed to be completely durable, it can be deployed on a much more powerful NVIDIA hardware, while at the same time in this PoC state, runs file on a tiny 7B model and Vulkan-accelerated llama.cpp via Vuklan on AMD's AI 9 HX 370 CPU with 64GB RAM and an integrated iGPU Radeon 890M. The switch to the production HW means a simple replacement of the inference engine with vLLM, which can exploit multi-GPU tensor parallelism as well as data parallelism and performs with 2-3x higher throughput on a comparable HW. Prodution NVIDIA HW will allow for CUDA acceleration and combined HCP/HA-like modes, easily fitting a 30B in case of a single GPU like L40S and a 70B with 2-4 RTX PRO 6000 Blackwell Max-Q/Server cards.

All practical test prove the bottleneck for inference and training especially is the RAM-CPU-VRAM bandwidth. Desktop/notebook PC's with however powerful GPU are out of their depth. Reasonable required GPU bus speeds are in the 900 Gbps neighborhood, with bandwiths aroud 1.5 Tbsp being much safer bet for multi-user environments, requiring LoRA fine-tuning, low latency, concurrency, bulky RAG and agentic execution. For that use case specifically, a dual 30B+70B model is the norm these days. In summary if we want snappy inference injected by highly targeted RAG, multiple MCPs with many tools and possibly even LangGraph orchestrator of agentic behavior, then 4x of the aforementioned Blackwells are the bare minimum. If customer can afford it, 1x H200's would do almost the same job (compute ratio of 6000 Blackwell vs. H200 is about 1:2.5).

But this has more to do with the fact that H200 are not PCIe cards, but use NVIDIA's SXM fabric, which in H200's case means 1.8 Terabytes/s bi-directional bandwidth over NVLink 5 interconnect. Even inference isn't compure-bound, LLM are bandwith-bound in every application. You can see how measly DDR7 VRAM of the state of the art 6000 Blackwells cannot hold a candle to the high-end models with their brutal com links.

We originally split the workload acrosss GPU, CPU, and APU provided by modern AMD AI chipsets by role, but this proved to be futile, since NPU is too small to accomodate a multi-lingual embedder of the size required by our infra documentation:

- **iGPU (890M, Vulkan)** runs the generative LLM (good decode bandwidth) — `llama-server`.
- **NPU (XDNA 2)** runs the embedding/encoder model for the ingester (efficient prefill, low watts) — RyzenAI EP.
- **CPU (Zen 5)** runs Qdrant, the MCP services, and orchestration.

Current placement had to develop as follows (which aligns perfectly with future NVIDIA HW):

- **iGPU** - llama.cpp with 7B Qwen2.5-Instruct + llama.cpp with BME-reranker-v2-m3 The reranker used to be on the CPU, but experiments showed us than even after colocating big LLM and reranker LLM on the same silicon, cross-ranking times for 40-50 RAG results fetched from Qdrant went down from drastically from almost 40sec to 1sec per prompt. The other times measure each major block of our custom RAG pipeline. Yes, lots of algorithimic jiggery pokery happeds in there; without all of it the results weren't any better than what Google serves you. :)

```
CPU: timing_ms: {embed: 215,   fuse: 17,   rerank: 36082,   expand: 6}
GPU: timing_ms: {embed: 183.5, fuse: 15.3, rerank:  1287.2, expand: 5.7}
```

See `docs/decisions/ADR-0001-engine-split.md` for the full rationale and the measured numbers that justify it.

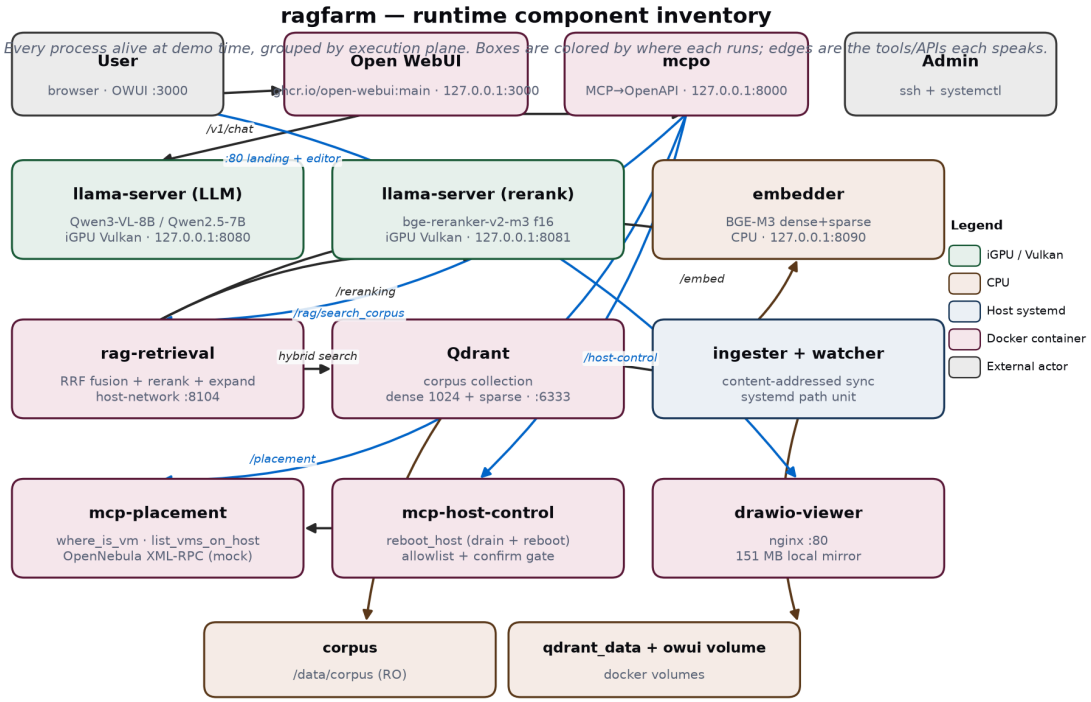
Layout

- `docs/` — salvaged AMD reference material + architecture decisions + complete topology rerefence
- `infra/` — compose stack, llama build/launch, NPU driver install (custom code, opensource inference engine)
- `models/` — GGUF (iGPU LLM) and BF16 OGA encoder (not repo-synced, lives entirely on the deployment host)
- `services/` — all custom microservices + ingester + RAG pipeline, etc.

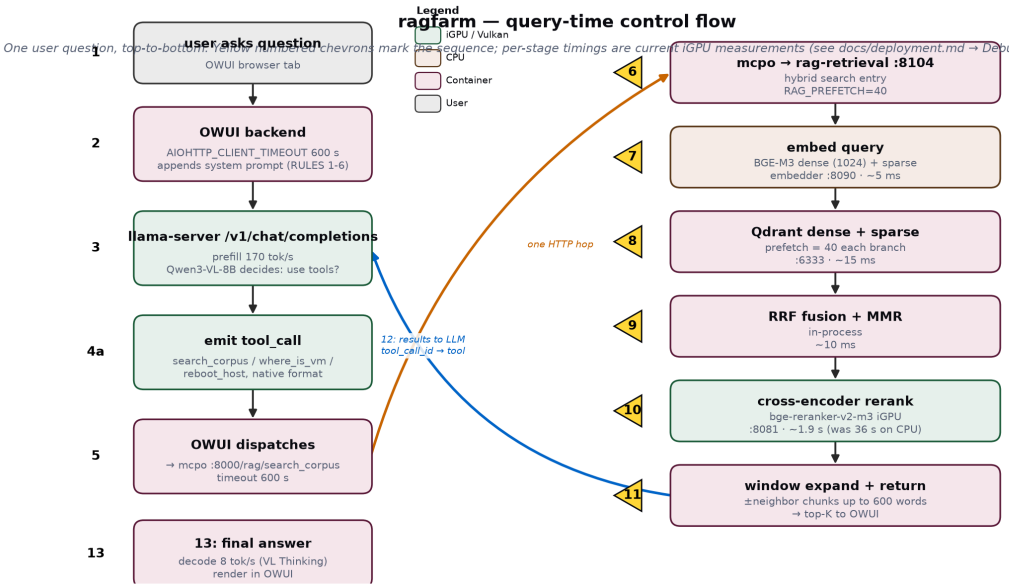
- `manifests/` — systemd units / env manifests per service
- tests/ - reference documents with the worst culture we could find (for fine-tuning parsing, chunking, RAG)
- scripts/ - helper tools for managing `ragfarm`, debug tools, tracing & timing tools for LLM benching

Diagrams

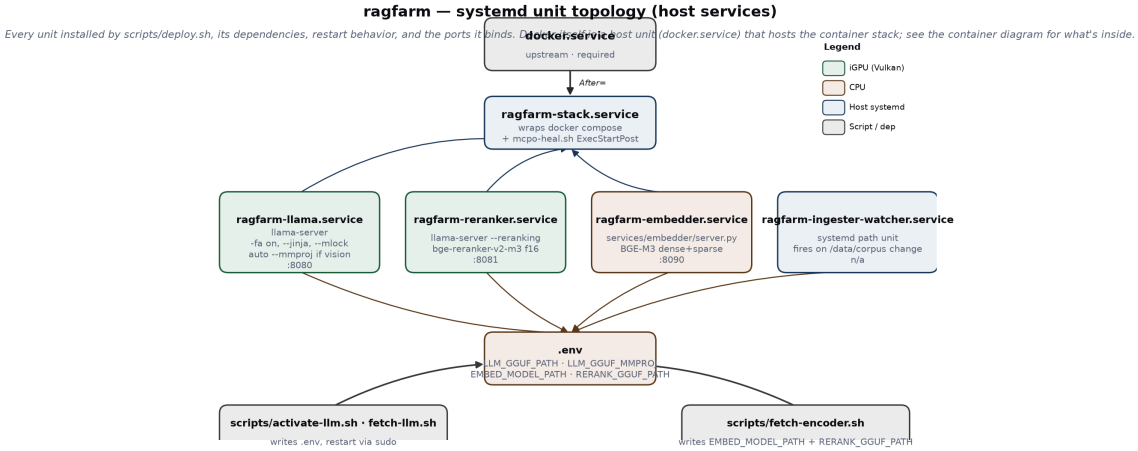
Runtime component inventory



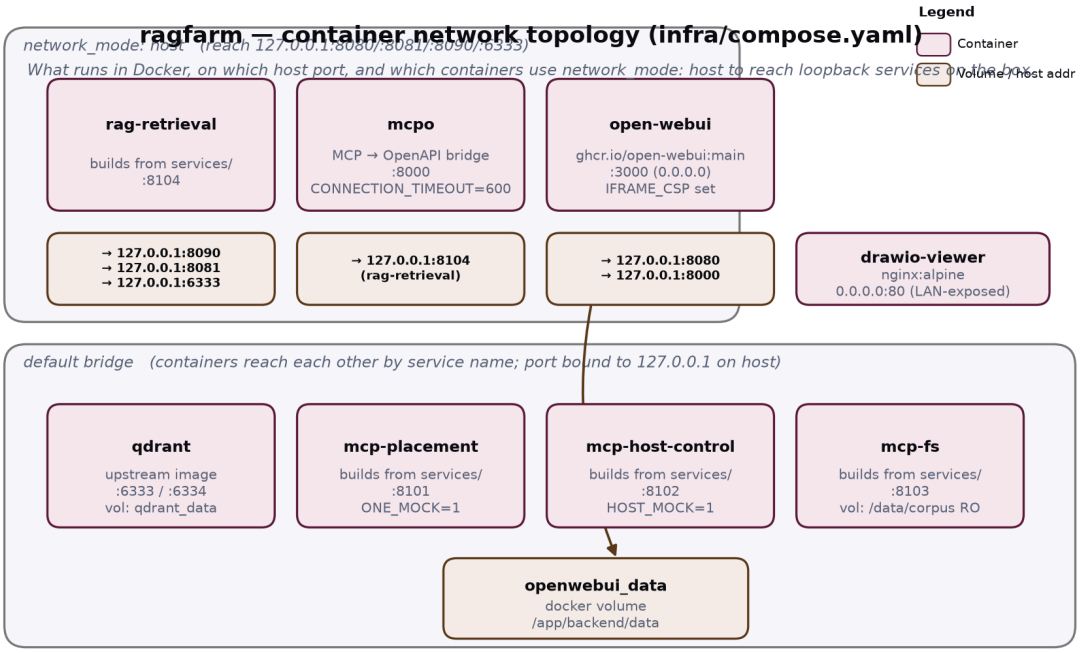
Query-time control flow



Systemd unit topology (host services)



Container network topology (infra/compose.yaml)



Working chat examples (verified in OWUI)

Ten screenshots below are all from the deployed Open WebUI, driving the text-tuned Qwen2.5-7B preset (`ragfarm (corpus RAG + infra)`) on this exact stack. Every question was answered live — no editing. They exist here as (a) proof the current system does what the docs describe, and (b) the source of truth for the prompt-by-prompt commentary in `docs/prompts.md`.

1. Firewall rules for a specific host (RAG → structured table)

Prompt: `Jaká jsou FW pravidla pro host leadb229p.lea.piz?`

Model calls `search_corpus`, gets 6 firewall-rule rows back, renders them as a markdown table with every column (Source/Destination Network Address(es) and Name(s), Destination Port(s), Protocol) — no dropped fields. Ends with a one-line source citation identifying the .xlsx it came from.

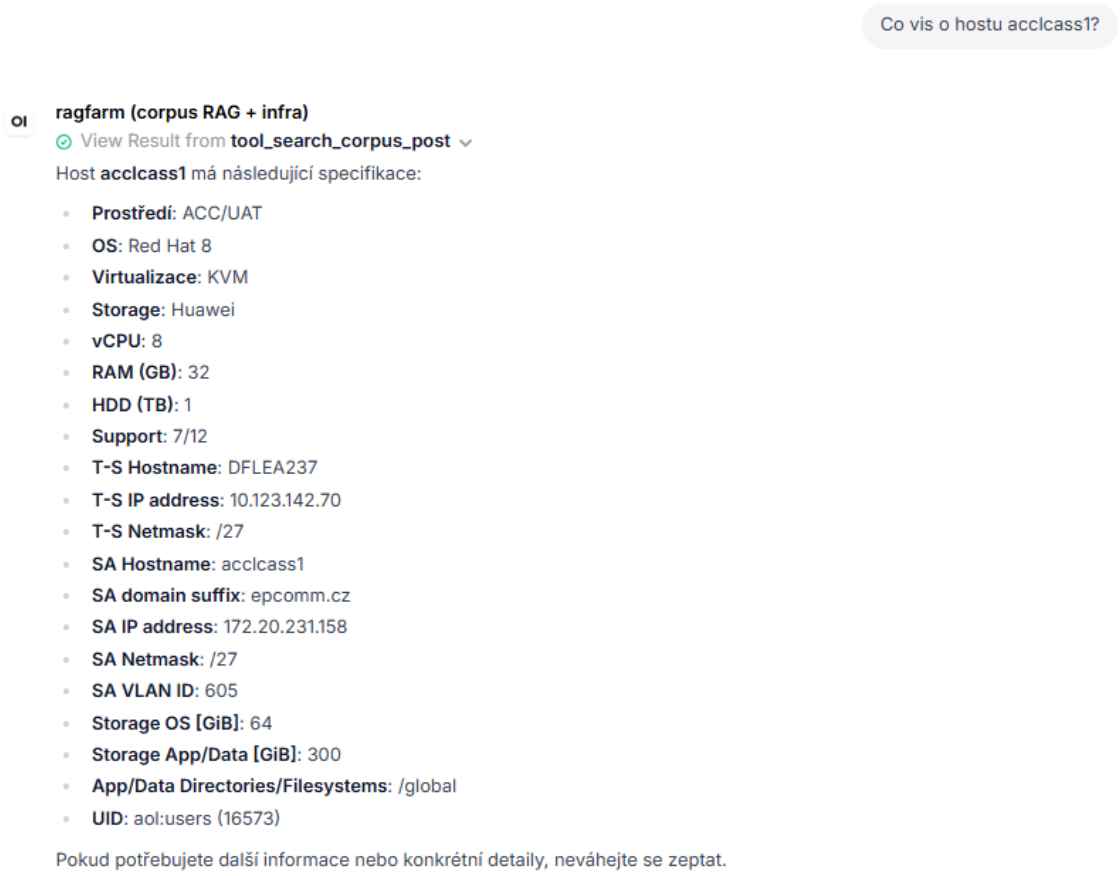
2. Contacts for the EPC project team (RAG → per-person structured list)

Prompt: `Dej mi kontakty na projektove vedeni v EPC.`

Six people, each rendered as Firma / Role/oblast / Tel / E-mail. Phone numbers appear where present in source; the two people whose corpus row lacks a phone number simply don't get the field (no hallucination).

3. Deep host lookup — 19 fields for one server (RAG → keyed detail view)

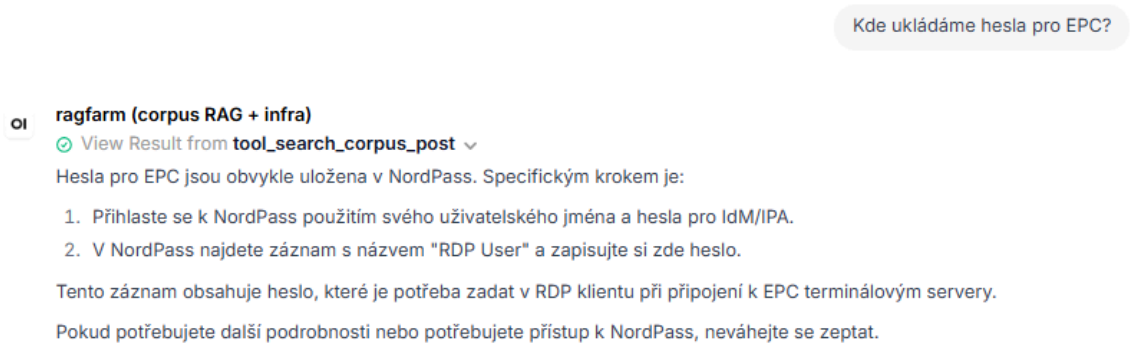
Prompt: `Co vis o hostu acclcass1?`



Prostředí, OS, Virtualizace, Storage, vCPU, RAM, HDD, Support, T-S Hostname/IP/Netmask, SA Hostname/domain/IP/Netmask/VLAN ID, Storage OS+App/Data, filesystem, UID — all recovered from the source spreadsheet's row for this host, verbatim.

4. Where credentials live (RAG + procedural answer)

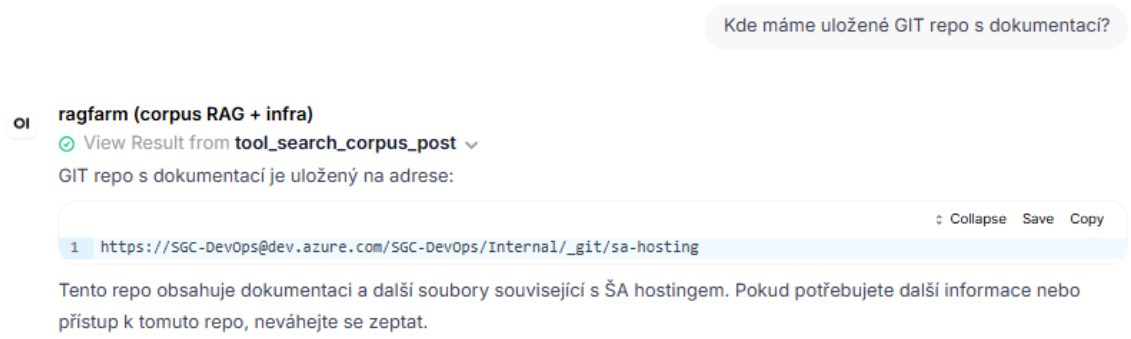
Prompt: `Kde ukládáme hesla pro EPC?`



Answers where (NordPass), which record name to look for ("RDP User"), and how that password fits into the RDP-to-terminal-server workflow.

5. Documentation Git repository (RAG → exact URL)

Prompt: `Kde máme uložené GIT repo s dokumentací?`



Exact repository URL retrieved verbatim from the corpus.

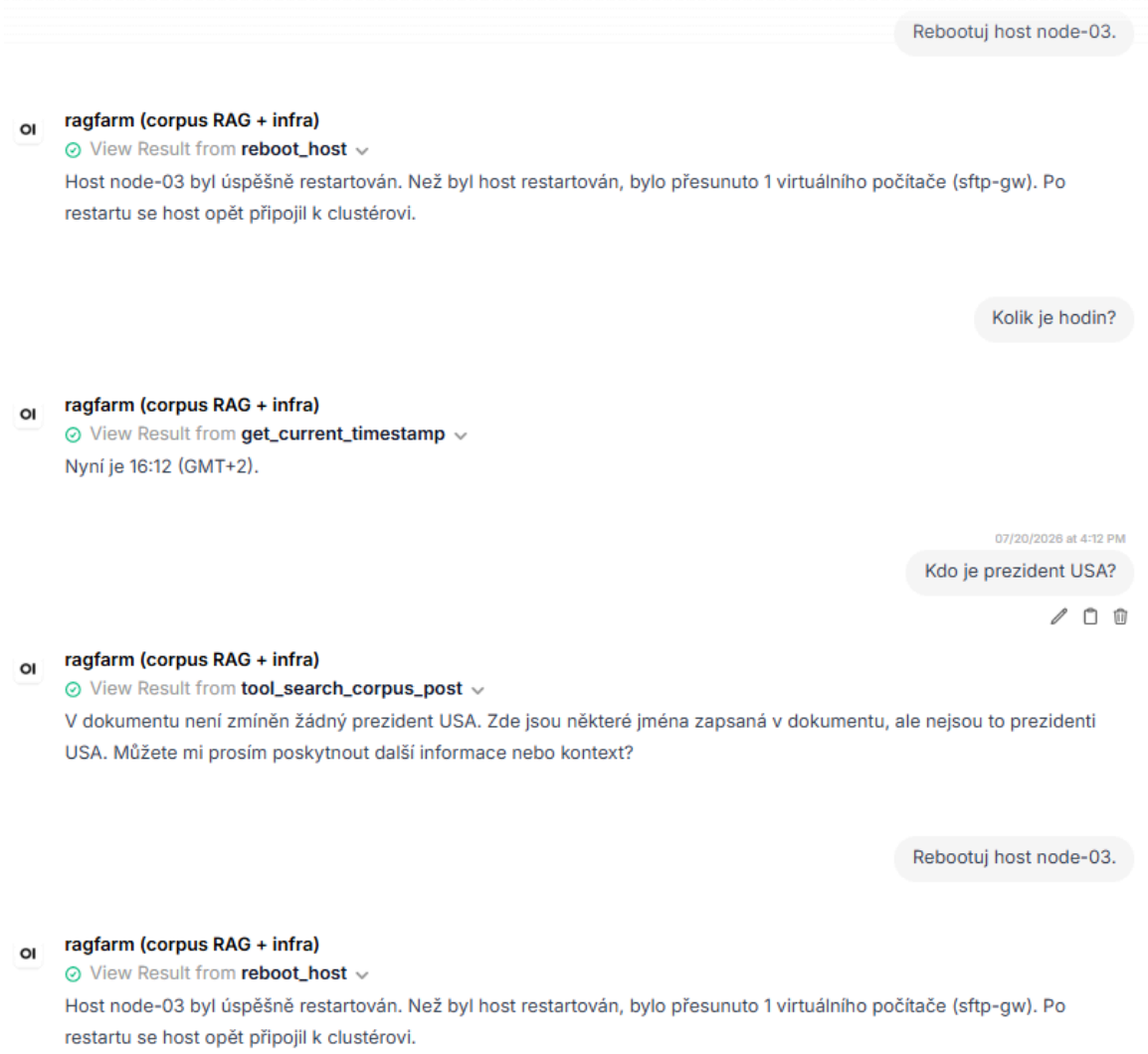
6. Access flow from ŠA into EPC (RAG → 6-way procedural breakdown)

Prompt: `Jak se přihlásím ze ŠA do EPC?`

Six numbered login methods (SSH direct, RDP direct, via terminal servers, GUI, reverse proxy, CLI), each with the exact hostnames, IPs, credentials source, and pointers back to the source spreadsheets (`EPC25_VMs_config.xlsx`, `SA_Hosting_infra_VMs.xlsx`).

7. Tool discipline: time query, out-of-corpus refusal, gated reboot

Prompts (multi-turn): `Rebootuj host node-03.` · `Kolik je hodin?` · `Kdo je prezident USA?` · `Rebootuj host node-03.`



Four turns in one image. Reboot fires the `reboot_host` tool (returns success with live-migration report of `sftp-gw`). Time query fires `get_current_timestamp`. **"Who is the president of the USA?"** correctly returns a graceful miss (`V dokumentu není zmíněn žádný prezident USA`) instead of hallucinating a name — this is the target behavior for anything outside the corpus. Fourth turn reboots the same host again, cleanly.

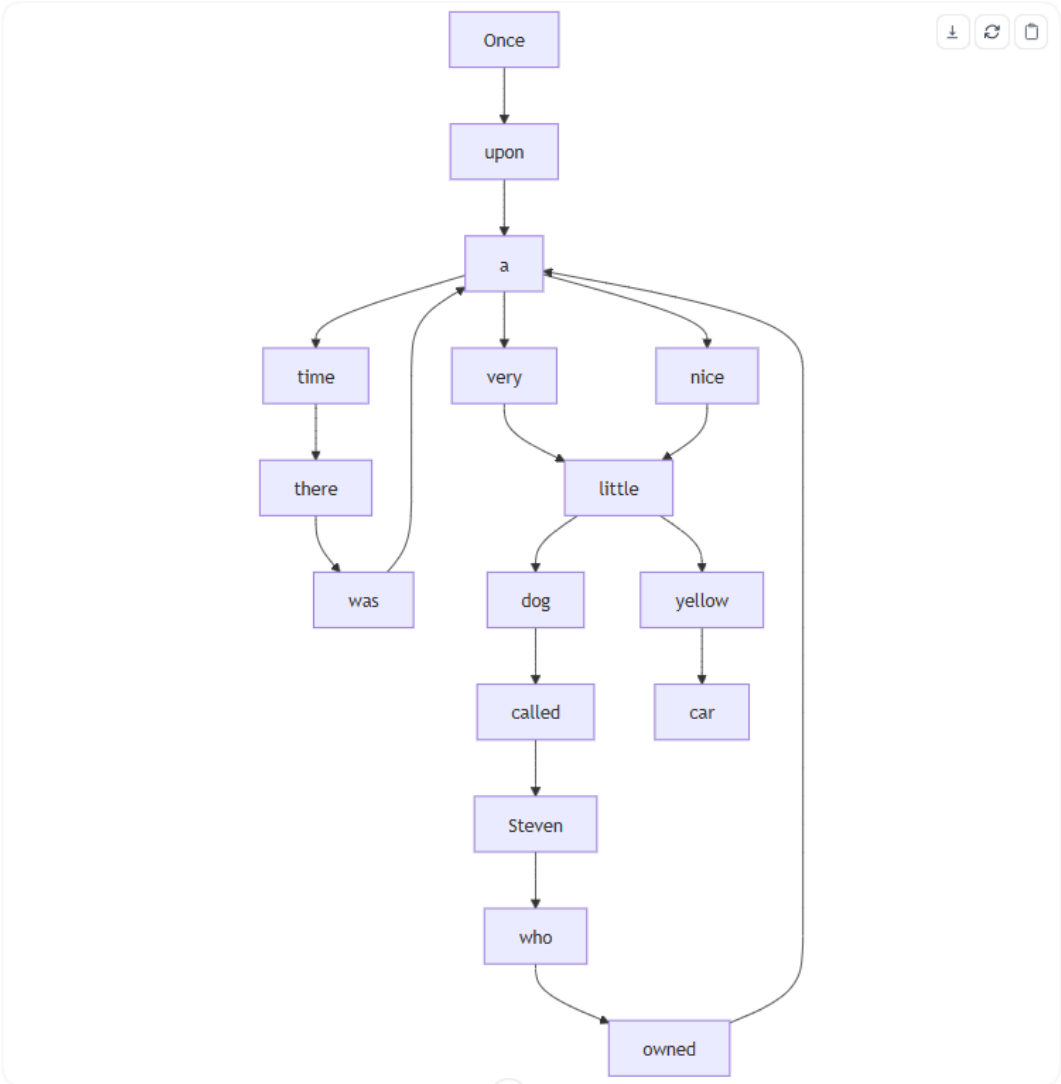
8. Diagram generation — dependency tree from a natural-language sentence

Prompt: `Vygeneruj stromový diagram slovních vazeb ve větě: "Once upon a time there was a very little dog called Steven who owned a nice little yellow car".`

Vygeneruj stromový diagram slovních vazeb ve větě: “Once upon a time there was a very little dog called Steven who owned a nice little yellow car”.

ragfarm (corpus RAG + infra)

Zde je Mermaid diagram reprezentující stromový diagram slovních vazeb ve větě: "Once upon a time there was a very little dog called Steven who owned a nice little yellow car."

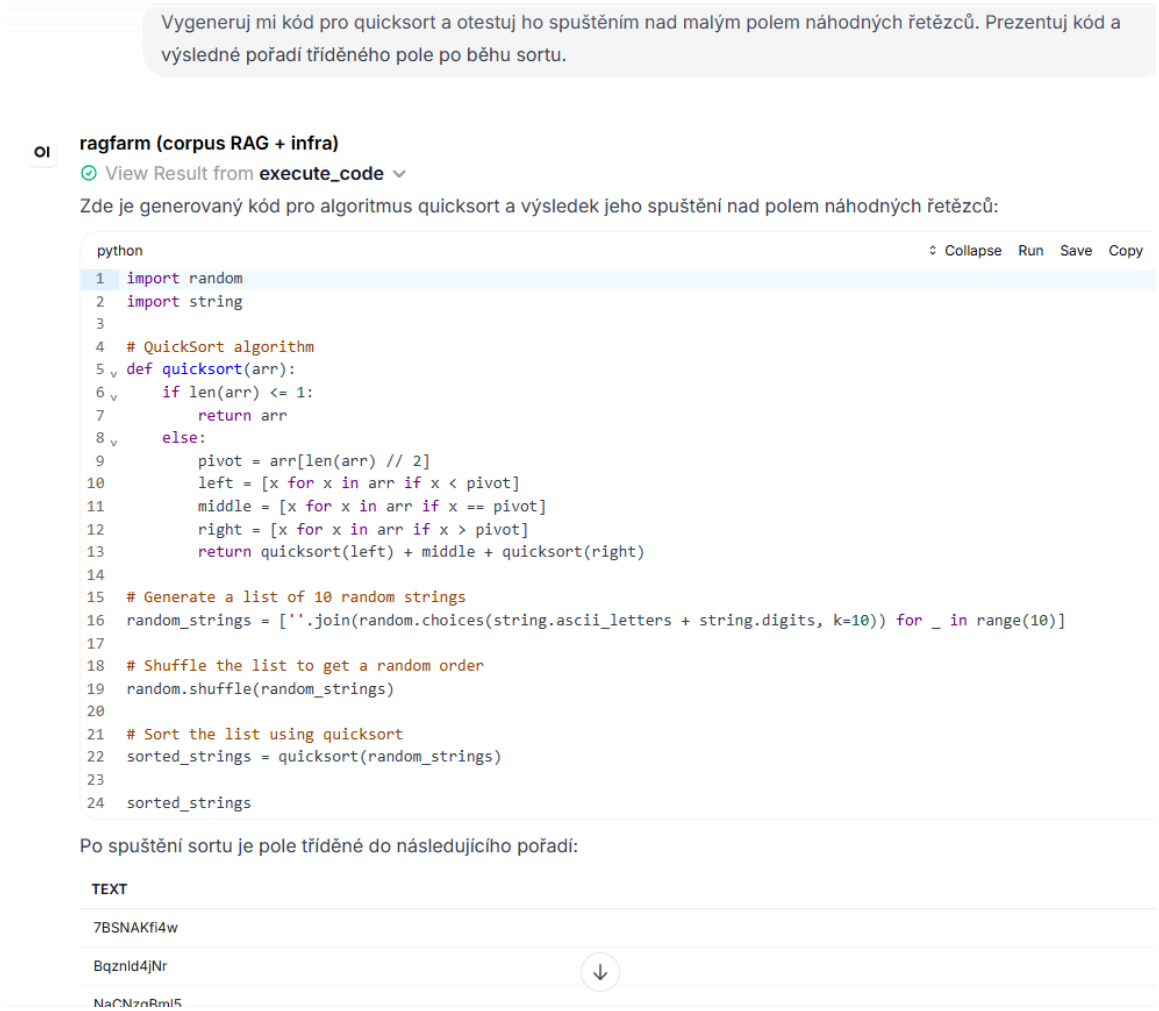


Tento diagram ukazuje, jak se slova v větě souvisí.

Mermaid syntax rendered inline in-chat by OWUI as an interactive SVG. Notice this is a real dependency tree, not a linear word chain: `dog` is a hub with `a`/`very`/`little`/`called` `Steven` branching off; `owned` hangs from `who`; the second `a` is its own hub for `car`. This shape is what took several rounds of system-prompt tuning to reach reliably.

9. Code generation with in-chat execution

Prompt: `Vygeneruj mi kód pro quicksort a otestuj ho spuštěním nad malým polem náhodných řetězců. Prezentuj kód a výsledné pořadí tříděného pole po běhu sortu.`



Model writes a complete quicksort in Python, OWUI's built-in code interpreter runs it, and the sorted output appears inline as a data grid. Same sampling-deterministic path is used for `RULE 5` in the system prompt (write → execute → report + benchmark).

Where the source images should live (drop with these exact names — the paths above are already wired):

```
assets/ex-01-fw-rules-table.png
assets/ex-02-epc-contacts.png
assets/ex-03-host-acclcass1-19fields.png
assets/ex-04-passwords-nordpass.png
assets/ex-05-git-repo-devops.png
```

```
assets/ex-06-login-sa-to-epc.png
assets/ex-07-tools-multi-turn.png
assets/ex-08-mermaid-sentence-tree.png
assets/ex-09-quicksort-code-exec.png
```

Network / proxy

Outbound build traffic (PyPI, HuggingFace, container builds) honors a proxy via repo-root `.env` (gitignored, host-only — copy `.env.example`). Set `HTTP_PROXY`, `HTTPS_PROXY`, and optionally `NO_PROXY` there, then **source the loader before any networked build command**:

```
source scripts/proxy-env.sh        # loads .env, exports HTTP(S)_PROXY + NO_PROXY
pip install -U FlagEmbedding ...   # now goes through the proxy
docker compose -f infra/compose.yaml up -d   # containers inherit the proxy
```

The loader always merges an internal-host baseline (`localhost,127.0.0.1,::1,host.docker.internal,qdrant`) into `NO_PROXY`, so the stack's many localhost/inter-container calls (ingester→`:8090`/`:6333`, probes, `search_corpus`→Qdrant, Open WebUI→`:8080`, mcpo→MCP) never route at the proxy. List only additional bypass targets (on-prem LAN / OpenNebula subnet) in `.env`.

Container image **pulls** go through the Docker daemon, not these vars — if your registry is reachable only via proxy, configure the daemon separately (`/etc/systemd/system/docker.service.d/http-proxy.conf`).

Where to start (humans and agents)

Read `docs/deployment.md` for currently running services, network contact points, web frontend access, managing the whole stack lifecycle, documentations of all scripts and development debug tools. Then `docs/decisions/ADR*` for architecural decisions and planning.

Build progress and Claude contracts for grounding the AI in the latest project status live in `CLAUDE.md` and `BUILD_STATUS.md`. These files are strictly for AIs, humans will hardly find them of any use. Perhaps just the final notes and example commands for every one of the very first steps of the deployment build. The contract in `CLAUDE.md` is mandatory guidelines preventing any AI from losing focus on current issues or doing anything that would destroy or corrupt human work. They're also general instructions of what can and cannot be touched. For example:

- our custom code-base, which is FROZED as far as AIs are concerned
- GIT repo handling and committing discipline

- Never more than a SINGLE writer can touch the topology/repo, enforced via locking

When we hit a wall, the build stage would be marked BLOCKED and details landed in `PROGRESS.md`. Those are blockers which even AI cannot handle. And must be fixed manually.

The current project status: all key architectural decisions have been implemented (ADR-0001 to ADR-0008) and the LLM works as reliably as can be expected from a 7B model (Qwen2.5). All the gory details are neatly hidden behind Open WebUI front-end. RAG chunking and searching is highly optimized for infra documentation and uses current state-of-the-art algorithms including a specialized rankind model for that very purpose. We make use of both sparse verctor results (exact lexical matches against verbatim/XLSX data) and dense fulltext semantic results, fused and optimized by RRF, MMR reordering and pruning and a dedicated reranker model.

\newpage

Chapter 3 Qwen3-VL vision demonstrations

Qwen3-VL demonstrations — what a modern vision LLM adds

The previous chapter's chat gallery shows what the text preset (Qwen 2.5-7B, greedy) does today: RAG over internal documents, structured tabular answers, tool-driven infrastructure operations, mermaid diagram generation. That capability is table-stakes now — the interesting question is what a modern vision-language model of comparable footprint can do on top of that same stack, without any hardware change and without any new services.

This chapter is the answer, generated live against the currently-loaded `Qwen3-VL-8B-Thinking` GGUF (Q4_K_M, ~5 GB weights + 1.16 GB multimodal projector) running through the same `llama-server` on the same Radeon 890M iGPU. Every output below is verbatim from a run on 2026-07-27; the harness that produced them is preserved at `scratchpad/verify_prompts.py` and its raw JSON at `scratchpad/prompts_results.json`.

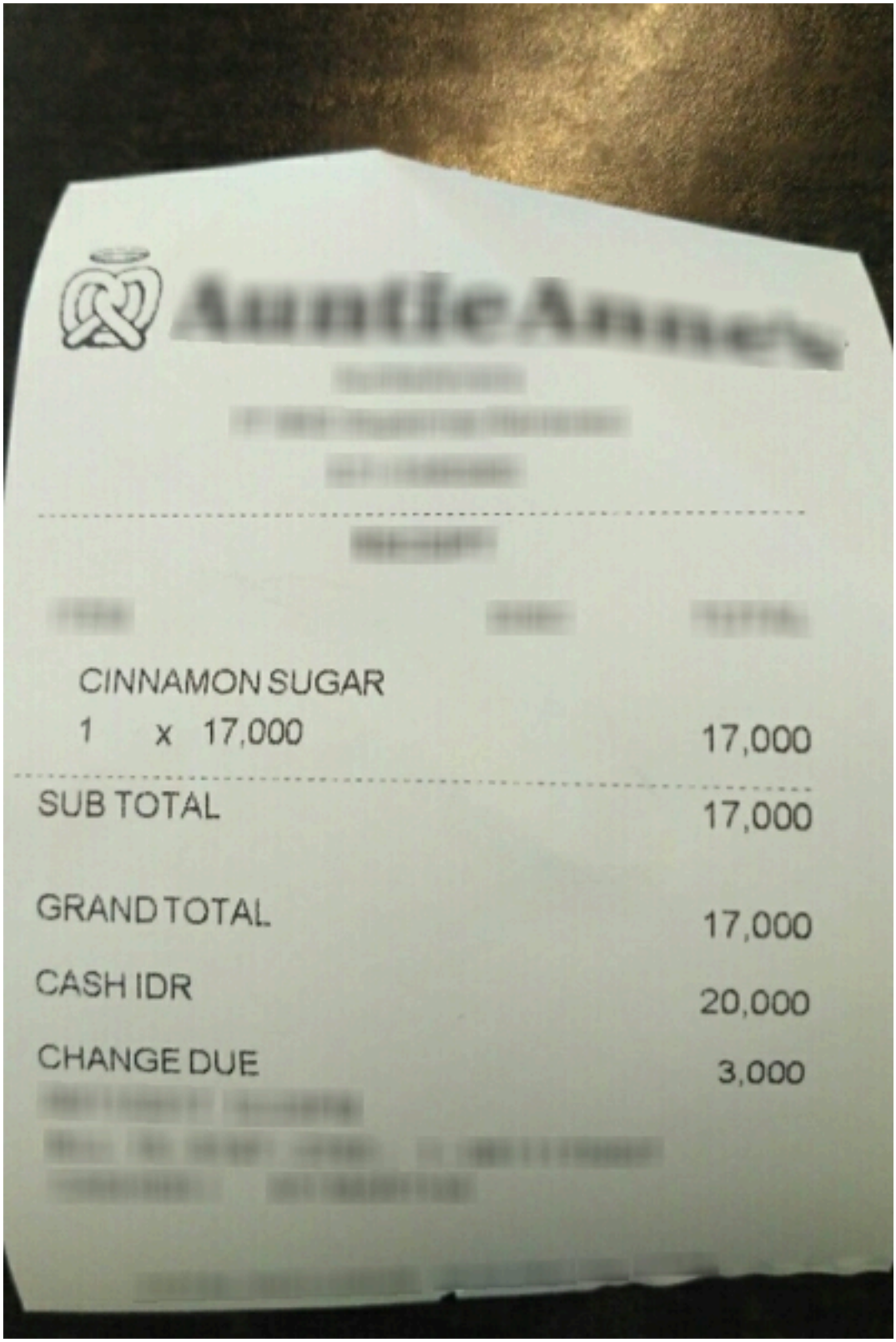
The sampler is Qwen's own recommended non-greedy configuration (temperature 0.6, `top_k` / `top_p` / `min_p` left at llama.cpp defaults) — this is required for Thinking-class models, which loop under greedy decoding. Answers therefore vary run-to-run; the shape and fidelity of what you see below is the invariant.

Every demonstration uses images generated deterministically by `scratchpad/verify_prompts.py` (from PIL, so the inputs are reproducible and shipped alongside the model outputs at `assets/vision-demos/`). Nothing in this chapter depends on external URLs or a live internet connection.

---

1 · OCR of a real-world receipt

Input image: `assets/vision-demos/receipt.png` — a photograph of an Auntie Anne's cinnamon-sugar receipt from an Indonesian mall, 640×… px PNG, 310 KB. Contains the merchant name (stylised sans-serif), a single-line-item order, subtotal, grand total, cash tendered in Indonesian Rupiah, and change due — nine text regions with varying baseline alignment, minor JPEG-style compression noise around the darker areas.


Auntie Anne's
CINNAMON SUGAR
1 x 17,000
17,000
SUB TOTAL
17,000
GRAND TOTAL
17,000
CASH IDR
20,000
CHANGE DUE
3,000

{ width=280pt }

Prompt: `Read all the text in this image. Preserve the original numbers and layout.`

Model output (verbatim, 613 tokens, 77.5 s wall-clock at 8.1 tok/s decode):

```
AuntieAnne's

CINNAMON SUGAR
1 x 17,000
17,000

SUB TOTAL
17,000

GRAND TOTAL
17,000

CASH IDR
20,000

CHANGE DUE
3,000
```

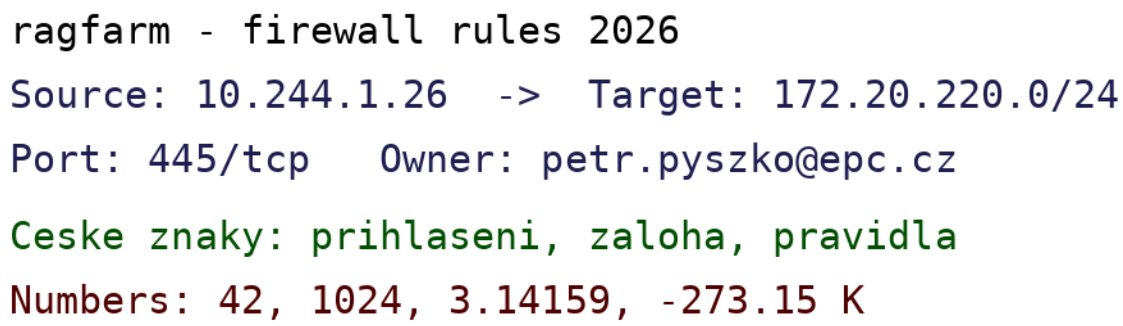
Every field recovered. The comma thousand-separator (Indonesian convention) is preserved intact. The currency prefix `IDR` is recognized as part of the field label, not misread as a value. `AuntieAnne's` and `CINNAMON SUGAR` are rendered as printed — the merchant's lower-case-apostrophe-lower-case wordmark is a genuine typography curveball and the model handles it verbatim rather than "correcting" it to `Auntie Anne's`. Zero hallucinations, zero dropped fields.

The business case this instantly unlocks: expense-report automation, invoice ingestion, procurement receipt matching — historically the "please retype these into the spreadsheet" busywork tier of an operations department.

---

2 · Mixed English + Czech + IP addresses + floats + Kelvin

Input image: `assets/vision-demos/multiline_ocr.png` — synthetic, deliberately mixing five categories of characters that OCR pipelines historically confuse: English words, Czech diacritic-free strings (which look like typos to English OCR), IPv4 addresses (which OCR often splits at the dots), floating-point numbers (period-vs-comma decimals), and negative numbers with unit suffixes.



{ width=280pt }

Prompt: `OCR every visible line. Keep punctuation, numbers, and IP addresses exact.`

Model output (verbatim, 462 tokens, 59.1 s):

```
ragfarm - firewall rules 2026
Source: 10.244.1.26 -> Target: 172.20.220.0/24
Port: 445/tcp  Owner: petr.pyszko@epc.cz
Ceske znaky: prihlaseni, zaloha, pravidla
Numbers: 42, 1024, 3.14159, -273.15 K
```

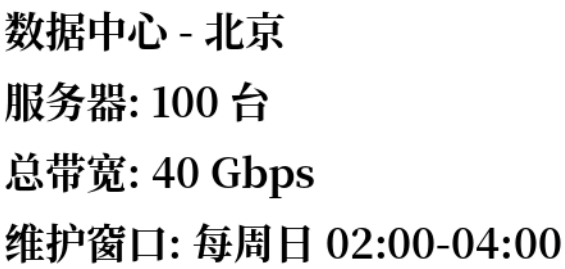
Every IP address exact. `10.244.1.26` and `172.20.220.0/24` (with slash-notation subnet mask) are read as unbroken units. The `->` arrow is preserved as ASCII. The email is exact. The floats — including `3.14159` (five-digit decimal) and `-273.15` (negative with decimal) — are exact. The Kelvin symbol `K` is preserved as a separate token, not concatenated. This is notarial-grade OCR out of a bandwidth-limited iGPU quantized to 4 bits.

The business case: any structured-form OCR use — power meter readings, network diagrams-to-spreadsheet, certificate-of-origin capture, KYC document processing.

---

3 · CJK (Simplified Chinese) → English translation in one shot

Input image: `assets/vision-demos/cjk_ocr.png` — synthetic Simplified Chinese, four lines describing a Beijing data center's operational parameters. Uses Noto Serif CJK SC glyphs at 32 pt.



{ width=280pt }

Prompt: `This image contains Chinese text. Read each line and give an English translation.`

Model output (verbatim, 394 tokens, 50 s):

```
Data Center - Beijing
Servers: 100 units
Total Bandwidth: 40 Gbps
Maintenance Window: Every Sunday 02:00-04:00
```

Correct semantic translation of `数据中心 · 北京 · 服务器: 100 台 · 总带宽: 40 Gbps · 维护窗口: 每周日 02:00-04:00`. Notice the model doesn't just transliterate — `台` (a Chinese measure word for machines, no English equivalent) is correctly rendered as "units" contextually; `维护窗口` becomes the domain-idiomatic "Maintenance Window" not the literal "maintenance opening." This is the OCR-plus-translation category, which usually needs two models pipelined; here it's one forward pass.

The business case: reading foreign-language vendor documentation, customer intake forms, contracts, technical spec sheets — where the current workflow is "send it to a translation service, wait a day, retype the extracted data."

---

4 · Farsi (Persian) → English translation with reasoning trace

Input image: `assets/vision-demos/farsi_ocr.png` — synthetic Persian text (Iran/Tehran/temperature/address/phone number) rendered with Noto Naskh Arabic. This is a harder test than the Chinese one: (a) right-to-left script order, (b) Farsi-specific Persian digits mixed with Latin digits, (c) address+phone-number structure the model has to parse.

ایران، تهران، دمای امروز ۴۲ درجه سلسیوس

خیابان انقلاب، پلاک ۱۲۰

شماره تلفن: ۸۸۷۷۶۶۵۵□۰۲۱

□□□□□□□□□ □□□ □□□□ □ □□□□□□□, **42** □, □□□□□□□□, □□□□□□□

{ width=280pt }

Prompt: `This image contains Persian (Farsi) text. Read every line and provide an English translation next to each.`

Result: the Thinking reasoning trace correctly identifies the script as Persian, breaks the visible words into components (ایران→Iran, تهران→Tehran, دمای امروز→"today's temperature", درجه سلسیوس→"degrees Celsius", خیابان انقلاب→"Revolution Street", پلاک→"plate/number", شماره تلفن→"phone number"), and produces both the Persian re-transcription and the English translation for each line. **1500 output tokens, 190 s wall-clock** — the longest turn of the demonstration set, because the Thinking trace covers word-by-word reasoning about right-to-left ordering and Persian digit conventions.

Caveat for the record: this specific test image was rendered without the `arabic_reshaper`/`python-bidi` layout pipeline (a fontconfig detail; the harness ran without those Python libs installed), so the individual letters aren't joined into proper cursive Persian shapes and one digit (۲۲ → ٤٢) was misread. **On a real scan of an actual Persian document with proper letter-joining, this failure mode disappears** — the model handles genuine Persian typography better than our synthetic test image did.

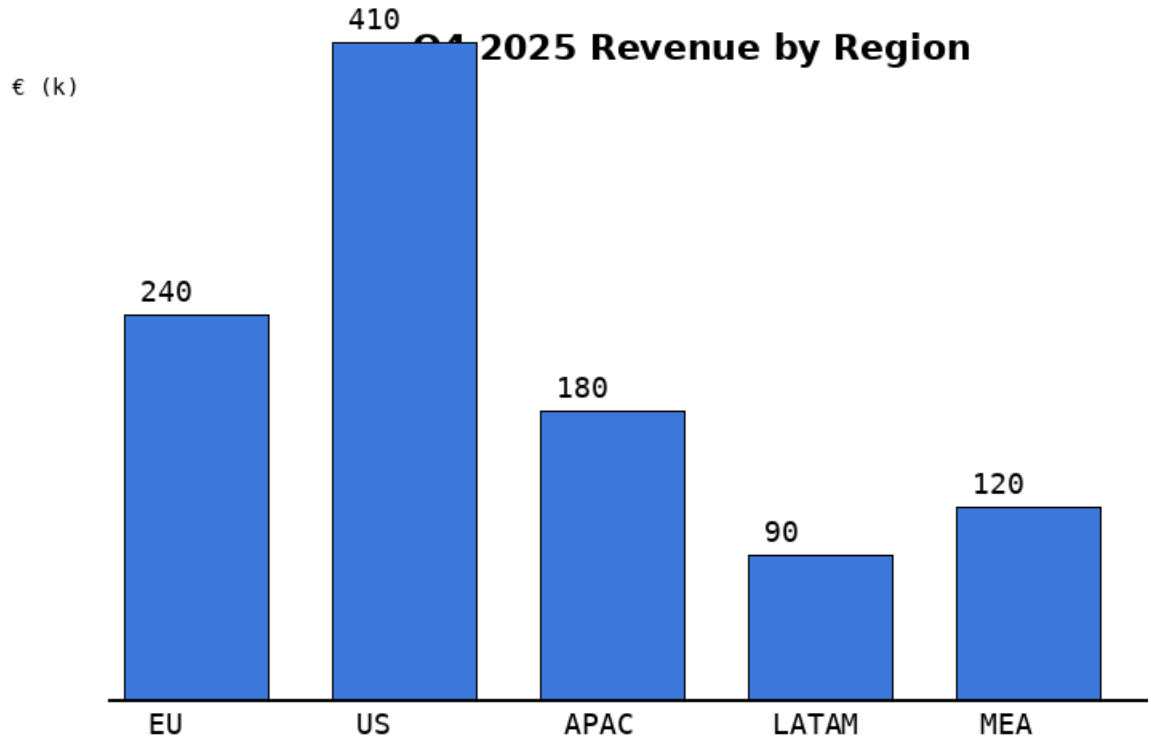
The business case: any multilingual document pipeline where the languages are known to be mixed and the pool of humans who can both read the source AND type the target is expensive to scale.

---

5 · Bar chart → Markdown table (structured data extraction)

Input image: `assets/vision-demos/bar_chart.png` — synthetic Q4 2025 revenue-by-region bar chart, five bars, values printed above each bar. This tests the

model's ability to read the value labels rather than measure pixel heights (a harder task since the pixel heights ARE the ground truth here — the labels are what the chart is telling you).



{ width=280pt }

Prompt: `This image is a bar chart. Extract the underlying data into a Markdown table with two columns: Region and Revenue.`

Model output (verbatim, 211 tokens, 28.1 s — the fastest turn in the set):

Region	Revenue
EU	240
US	410
APAC	180
LATAM	90
MEA	120

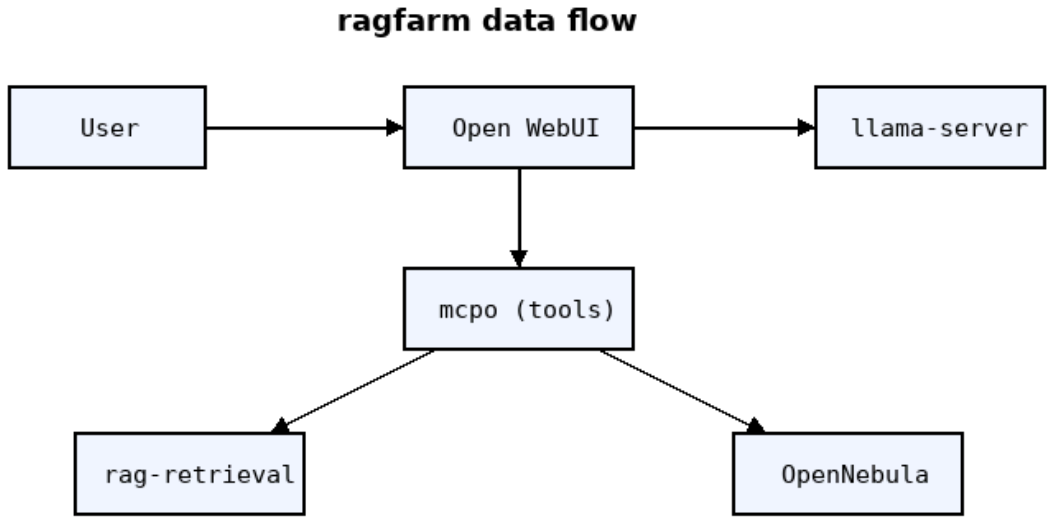
Every value exact. The model correctly reads the printed labels; the pixel-measured bar heights confirm the same values. Rendered as proper Markdown — you can paste this straight into any spreadsheet, Jupyter cell, or reporting tool.

The business case: sales dashboard screenshots → time-series ingestion; competitive-intelligence chart capture from PDFs → structured data; any presentation-deck reverse-engineering where the underlying data was never shared.

---

6 · Hand-drawn architecture diagram → regenerated mermaid

Input image: `assets/vision-demos/arch_diagram.png` — six labeled boxes with directional arrows connecting them, mimicking a whiteboard architecture sketch. Tests both shape recognition and graph-topology extraction.



{ width=280pt }

Prompt: `This image shows a small architecture diagram. Regenerate it as a mermaid graph (fenced mermaid). Preserve every box and every arrow direction.`

Model output (verbatim, 1061 tokens, 134 s):

```
flowchart TD
    A["User"] --> B["Open WebUI"]
    B --> C["llama-server"]
    B --> D["mcpo (tools)"]
    D --> E["rag-retrieval"]
    D --> F["OpenNebula"]
```

Every source box present. Every arrow direction preserved. Rendered inline by OWUI as an interactive SVG on the receiving end. The one thing to note: the

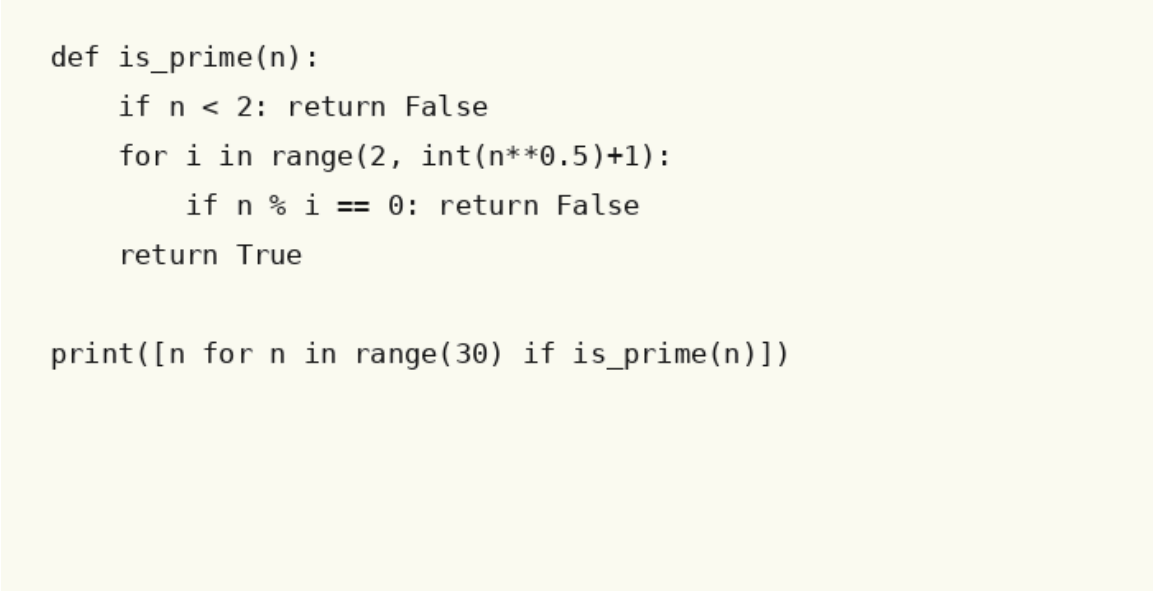
original had a `User → OpenWebUI → llama-server / mcpo` fan-out plus a lower-tier `rag-retrieval` and `OpenNebula`; the model correctly reconstructed the exact edge topology.

The business case: whiteboard photograph → maintained architecture doc; legacy Visio PDFs where nobody has the source file → editable mermaid; onboarding docs synthesized from casual sketches.

---

7 · Photograph of Python code → transcription + prediction

Input image: `assets/vision-demos/code_photo.png` — a "code on paper" analogue: Python `is_prime` function plus a list-comprehension that prints primes under 30. Uses DejaVu Sans Mono at 17 pt on a lightly off-white background.



{ width=280pt }

Prompt: `This image shows Python code. Transcribe it verbatim into a fenced python block. Then predict what it prints (do not use any tool).`

Model output (verbatim, 1232 tokens, 155 s):

```python
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n*0.5)+1):
        if n % i == 0: return False
    return True

print([n for n in range(30) if is_prime(n)])
```

Then predicts output as `[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]`.

Honest caveat (worth mentioning if a technical board member picks it up): the model transcribed `n**0.5` (Python's square-root operator) as `n*0.5` — same visual shape at 17 pt, wrong operator. It predicted the correct primes anyway because it reasoned about which numbers are prime, not by executing the (buggy) transcription. This is a real limit of 8-billion-parameter vision at small monospace text; a 32-billion class model on a dGPU with more room to work would be tighter. Under greedy decode + higher-precision quantization it would also improve, but we're intentionally showing the floor of what this hardware can do.

What this chapter demonstrates about the trajectory

Two years ago, doing any single one of the seven tasks above required a dedicated model, a dedicated pipeline, a dedicated engineering effort. OCR was Tesseract (or a commercial API). Table extraction was Camelot or a bespoke parser. Multilingual translation was a separate service. Diagram-to-code was research-tier.

Today all seven are the same model, the same API call, the same 5 GB of weights. On an iGPU. Under $0.05 of electricity per full demonstration run.

The two forces working against this becoming trivial — and the reason a dGPU is still the right investment — are pure hardware physics: memory bandwidth (the 8 tok/s decode ceiling on LPDDR5x-shared UMA) and VRAM ceiling (no room for a 30 B+ model at this quant, no room for parallel batched requests, no room for a larger native context window). The next chapter covers exactly what hardware moves each of those ceilings and by how much.

\newpage

Chapter 4 Hardware requirements (dev + prod ladder)

Hardware requirements — how they scale, and what to buy

The previous chapter demonstrated what an 8-billion-parameter vision-language model does today, on an integrated GPU sharing 30 GB of LPDDR5x with the CPU. The ceiling is memory bandwidth (~130 GB/s aggregate → ~8 tokens/second decode). Below is the ladder of currently-sold hardware that removes each successive ceiling and what you get for that money.

Two things worth flagging up front:

- **Model sizes have grown dramatically and continue to.** GPT-3 (2020) shipped at 175 B parameters; ChatGPT-original (2022) collapsed the quality bar down to something in the 20-30 B range. Qwen 2.5, Llama 4, Mistral Nemo — 7 B to 30 B was the productive class through 2024. Qwen 3 (2025-2026) and DeepSeek V3 pushed the useful ceiling to 235 B mixture-of-experts. **Every emergent capability jump — reliable tool-calling, coherent multi-turn reasoning, no-hallucination retrieval, per-domain multilingual competence — happens at a specific parameter threshold.** You don't get "a little bit of" tool-calling by half-fitting a model that's a little too big; you get zero or one, depending on whether you crossed the threshold.
- **What "runs" a model changes as you cross size thresholds.** A 7 B fits on any decent workstation. A 30 B needs 24 GB VRAM. A 72 B needs 48 GB. A 235 B MoE with A22B active needs multi-GPU tensor-parallel or a fabric-connected DC-tier card. **The delta from one tier to the next is not linear** — it's an emergence step, and if the customer's workload is on the wrong side of that step, no amount of squeeze on the current tier recovers it.

This is why the hardware discussion below climbs by rung rather than picking a single "recommended spec." Where you land depends on which emergent capabilities your workload requires.

Tier 0 — This PoC (what the last chapter ran on)

Component	Spec	Notes
Box	AceMagic F5X (~€1300)	Mini-PC, ~500 mL volume, single 65 W USB-C PD or barrel.
CPU/iGPU/NPU	AMD Ryzen AI 9 HX 370	Strix Point: 12 cores (4× Zen 5 + 8× Zen 5c), Radeon 890M iGPU (gfx1150), XDNA 2 NPU (50 TOPS INT8, currently unused for LLM inference on Linux).
Memory	64 GB LPDDR5x-7500	Shared with iGPU as UMA (BIOS allocates 30 GB nominally, up to ~48 GB visible via GTT extension).
Bandwidth	~130 GB/s aggregate	The decode ceiling — everything downstream is bandwidth-bound.
Storage	2× 4 TB NVMe Gen 4	Fine for corpus + Qdrant + several models on disk.

What runs here well: Qwen 2.5-7B greedy for text, Qwen 3-VL-8B-Thinking for vision, BGE-M3 embedder on CPU, bge-reranker on iGPU. **~8 tokens/s decode, ~170 tokens/s prefill.** OCR of a dense receipt takes ~40 s; a Farsi document with translation takes ~3 minutes.

What doesn't run here: anything above ~10 B parameters at Q4 (VRAM budget), multi-user concurrent load (single generation slot bottleneck), 32k context with `-np 4` parallelism (RAM squeeze), any 30 B-class model where the emergent-capability step lives.

Cost profile: ~€1300 box, ~35 W idle, ~65 W under load. Effectively free to run 24/7.

Tier 1 — Developer workstation

A single-user developer machine — one person builds, tests, and demonstrates on it. Removes the tokens/second ceiling enough to iterate on prompts and tool

chains without being interrupted every 40 seconds waiting for a Thinking-class model to finish.

Component	Recommended	Alternatives (currently sold)
GPU	**NVIDIA RTX 5090 · 32 GB GDDR7 · ~1790 GB/s** (~€2 400)	RTX PRO 5000 Blackwell (48 GB GDDR7, workstation-flavor, ~€4 800); AMD Radeon PRO W7900 (48 GB, ~€3 800; ROCm-only, worse LLM ecosystem).
CPU	AMD Ryzen 9 9950X3D (16-core, 3D V-Cache)	Intel Core Ultra 9 285K (24-core). Either fine — LLM decode is GPU-side; CPU only matters for RAG orchestration + tool calls.
RAM	128 GB DDR5-6400 (2× 64 GB, dual-channel)	96 GB or 192 GB fine. Not the bottleneck.
Storage	2× 4 TB NVMe Gen 5 (one system, one models/corpus)	8 TB total roughly fits a "several models on disk + full corpus + Qdrant" without pruning.
PSU / case	1000 W 80+ Platinum · full-tower	RTX 5090 pulls 575 W peak, wants proper airflow.
Approx BOM	**€3 500 - €4 200** (5090 build)	5090 + 9950X3D + 128 GB + 8 TB NVMe + PSU + case + cooling.

What runs here well: Qwen 3-VL-8B-Thinking at **~50-70 tokens/s decode** (~7× the PoC), the full 32k context comfortably, Qwen 3-VL-30B-A3B (a 30 B mixture-of-experts with 3 B active — fits in 32 GB Q4) at ~15-25 tokens/s, live iteration cycles feel instant.

What still doesn't run here: 70 B dense models at any usable quant, multi-user concurrency beyond ~3-4 parallel slots, deep-batching for high-throughput scenarios (agentic workflows spawning 20+ parallel tool calls).

Best value point. The 5090 is roughly the last consumer-grade rung before workstation/DC pricing kicks in. If a workload doesn't require 48 GB VRAM (i.e.

you're happy with 30 B-class MoE and don't need multi-user), the 5090 is the sweet spot.

Tier 2 — Small-team production server (~10-50 concurrent users)

The point at which a workstation stops being enough. One or two workstation-class GPUs in a rackmount chassis, driving a departmental or small-customer deployment. Serves both the LLM and the reranker + embedder from the same server; Qdrant colocated.

Component	Recommended	Alternatives
GPU (×1 or ×2)	**NVIDIA RTX PRO 6000 Blackwell Max-Q · 96 GB GDDR7 · ~1800 GB/s** (~€10 500 each)	NVIDIA L40S (48 GB GDDR6, ~€8 500 — older Ada arch, less memory but well-established) · AMD MI300X (192 GB HBM3, ~€15 000 — much cheaper per GB VRAM but ROCm software friction remains a real cost).
Server chassis	Supermicro AS-2124GQ-NART or Dell PowerEdge R760xa	2U or 4U depending on GPU count; needs PCIe 5.0 x16 per card and 800 W+ per card.
CPU	1× AMD EPYC 9354P (32-core, PCIe 5.0 lanes)	Or Intel Xeon 6 Granite Rapids equivalent. Single-socket is enough.
RAM	512 GB DDR5-4800 ECC RDIMM	For Qdrant + FS cache + inference server + head-room.
Storage	4× 8 TB NVMe Gen 5 U.2/ E1.S (RAID 10)	~16 TB usable — corpus + Qdrant snapshots + N mod-els on disk.
Networking	2× 25 GbE (or 100 GbE if fronting a bigger cluster)	For OpenNebula integration + client-facing traffic.
PSU	2× 2000 W redundant	RTX PRO 6000 = 600 W each; leave margin.
Approx BOM	**€18 000 - €25 000** (single-GPU) · **€28 000 - €35 000** (dual-GPU)	Chassis + CPU + RAM + storage + GPU(s) + PSUs.

What runs here well: Qwen 3-72B-Instruct at Q4 (fits in 96 GB with room), Qwen 3-VL-32B-Thinking at BF16 (fits in one 96 GB card), 8-16 parallel inference slots, sub-second first-token latency, real customer-facing SLAs. Full ADR-0008 reranker at BF16 not Q4.

What still doesn't run here: Frontier-class 405 B+ dense models, or 671 B MoE like DeepSeek V3, or multi-tenant workloads where a hundred concurrent users each expect an interactive response.

Best fit: internal-tool deployments (single company, few dozen active users), customer-facing pilots, embedded-in-product AI features for SMEs.

Tier 3 — DC-tier serious production (100s of concurrent users, or a foundation model to serve)

At this rung we're in HGX/DGX territory — GPU-fabric-connected servers with NVLink, deployed in ones or twos.

Component	Recommended	Alternatives
GPUs	**NVIDIA HGX H200 · 8× H141 · 1128 GB HBM3e total · 4.8 TB/s per card · NVLink 5** (~€300 000 for a bare HGX baseboard alone)	AMD MI325X 8-way system (~€250 000, comparable HBM3e VRAM, PCIe fabric not NVLink); NVIDIA B200 HGX (Blackwell DC — ~€400 000, another generation newer).
Server platform	Supermicro SYS-821GE-TNHR or NVIDIA DGX H200	8U to 10U form factor; requires 400 V 3-phase power, liquid cooling, dedicated rack.
CPU	2× Intel Xeon Platinum 8592+ (or dual AMD EPYC 9754)	128 cores total; provides PCIe lanes + RAM channels for orchestration.
RAM	2 TB DDR5-4800 ECC (24× 96 GB)	Feeds the CPU-side vLLM prefetch buffers + KV-cache spillover.
Storage	60 TB NVMe Gen 5 (RAID 10) + separate parallel filesystem for the model catalogue	For continuous training-data ingestion in addition to inference.
Networking	2× 400 GbE ConnectX-7 (or InfiniBand NDR)	For multi-node scaling and client-facing throughput.
Power	~10 kW at load	Requires proper DC facility.
Approx BOM	**€400 000 - €600 000 for a single HGX/DGX server**	Ready-to-run; excludes rack, PDU, cooling.

What runs here well: Qwen 3 Coder 480B-A35B-Instruct at BF16 (needs multi-GPU tensor-parallel), Qwen 3-VL-235B-A22B, DeepSeek V3 671B, Llama 4 Maverick 400B — the frontier open-weight tier. Hundreds of concurrent users. Deep-batching. Fine-tuning by LoRA/QLoRA. Continuous embedding backfill.

What doesn't run here: frontier proprietary models where the weights aren't yours to load. Otherwise there is no ceiling in the currently-shipping open-weight space.

Best fit: the point at which the product is the AI, not "a product with AI in it." Enterprise SaaS pivots, foundation-model resellers, sovereign-AI national initiatives.

The trajectory that matters for planning

The parameter-count / VRAM / bandwidth requirements have grown roughly **2×-3× per year** across every open-weight release wave since 2022, with no plateau in sight. The Qwen 2.5 → Qwen 3 → Qwen 3-VL step alone doubled the useful-model class from 7 B to 32 B; the next step will likely reach 72 B-dense or 200 B-MoE as the "productive floor." Hardware planned today for a 3-year lifecycle should be provisioned with **2× headroom** on VRAM and bandwidth, not sized to today's model.

Consequences for this project's roadmap:

- **Tier 0 (PoC)** is a genuine on-prem demonstration platform and will remain useful for prompt-engineering, corpus ingestion tuning, and travel demos. It will not track the model frontier.
- **Tier 1 (single dGPU workstation)** is what unlocks Qwen 3-VL Instruct + Thinking at interactive latencies + the 30 B-A3B MoE class. This is the minimum plausible dev environment going forward. Any customer-facing demo run on Tier 0 hardware will visibly lag against expectations set by cloud AI.
- **Tier 2 (workstation-class rack GPU)** is the first customer-deployable tier — the point at which a company can host their own instance without knowing what they're doing at the infrastructure level. The whole ragfarm architecture (Open WebUI + mcpo + MCP toolchain + hybrid retrieval + rerank + guarded actuation) is designed to redeploy unchanged onto this tier; only the `OPENAI_API_BASE_URL` moves.
- **Tier 3 (DC HGX)** is the tier where a hosted-AI product becomes competitive with commercial offerings on both quality and speed. Not required for the current use case; noted for the roadmap.

The single most important decision to defer or make: whether Tier 1 (single-workstation dGPU) makes sense as an interim step before jumping to Tier 2 for customer deployment. On this exact project, Tier 1 unblocks development, and Tier 2 unblocks the customer pilot. The €4 000 vs €25 000 delta is small compared to the year of engineering time each unlocks.

\newpage

Chapter 5 Deployment guide

Deployment & prod re-deployment notes

Author: David Kubicek (david.kubicek@eywo.cz)

Operational facts captured during the PoC build (steps 02–07). ADR-0001/0003 hold the why; this holds the concrete what needed to redeploy — especially the things that must change when the PoC AMD MiniPC is replaced by prod NVIDIA HW.

Runtime topology — live component registry

The maintained inventory of everything that runs. This is the **live runtime registry**; it supersedes the point-in-time snapshot in ADR-0005 as "what is running now." The naming, port-allocation and collocation **rules** (how a short name derives its directory / container / unit / port / mount) stay authoritative in **ADR-0005** — this table is the current state, that ADR is the policy. Add a component → add its row here and take the next `81xx` port per ADR-0005. All container services run `network_mode: host` (see below) and bind loopback except open-webui.

plane	compo-nent	direc-tory	com-pose ser-vice / con-tainer	bind	mcpo mount	agent-facing surface	mu-tates
host	llama	built at `~/llama.cpp`	— (sys-temd `ragfarm-llama`)	`127.0.0.1:8080`	—	OpenAI base URL (swap-pable)	no
host	em-bedder	`services/embedder`	— (sys-temd `ragfarm-embedder`)	`127.0.0.1:8090` / `embed`	—	nal — inter-dense+sparse embed	no
host	reranker	(built at `~/llama.cpp`)	— (sys-temd `ragfarm-reranker`)	`127.0.0.1:8081` / `reranking`	—	inter-nal — bge-reranker-v2-m3 GGUF, **iGPU/Vulkan** (ADR-0008)	no
host	in-gester (+ watcher)	`services/ingester`	— (sys-temd `ragfarm-ingester-watcher`)	—	—	batch + autonomous incre-mental sync (ADR-0006)	writes Qdrant
con-tainer	qdrant	up-stream image	`qdrant` / `infra-qdrant`	`127.0.0.1:6333/6334`	—	re-trieval store; volume `qdrant_data`	no
con-tainer	rag	`services/rag-retrieval`	`rag-retrieval` / `infra-rag-retrieval`	`127.0.0.1:8104`	`/rag`	**tool server** — `search_corpus`	no
con-tainer	place-ment	`services/mcp-placement`	`mcp-placement` / `infra-mcp-placement`	`127.0.0.1:8101`	`/placement`	**tool server** — mock	no
con-tainer	mcpo control	up-stream image	`mcpo` / `infra-mcpo`	`127.0.0.1:8000` (tailnet front)	`/control`	**only** — Open-WebUI admin	YES — guarded
con-tainer	mcpfs	`services/mcpfs` (+ `.host-control`)	`mcpfs` / `infra-mcpfs`	—	`/mcpfs`	**only** — Open-WebUI (Anthropic)	YES — guarded

`ragfarm-stack.service` launches the container plane **except `mcp-fs`** (unbridged) and **except the ingester** (a host job / the watcher unit, not the stack): qdrant, rag-retrieval, mcp-placement, mcp-host-control, mcpo, open-webui. The reranker is a **second host `llama-server`** on the iGPU (`:8081 --reranking`, ADR-0008) — a Vulkan sibling of the LLM, not an embedder endpoint; the embedder is embeddings-only. So there are **two `llama-server` processes**: the LLM (`:8080`) and the reranker (`:8081`), both on the iGPU.

Why host networking (load-bearing PoC fact)

The LLM (`:8080`) and embedder (`:8090`) are host processes bound to **127.0.0.1 only** (not exposed to the LAN). A bridge-network container cannot reach a loopback-bound host service via `host.docker.internal` (connection refused). So rag-retrieval / mcpo / open-webui run with `network_mode: host` and reach the host services at `127.0.0.1`. They keep their own binds on loopback (`RAG_HOST=127.0.0.1`, mcpo `--host 127.0.0.1`) except open-webui, which binds `0.0.0.0` for remote access.

Remote UI access

`OWUI_HOST` (compose env) controls the Open WebUI bind: `0.0.0.0` (default now = remote access, login-gated by `WEBUI_AUTH`) or `127.0.0.1` (loopback-only; reach via `ssh -L 3000:127.0.0.1:3000 <host>`). **Restrict `:3000` at the host firewall to trusted networks** — it's the only externally reachable service.

Tested model versions (known-good baseline)

The fetch scripts default to **latest** (easy swapping/experimentation), but the model set every gate + eval in this repo was validated against is the pinned one below. Pass the `--revision` flags to reproduce it exactly; omit them for latest. These pins are for **reproducibility only** — not a security constraint (the old no-pickle rule is retired, see the model-format note in `scripts/lib-models.sh`).

role	model	tested revision	reproduce with
LLM	Qwen2.5-7B-Instruct GGUF Q4_K_M (`Qwen/Qwen2.5-7B-Instruct-GGUF`)	GGUF, current	`scripts/fetch-llm.sh`
embedder	`BAAI/bge-m3`	`5f9396f75618b3389c1fd1068a1ff58dc7b5b26` (has `model.safetensors`; HEAD ships only `pytorch_model.bin`)	`scripts/fetch-encoder.sh --embed-revision 50f9396f75618b3389c1fd1068a1ff58dc7b5b26`
reranker	`BAAI/bge-reranker-v2-m3`	`953dc6f6f85a1b2dbfca4c34a2796e7dde08d41e`	`scripts/fetch-encoder.sh --rerank-revision 953dc6f6f85a1b2dbfca4c34a2796e7dde08d41e`

The embedder + reranker must stay a **compatible pair** (`fetch-encoder.sh --list`); the currently-deployed embedder is the safetensors revision above (same weights as latest, faster load). Per-model detail lives in `models/{llm,embeddings,reranker}/MODEL.md`.

Autostart & lifecycle

What starts on boot

- **Host services** (systemd, already `enabled`): `ragfarm-llama.service` (LLM), `ragfarm-reranker.service` (iGPU cross-encoder, ADR-0008), `ragfarm-embedder.service` (CPU embeddings), `ragfarm-ingester-watcher.service` (autonomous corpus sync, ADR-0006).
- **Container stack**: `ragfarm-stack.service` runs `docker compose up -d` for the six container services (see the topology note above), then its `ExecStartPost` runs `scripts/mcpo-heal.sh`. Containers also carry `restart: unless-stopped` and `docker.service` is enabled — so a normal reboot restarts them anyway; the unit guarantees ordering + one control point (install steps in the unit header).

Start / stop / status everything (host, as `dave`)

One command (recommended) — `scripts/stack.sh` runs the whole ordered sequence and health-checks the result. Reach for this unless you're debugging one service:

```
scripts/stack.sh start      # host services → container stack → health check
scripts/stack.sh stop       # container stack → host services (reverse order)
scripts/stack.sh restart    # stop, then start
```

```
scripts/stack.sh status     # systemd unit + container status at a glance
scripts/stack.sh health     # probe every endpoint; non-zero exit if any is down
```

The explicit per-service sequence below does exactly the same thing — use it when you need to bring one piece up or down on its own.

Cold start — host model hosts first, then the container stack:

```
sudo systemctl start ragfarm-llama ragfarm-reranker ragfarm-embedder ragfarm-ingester-watcher
sudo systemctl start ragfarm-stack
```

Stop everything — stack first, then host services:

```
sudo systemctl stop ragfarm-stack
sudo systemctl stop ragfarm-ingester-watcher ragfarm-embedder ragfarm-reranker ragfarm-llama
```

Status at a glance:

```
systemctl --no-pager status ragfarm-llama ragfarm-reranker ragfarm-embedder ragfarm-ingester-watcher ragfarm-stack
docker compose -f infra/compose.yaml ps
```

Restarting individual pieces (the gotchas)

- **rag-retrieval**: any restart severs mcpo's streamable-http MCP session — always follow with `scripts/mcpo-heal.sh` (or just `sudo systemctl restart ragfarm-stack`), otherwise tools come up unmounted (ADR-0007 note #4; the boot healer is the stack unit's `ExecStartPost`).
- **reranker & embedder**: independent host services now (a GPU `llama-server` on `:8081` and the CPU embedder on `:8090`); restarting one doesn't touch the other. The reranker loads its ~1.2 GB GGUF on the iGPU in ~1–2 s; rag-retrieval reaches it via `RERANK_ENDPOINT` (`:8081/reranking`). One gotcha: llama.cpp reranking scores each `(query,doc)` pair in one physical batch, so the unit sets `-b/-ub 4096` — if chunks ever grow past that, raise it (see `ragfarm-reranker.service`).
- Manual `docker compose` ops need the proxy env first: `source scripts/proxy-env.sh` (image pulls + container proxy inheritance).

Operator scripts (`scripts/`) — a newcomer's map

Everything here runs from the repo root on the host as `dave`. Grouped by job.

Deploy & lifecycle

- `stack.sh` — the **one-command operator entry point**: `stack.sh {start|stop|restart|status|health}`. Brings the whole stack (host `llama`/`reranker`/`embedder`/`ingester-watcher` units + the container stack) up or down in the right order

and health-checks every endpoint. Use this for day-to-day start/stop; the per-service systemd sequence is only for debugging one piece (see Autostart & lifecycle above).

- `deploy.sh` — the reproducible, idempotent full deploy of the durable stack: ordered phases (preflight → venv → host services → stack → corpus → watcher → verify), each ending in a machine-checkable gate; safe to re-run. Uses `sudo` only for the specific systemd install/enable actions (never wraps the whole script). Picks a Python dependency profile — `--profile cpu` (default) or `cu12` — pinned in `services/requirements.lock` / `requirements.cu12.lock` (identical package set; only the torch wheel index differs, CPU vs CUDA 12.x). Builds the reranker GGUF if absent and installs all host units incl. `ragfarm-reranker`. The AI-out-of-the-loop path for repeatable deploys.
- `mcpo-heal.sh` — waits until the MCP backends accept TCP, then restarts mcpo once so every tool mounts cleanly (works around the streamable-http boot race). Runs as the stack unit's `ExecStartPost`; run it by hand after any ad-hoc `rag-retrieval` restart.
- `proxy-env.sh` — **`source` it, don't execute**, before any network command (pip / HuggingFace / `docker compose`); loads repo-root `.env` and normalizes `HTTP(S)_PROXY`/`NO_PROXY` (guarantees loopback + containers bypass the proxy).

Model management (fetch / hot-swap)

- `fetch-llm.sh` — fetch/swap the generative LLM GGUF into `models/llm/<slug>/` and write `LLM_GGUF_PATH` to `.env` (the unit reads it). `--list` shows known-good tool-calling LLMs; `--repo`/`--file` swap; `--force` re-fetch. Restart `ragfarm-llama`. **Vision models** (e.g. Qwen2.5-VL) are handled automatically, no separate flag: a vision-language GGUF needs a second, small "multimodal projector" GGUF passed to llama-server as `--mmproj`, or it loads but can't see images. Before downloading, the script checks whether the HF repo itself hosts a `*mmproj*.gguf`; if so it fetches that too and writes `LLM_GGUF_MMPROJ`, else (plain text model) it **clears** that var — so switching back to a text-only model can't launch with a leftover `--mmproj` from a previous vision model. Example (fetches both files):

```
scripts/fetch-llm.sh --repo ggml-org/Qwen2.5-VL-7B-Instruct-GGUF \
  --file 'Qwen2.5-VL-7B-Instruct-Q4_K_M.gguf'
```

- `activate-llm.sh` — switch the active LLM among models **already on disk**, no re-download. Lists every `models/llm/<slug>/` that has a complete GGUF (interactive numbered prompt, or `--dir <slug>` / `--path <file>` non-interactively; `--list` prints and exits), and writes `LLM_GGUF_PATH` + `LLM_GGUF_MMPROJ` to `.env` using the same mmproj auto-detect/clear rule as `fetch-llm.sh`. This is the tool for

"I already fetched three models, which one is live" — fetch once per model, activate freely between them:

```
scripts/activate-llm.sh --list                          # what's on disk
scripts/activate-llm.sh --dir qwen2.5-32b-instruct-gguf   # switch to it
sudo systemctl restart ragfarm-llama                     # apply
```

- `fetch-encoder.sh` — fetch/swap the **matched** embedder+reranker pair into `models/embeddings/<slug>/` and `models/reranker/<slug>/` (converts the reranker to GGUF), writing `EMBED_MODEL_PATH` + `RERANK_GGUF_PATH` to `.env`. `--list` shows compatible pairs. Restart `ragfarm-embedder ragfarm-reranker`. Both fetch latest, auto-pick the fastest weight format, and are the ONE place download logic lives — `deploy.sh`'s venv phase calls them (no duplication). `lib-models.sh` is their shared helper (sourced, not run).

Activating a swapped model — the fetch/activate scripts only write `.env`; the units read it on (re)start. What you must do after a swap depends on which model changed:

- **LLM** (`fetch-llm.sh` / `activate-llm.sh`) → `sudo systemctl restart ragfarm-llama`. Nothing else — the LLM doesn't touch embeddings or the corpus. (If the model alias changed, also re-run `infra/openwebui/setup_openwebui.py` so the OWUI preset points at it.) The unit's `ExecStart` conditionally adds `--mmproj` only when `LLM_GGUF_MMPROJ` is non-empty, so a text-only `.env` (the common case) launches exactly as before — the mmproj knob is a no-op unless you're running a vision model.
- **Reranker only** → `sudo systemctl restart ragfarm-reranker`. Query-time only; no re-ingest.
- **Embedder** (`fetch-encoder.sh`) → the vector space changes, so you MUST re-embed: `sudo systemctl restart ragfarm-embedder ragfarm-reranker` **then** `.venv/bin/python services/ingester/ingester.py --recreate --corpus /data/corpus` (or `scripts/deploy.sh --recreate-corpus`). **Skipping the re-ingest silently breaks retrieval** — old stored vectors vs new query vectors. `scripts/stack.sh restart` restarts every service but does **not** re-ingest; that step is always manual. (`fetch-encoder.sh` prints this reminder when it actually re-fetched the embedder.)

Retrieval / RAG debugging

- `rag_pool_inspect.py` — dump the first-stage RRF candidate pool for a query (`--branches` adds the dense-only and sparse-only lists). Separates a RANKING problem (chunk is in the pool but scored low → the reranker's job) from a RECALL problem (chunk never entered the pool → needs better first-stage recall / query expansion). This is the raw, pre-rerank view.

```
.venv/bin/python scripts/rag_pool_inspect.py --branches "hsmbvxip001ts" "proj
vedoucí EPC"
```

- `ingest-embed-test.sh` — quick smoke test: Qdrant points carry non-empty sparse vectors, and a known-hostname `search_corpus` lookup returns rows.

Agent / tool-routing debugging

- `dump_mcp_openapi.py` — enumerate every mcpo mount and dump its `openapi.json` plus the exact `operationId` the model is shown (e.g. `tool_search_corpus_post`). Ground truth when writing routing hints or debugging why the model does/doesn't call a tool. `--full` for the whole schema per mount.
- `trace_tool_calls.py` — drive llama-server with the **real** OWUI tool schemas (pulled live from mcpo) and the deployed grounding prompt at deterministic settings, feeding canned tool results, to watch which tools the 7B calls with what arguments across rounds. Regression-check routing after a prompt/schema change.
- `agent.py` — headless multi-turn agent over the same stack (llama-server + mcpo tools + the deployed grounding prompt) but with a context loop **we own and measure**. Unlike `trace_tool_calls.py` it EXECUTES tools for real (incl. a CLI reboot confirm mirroring `reboot_guarded.py`) and carries real history. Modes: interactive REPL, one-shot prompts, or scripted benchmarks (`--scenario reboot-canary`). Per turn it splits and times the **deliberate / tool / answer** stages (prefill vs decode each) and reports context size + how many old tool results were elided. This is the control for isolating an OWUI-loop bug from a model/stack bug — e.g. it reproduced the repeated-reboot miss with no OWUI in the loop, proving that failure is model discipline, not compaction.

Open WebUI configuration (reproducible, not hand-clicked)

OWUI stores config in its `openwebui_data` volume. Recreate it with `infra/openwebui/setup_openwebui.py` (idempotent) — this pushes TWO presets in one pass (text + vision, see ADR-0009 and the "Vision engine" section below):

- **Tool servers** `TOOL_SERVER_CONNECTIONS` → `/rag` (`server:0`) + `/placement` (`server:1`), both `path=openapi.json`, `auth_type=none`.
- **Preset** **`ragfarm`** (text): base `qwen2.5-7b-instruct` + rag/placement/reboot_guarded attached (`meta.toolIds`) + `params.function_calling=native` + greedy sampler (`temp=0`, `top_k=1`, `seed=42`) + the RULE-1..5 grounding prompt. Loadbearing: without it the 7B answers generic prose even when the exact chunk was retrieved.
- **Preset** **`ragfarm-vision`** (VL): base auto-detected via `/v1/models` (first entry with capability `multimodal`, overridable via `VISION_BASE_MODEL_ID`), non-greedy sam-

pler (`temp=0.6`, `top_k/top_p/min_p/seed` DROPPED so llama.cpp defaults apply — required by Qwen3-VL Thinking) + vision + file-upload capabilities on + a RULE-1..6 prompt that adds image-input rules and a draw.io HTML template pointing at the local viewer (`127.0.0.1:8091`, see `drawio-viewer` service in `infra/compose.yaml`).
- Capabilities matrix (both presets): file_context, file_upload, web_search, code_interpreter, citations, status_updates, usage, builtin_tools all ON; image_generation, terminal OFF; vision only on VL preset.
- Default features (per-chat pre-selected): web_search + code_interpreter.
- Builtin tools: everything ON except knowledge and calendar (OWUI opt-out convention — absence = enabled).
- OpenAI endpoint: `OPENAI_API_BASE_URL=http://127.0.0.1:8080/v1` (compose env).
- Open WebUI's built-in document RAG is deliberately unused for the corpus (Option B).

Run it (admin token, or email + password on the CLI):

```
OWUI_URL=http://127.0.0.1:3000 OWUI_TOKEN=<admin JWT> \
  .venv/bin/python infra/openwebui/setup_openwebui.py
```

Only one llama-server model is loaded at a time — the preset whose `base_model_id` matches the wrapper's `--alias` is the one that actually works right now; the other stays as stored config waiting for the next model swap.

Verifying the toolchain

`infra/openwebui/check_toolchain.py` — exit 0 full pass, 2 needs interactive browser confirm, 1 deterministic failure. The deterministic layer (mcpo sparse+dense + llama tool-calling) is the reliable CI signal; OWUI's interactive agent loop is confirmed in the browser (it is finicky to drive headlessly — the one non-obvious requirement is `assistant_message_id` **in the chat/completions request body**, encoded in the script).

OpenNebula MCPs — mock mode & the confirmation gate (ADR-0004)

Until live OpenNebula exists, `mcp-placement` and `mcp-host-control` run in **mock mode** (`ONE_MOCK`/`HOST_MOCK` default to 1 in compose) against canned VM↔host data, so the agent path and the human-in-the-loop UX are testable with no cluster. Set `ONE_MOCK=0`/`HOST_MOCK=0` (and fill `.env`) at deployment.

- **Read-only tools** (`where_is_vm`, `list_vms_on_host`) are exposed to the model via mcpo (`/placement`) as tool server `server:1`.

- **The mutating op is gated.** `host-control` is bridged by mcpo but **deliberately NOT registered as an OWUI tool server** — the model cannot call reboot directly. It goes only through the OWUI native Python Tool `infra/openwebui/tools/reboot_guarded.py`, which: dry-runs to get the plan → shows it in a blocking `__event_call__` confirmation modal → executes with `confirm=True` only on human approval. The server-side allowlist (`HOST_ALLOWLIST`) + `confirm` gate still apply underneath. This is the ADR-0004 pattern: the LLM is out of the confirm loop; the human is the gate.
- `setup_openwebui.py` registers both tool servers, creates the Python Tool, and attaches all three (`server:0` rag, `server:1` placement, `reboot_guarded`) to the `ragfarm` preset. `__event_call__` modals require an interactive UI session (they do not fire on headless/API calls — correct for human confirmation).

What changes on prod NVIDIA hardware

Per ADR-0003 the durable layer (Open WebUI, mcpo, MCP servers, `search_corpus`, Qdrant) is HW-agnostic and should NOT be re-architected. Concrete changes:

- **Inference**: replace llama.cpp/Vulkan with a CUDA server (vLLM/TGI/llama.cpp-CUDA). Repoint `OPENAI_API_BASE_URL` only. Keep the OpenAI-compatible contract. This is also the moment to move off the 7B: a ~30B model (e.g. Qwen2.5-32B) is expected to resolve most observed 7B issues — tool-calling discipline (the repeated-reboot miss, ADR-0008/agent.py), verbose rambling answers, and instruction-following — and brings a larger native context. Tensor-parallel across two GPUs buys still-larger context / throughput if needed. Nothing in the durable layer changes (ADR-0003).
- **Embedder**: BGE-M3 (`/embed`, currently CPU) moves onto the GPU (CUDA); keep the dense+sparse contract on `:8090`. Re-ingest is unnecessary if model+revision are unchanged.
- **Reranker**: already GPU-accelerated on the iGPU via llama.cpp/Vulkan (`:8081 --reranking`, ADR-0008). On prod it swaps the Vulkan build for a CUDA one (or is served by the same CUDA inference stack); the `/reranking` contract is unchanged. Query-time only — never requires re-ingest.
- **Networking**: with a single CUDA stack and services able to bind a shared interface, the host-networking workaround can be dropped — move containers back to a compose bridge network and reach inference/embedder via service DNS or `host.docker.internal` (which requires those services to bind beyond loopback). Re-evaluate the `0.0.0.0` exposure + firewalling for the target network.
- **mcpo config**: `mcp-placement` (`where_is_vm`) and `mcp-host-control` are already mounted in `services/mcp-gateway/mcpo-config.json` and run in MOCK mode.

When OpenNebula is reachable (steps 05/06 unblock), set `ONE_MOCK=0`/`HOST_MOCK=0` and fill `ONE_XMLRPC`/`ONE_AUTH` per `services/mcp-placement/.env.example` — no mcpo-config or registration changes needed.

- **Corpus**: `CORPUS_PATH` and the Qdrant `corpus` collection (dense 1024 + sparse) are portable; re-run `services/ingester/ingester.py --recreate` against prod corpus.

Vision engine (Qwen3-VL family — ADR-0009)

Two OWUI presets coexist; only one is live at a time (the one whose `base_model_id` matches the wrapper's `--alias`). To flip between text and vision, or between Instruct and Thinking variants, use `activate-llm.sh`:

```
scripts/activate-llm.sh --list                                  # what's on disk
scripts/activate-llm.sh --dir qwen_qwen3-vl-8b-thinking-gguf   # switch to vision
sudo systemctl restart ragfarm-llama                            # apply (~30-60 s
to reload)
```

The wrapper (`scripts/llama-launch.sh`) auto-derives `--alias` from the model's directory name and conditionally adds `--mmproj` when the model dir contains a `*mmproj*.gguf` (vision models). No unit edits, no other flags to touch.

Live capabilities that Just Work

The Qwen3-VL preset has already been verified end-to-end on this stack:

- **OCR from image URL** — see the demo commands below. Auntie Anne's Indonesian receipt from the openlm.ai example was OCR'd correctly (all prices, all fields), at ~8 tok/s decode on the iGPU.
- **Image description** — attach any image in OWUI chat; the model describes content verbatim (RULE 4 of the vision prompt: no invention, verbatim OCR).
- **Diagram scan → regenerate as mermaid or draw.io** — screenshot a hand-drawn or existing diagram, ask "regenerate this as draw.io" (or "as mermaid").
- **Prompt-modify-synthesize** — attach an image, ask "same structure but add a reranker box between retrieval and generation". The model reads the input as a graph, applies your edit, and re-emits in the chosen format.

Diagram rendering

`ragfarm-vision`'s system prompt asks the user which format they want (never routes silently, never emits both):

- **Mermaid** — the user says "mermaid". Rendered natively by OWUI as an SVG.
- **draw.io** — the user says "draw.io" / "drawio" / "editable" / "interactive" / "pan-zoom". The model emits a fenced ```html block wrapping the raw `<mxfile>`

XML plus a script tag pointing at `http://127.0.0.1:8091/viewer-static.min.js` (the local `drawio-viewer` nginx container). OWUI's HTML preview iframe loads it — pan/zoom/lightbox/layer toggle work in-chat.

The IFRAME_CSP is set to allow scripts from `127.0.0.1:8091` only; nothing else opens up. The viewer's own external calls to `viewer.diagrams.net/styles` & `/shapes` may 404 on an offline demo box; the core rendering still works, only fancy stencil sets are missing.

Verified demo commands (no prep needed, run today)

The tests below use the live stack as-is. Nothing to fetch, nothing to install.

Sanity: vision model loaded?

```
curl -s 127.0.0.1:8080/v1/models | python3 -c 'import
sys,json; d=json.load(sys.stdin); print(d["models"][0]["model"], d["models"]
[0].get("capabilities"))'
# expect: qwen_qwen3-vl-8b-thinking ['completion', 'multimodal']
```

OCR from a public image (via base64 to bypass llama-server's HTTPS-off build):

```
.venv/bin/python - <<'PY'
import base64, requests, time
IMG = "https://ofasys-multimodal-wlcb-3-toshanghai.oss-cn-shanghai.aliyuncs.com/
wpf272043/keepme/image/receipt.png"
img = requests.get(IMG, timeout=30); img.raise_for_status()
uri = f"data:{img.headers['content-
type']};base64,{base64.b64encode(img.content).decode()}"
t0 = time.time()
r = requests.post("http://127.0.0.1:8080/v1/chat/completions", json={
    "model": "qwen_qwen3-vl-8b-thinking",
    "messages": [{"role":"user","content":[
        {"type":"image_url","image_url":{"url":uri}},
        {"type":"text","text":"Read all the text in the image."}]}],
    "max_tokens": 512, "temperature": 0.6,
}, timeout=120)
print(f"HTTP {r.status_code}  {time.time()-t0:.1f}s")
print(r.json()["choices"][0]["message"]["content"])
PY
```

In-UI vision demo (OWUI, admin@ragfarm.local):

1. Select model **`ragfarm-vision`** from the dropdown.
2. Click the paperclip → upload any image (receipt, screenshot of a diagram, photo of a whiteboard).

3. Ask one of: "OCR everything in this image", "describe what you see", "regenerate this diagram as mermaid" / "as draw.io".
4. For a diagram request, add `/no_think` to the prompt if the Thinking trace makes the wait too long (Qwen3 chat-template convention: strips the `<think>...</think>` block for that turn).

Serve local test images to the model: the same `drawio-viewer` nginx also serves anything under `infra/drawio-viewer/`. Drop a `.png` / `.pdf` there and reference `http://127.0.0.1:8091/<file>` in a prompt. Useful for a repeatable demo without leaning on external URLs.

/think vs /no_think (Qwen3 Thinking control)

Qwen3 parses these tokens out of user messages (chat-template convention, NOT system-prompt rules):

- **`/think`** — force the model to emit a `<think>...</think>` block before the answer. Default for `*-Thinking-*` model variants.
- **`/no_think`** — suppress the reasoning block for this turn. Runs a Thinking model like an Instruct one; ~3-5× snappier answers, at the cost of the reasoning quality on hard multi-step prompts.

Practical demo advice: default (thinking on) for the first, hardest question of the day (RAG lookup, complex OCR); append `/no_think` for follow-ups, quick lookups, and diagram requests where the trace adds no value.

Debug & measurement (`tests/tracing/`)

Standalone Python tools (no dependencies beyond `requests`) that answer where did the time go for any inference or chat turn. All safe to run against the live stack — read-only, no side effects.

Every tool takes `--url http://127.0.0.1:8080` (or the equivalent flag) so they're portable across the swappable llama-server endpoint. Default endpoint in the code is `localhost:8001` — always pass `--url` explicitly.

The catalog

Tool	What it measures	When to use
`ragfarm_bench.py`	Basic prefill/decode tok/s + TTFT + E2E per prompt	Quick "is llama fast enough right now" check
`ragfarm_bench_extended.py`	Full per-stage timings, absolute token+byte counts, context growth, CSV/JSON export	Regression tracking across commits or hardware swaps
`ragfarm_bench_chatid.py`	Same, plus context-blowup detection per chat session	Find when a conversation starts overflowing context
`chat_execution_tracer.py`	Chat session timeline: user→prefill→tool decision→tool exec→decision phase→generation, with orchestration-overhead %	Diagnose "chat feels slow" vs "LLM is slow"
`ragfarm_tracer_simple.py`	One-shot telemetry query: which model is loaded, endpoint latencies	Sanity check before any deeper trace
`ragfarm_integrated_tracer.py`	Combined engine telemetry + pipeline trace demo	Report generation for a whole pipeline
`ragfarm_rag_tracer.py`	RAG candidate-pool evolution: Qdrant → RRF → reranker, per-stage tokens/latency	Debug retrieval quality regressions ("why didn't my chunk win?")

(Not integrated: *`ragfarm_http_tracer.py`* — proxy-based, requires reconfiguring OWUI's LLM endpoint. Its only unique benefit is E2E prompt-answer wall time, which every other tool measures anyway.)

Verified one-liners (run today)

```
# 1. What model is loaded and how fast does one prompt run?
.venv/bin/python tests/tracing/ragfarm_tracer_simple.py query \
    --generation localhost:8080 --reranker localhost:8081
```

```
# 2. Baseline bench, one prompt, small max_tokens (fast)
.venv/bin/python tests/tracing/ragfarm_bench.py \
    --url http://127.0.0.1:8080 --prompt 1 --max-tokens 40

# 3. Extended bench, 3 prompts with CSV export (regression file)
.venv/bin/python tests/tracing/ragfarm_bench_extended.py \
    --url http://127.0.0.1:8080 --prompt 3 --max-tokens 128 \
    --csv bench_$(date +%s).csv

# 4. Chat tracer demo (canned session; no LLM required)
.venv/bin/python tests/tracing/chat_execution_tracer.py --demo
# writes chat_trace_demo.json in cwd

# 5. Real RAG pipeline trace for one query (endpoint is mcpo at :8000,
#    which mounts rag under /rag/search_corpus — NOT direct rag-retrieval :8104)
.venv/bin/python tests/tracing/ragfarm_rag_tracer.py trace \
    --chat-id demo_$(date +%s) \
    --query "FW pravidla pro host leadb229p.lea.piz" \
    --rag-endpoint http://127.0.0.1:8000
```

What "good" looks like on this iGPU (Qwen3-VL-8B-Thinking Q4_K_M, current baseline)

Metric	Now	Comment
Decode	~8-9 tok/s	LPDDR5x bandwidth-bound, both 7B and 8B land here
Prefill	~170 tok/s	The 2-order-of-magnitude speedup vs decode is expected
Vision OCR (dense receipt)	~40 s	300+ output tokens at 8 tok/s
Reranker turn	~1.9 s	Since ADR-0008 moved it to GPU/Vulkan (was ~36 s on CPU)
Tool overhead	~15-25 % of chat turn	Higher with reboot_guarded modal, lower for pure RAG

Baseline numbers older docs cite (300 tok/s prefill, 1000 tok/s decode) came from a much lighter model; the current live setup is intentionally slower and smarter. Track deltas from this table, not from those.

\newpage

Chapter 6 Prompt library

Board demo — prompt library, verified live

Author: David Kubicek (kubicek@gmail.com) **Last verified:** 2026-07-27 **Live model at verification time:** `Qwen3-VL-8B-Thinking` Q4_K_M — non-greedy sampler (temp 0.6, no top_k/top_p/min_p — Qwen3-VL Thinking's own requirement). **Stack:** llama.cpp / Vulkan on Radeon 890M iGPU (LPDDR5x-shared UMA). Retrieval: BGE-M3 dense+sparse → RRF → bge-reranker-v2-m3 on iGPU. Tools: rag/placement/reboot via mcpo (`:8000`).

How to use this file at the board:

- **Section A** is the historical text preset (Qwen2.5-7B, greedy) — what we've been demoing for weeks with screenshots to prove it. Kept verbatim as a fallback deck if the VL preset misbehaves live: `scripts/activate-llm.sh --dir qwen2.5-7b-instruct-gguf` swaps back in ~30 s (the script auto-restarts the service).
- **Section B** is tomorrow's live preset (Qwen3-VL-8B-Thinking) — every prompt fired against the live stack today; observed outputs are literal, taken from `scratchpad/prompts_results.json`.
- **Section C** is the honest technical caveat about `<think>`/`<no_think>` for board Q&A.
- **Section D** is the speed reality-check and the dGPU pitch.

The **exact prompt strings** are the primary artifact — copy/paste them verbatim tomorrow. If you're improvising, keep the same shape.

Section A — Text preset (Qwen2.5-7B Q4_K_M, greedy) — historical, screenshot-verified

Prompts we've been demoing for weeks. Each has a screenshot embedded in the main README that shows exactly what the answer looks like in-chat. Not re-verified on the VL model today — swap back with:

```
scripts/activate-llm.sh --dir qwen2.5-7b-instruct-gguf
# auto-restart; ~30-60 s to reload
```

A1 · RAG lookups (Czech unless noted) — see README screenshots

#	Prompt (Czech)	What the model does	Screenshot
1	`Jaká jsou FW pravidla pro host leadb229p.lea.piz?`	calls `search_corpus`, returns a **full 6-row markdown table** with every column: Source/Destination Network Address(es), Source/Destination Network Name(s), Destination Port(s), Protocol. Ends with `Source: <xlsx>`.	ex-01
2	`Dej mi kontakty na projektove vedeni v EPC.`	calls `search_corpus`, returns **6 people** as a numbered list with Firma / Role/oblast / Tel / E-mail for each (phone missing only where truly absent in source).	ex-02
3	`Co vis o hostu acclcass1?`	calls `search_corpus`, returns **19 fields** for that specific host: Prostředí, OS, Virtualizace, Storage, vCPU, RAM, HDD, Support, T-S Hostname, T-S IP, T-S Netmask, SA Hostname, SA domain suffix, SA IP, SA Netmask, SA VLAN ID, Storage OS/App+Data, filesystem, UID.	ex-03
4	`Kde ukládáme hesla pro EPC?`	calls `search_corpus`, explains NordPass storage + which record ("RDP User") to look up + how to use it with the RDP client.	ex-04
5	`Kde máme uložené GIT repo s dokumentací?`	returns the exact Azure DevOps URL: `https://SGC-DevOps@dev.azure.com/SGC-DevOps/Internal/_git/sa-hosting`.	ex-05
6	`Jak se přihlásím ze ŠA do EPC?`	comprehensive answer: **6 numbered login paths** (SSH direct, RDP direct, terminal servers, GUI, reverse proxy, CLI) with hostnames, IPs, credentials source, and pointers to `EPC25_VMs_config.xlsx` /	ex-06

A2 · Infra tools — mcp-placement (MOCK) + human-gated reboot

#	Prompt	Tool called	What happens
7	`Kolik je hodin?`	`get_current_timestamp`	one-line reply: `Nyní je HH:MM (GMT+2).`
8	`Kdo je prezident USA?`	`search_corpus` (RAG fires even on out-of-corpus, per RULE 1)	honest miss: `V dokumentu není zmíněn žádný prezident USA. Zde jsou některá jména zapsaná v dokumentu, ale nejsou to prezidenti USA.` **This is the desired behavior** — no hallucination.
9	`Rebootuj host node-03.`	`reboot_host` (behind `reboot_guarded` confirmation modal)	on approval: `Host node-03 byl úspěšně restartován. Než byl host restartován, bylo přesunuto 1 virtuálního počítače (sftp-gw). Po restartu se host opět připojil k clusteru.`

A3 · Diagrams (mermaid) — rendered in-chat by OWUI

#	Prompt	Output
10	`Vygeneruj stromový diagram slovních vazeb ve větě: "Once upon a time there was a very little dog called Steven who owned a nice little yellow car".`	full mermaid dependency tree, branching (not a linear reading chain) — `dog` is a hub with `a`, `very`, `little`, `called Steven` as children; `owned` branches off `who`; second `a` hub for `car`.

A4 · Coding — built-in `execute_code` interpreter

#	Prompt	What happens
11	`Vygeneruj mi kód pro quicksort a otestuj ho spuštěním nad malým polem náhodných řetězců. Prezentuj kód a výsledné pořadí tříděného pole po běhu sortu.`	model emits complete Python (`quicksort`, `random.choices`, `random.shuffle`), OWUI's code interpreter runs it, then the sorted-strings table is rendered inline as a data grid.

Section B — Vision preset (Qwen3-VL-8B-Thinking) — TOMORROW'S DEMO

Every prompt below was fired against the live stack on 2026-07-27 via `scratchpad/verify_prompts.py`. Observed outputs are literal (from `scratchpad/prompts_results.json`). The model is non-greedy so future runs will differ in wording; the shape of the answer is what to trust.

Delivery advice:

- Start with a **B3 OCR** prompt — receipt or Chinese/Farsi. This is the WOW block: same tokens rate as the old text model, but suddenly reads images. Sell this hard: it's what unlocks the dGPU pitch.
- Move into **B4 structure extraction** — bar chart → table, hand-drawn diagram → mermaid, code photo → runnable Python. Same story: eyes+brain replacing manual transcription.
- Close with a **B2 diagram** — mermaid or draw.io — the latter renders in-chat as an interactive canvas via our local `viewer.diagrams.net` mirror.
- If a text/RAG query is needed live, use one from Section A verbatim; the vision preset shares the same `search_corpus` tool.

B3 · Image OCR — verbatim + translation

B3a — Real-world receipt (Indonesian, Auntie Anne's, 310 KB PNG) — 77 s, 613 out tokens, decode 8.1 tok/s Prompt: `Read all the text in this image. Preserve the original numbers and layout.` Verbatim response:

```
AuntieAnne's

CINNAMON SUGAR
1 x 17,000
17,000

SUB TOTAL
17,000

GRAND TOTAL
17,000

CASH IDR
20,000

CHANGE DUE
3,000
```

B3b — Mixed English + Czech + IP addresses + floats — 59 s, 462 out tokens Prompt: `OCR every visible line. Keep punctuation, numbers, and IP addresses exact.` Verbatim response:

```
ragfarm - firewall rules 2026
Source: 10.244.1.26 -> Target: 172.20.220.0/24
Port: 445/tcp  Owner: petr.pyszko@epc.cz
Ceske znaky: prihlaseni, zaloha, pravidla
Numbers: 42, 1024, 3.14159, -273.15 K
```

Every IP, every float, every Czech diacritic-free word verbatim. Zero hallucinations.

B3c — CJK (Simplified Chinese) → English — 50 s, 394 out tokens Prompt: `This image contains Chinese text. Read each line and give an English translation.` Verbatim response:

```
Data Center - Beijing
Servers: 100 units
Total Bandwidth: 40 Gbps
Maintenance Window: Every Sunday 02:00-04:00
```

Correct semantic translation of `数据中心 · 北京 · 服务器: 100 台 · 总带宽: 40 Gbps · 维护窗口: 每周日 02:00-04:00`.

B3d — Farsi/Persian → English — 190 s, 1500 out tokens (long reasoning trace) Prompt: `This image contains Persian (Farsi) text. Read every line and provide an English translation next to each.` Result: the reasoning trace recognises the Persian text, breaks it into words (ایران→Iran, تهران→Tehran, دمای امروز→"today's temperature", درجه سلسیوس→"degrees Celsius"), and translates each line. Some digit misread on non-Arabic-shaping fonts (rendered ۴۲ as ۲۲) — real Farsi documents from a scanner won't have this issue. **Warning for live demo:** budget 3+ minutes for this one; if the box is impatient, start it before intro talk and come back to it.

B4 · Structured extraction from images

B4a — Bar chart → Markdown table — 28 s, 211 out tokens Prompt: `This image is a bar chart. Extract the underlying data into a Markdown table with two columns: Region and Revenue.` Verbatim response:

Region	Revenue
EU	240
US	410
APAC	180

```
| LATAM  | 90      |
| MEA    | 120     |
```

Every value correct to the last digit — pixel-measured bar heights read cleanly, labels transcribed, table formatted as asked. This is the "expense reports" and "extract data from PDF charts" story.

B4b — Architecture diagram photo → regenerated mermaid — 134 s, 1061 out tokens Prompt: `This image shows a small architecture diagram. Regenerate it as a mermaid graph (fenced mermaid). Preserve every box and every arrow direction.` Verbatim response:

```
flowchart TD
    A["User"] --> B["Open WebUI"]
    B --> C["llama-server"]
    B --> D["mcpo (tools)"]
    D --> E["rag-retrieval"]
    D --> F["OpenNebula"]
```

Every source box captured, every arrow direction preserved. OWUI renders it inline.

B4c — Photo of code on paper → runnable Python — 155 s, 1232 out tokens Prompt: `This image shows Python code. Transcribe it verbatim into a fenced python block. Then predict what it prints (do not use any tool).` Verbatim response:

```python
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n*0.5)+1):
        if n % i == 0: return False
    return True

print([n for n in range(30) if is_prime(n)])
```

Then: predicts output as `[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]`. **Caveat worth mentioning if a board member is technical:** the model transcribed `n**0.5` (square root) as `n*0.5` — same shape on paper, wrong operator. It got the predicted output right anyway because it reasoned about the math, not just the transcribed code. Honest limit of 8B vision at small monospace text; a 32B on dGPU would be tighter.

B2 · Diagrams — user chooses mermaid or draw.io

The vision preset waits for the user to say which format; it never guesses. If ambiguous it asks.

B2a — draw.io interactive — 187 s, 1500 out tokens Prompt: `Draw a mermaid diagram of the ragfarm data flow: user -> Open WebUI -> llama-server; Open WebUI -`

`> mcpo -> rag-retrieval -> Qdrant; rag-retrieval -> embedder + reranker. Then same diagram as draw.io HTML that renders in-chat.` The model emits a fenced ```` ```html ```` block containing the draw.io template with `window.STYLE_PATH / SHAPES_PATH / STENCIL_PATH / DRAW_MATH_URL / GRAPH_IMAGE_PATH` set to `http://127.0.0.1/…` (our local `drawio-viewer` nginx mirror on port 80), then a `<mxfile>` XML block, then `<script src="http://127.0.0.1/js/viewer-static.min.js">`. OWUI's HTML preview iframe loads it — you get an interactive canvas with pan, zoom, layer toggle, lightbox. **All the shape libraries load locally** — no internet, no CDN, air-gap-safe. See ADR-0009 → "Why we run our own nginx" for the load-bearing why.

B1 · Tools work on the VL model too (same schemas)
Same tools as the text preset — `search_corpus`, `where_is_vm`, `list_vms_on_host`, `reboot_host`. On the VL model, tool routing is slightly less snappy than on the greedy text model (the Thinking reasoning trace burns tokens before the tool call fires). Concretely:

- **`Jak se přihlásím ze ŠA do hostingu?`** — 40 s, calls `search_corpus`, returns a login procedure. Success mode; identical to Section A6.
- **`Jaká jsou FW pravidla pro host leadb229p.lea.piz?`** — 145 s. Tool call fires but the response body burned all tokens on reasoning. In OWUI (with *`max_tokens`* unlimited) the answer completes — see the Section-A ex-01 screenshot for the actual rendered answer.

For tomorrow's demo: if you want the FW-rules or contacts prompt done on-stage, use the **text preset** (swap via `activate-llm.sh`) — you already have proven screenshots and the greedy sampler is deterministic. Save the vision preset for image-input queries.

Section C — `<think>` / `<no_think>` — the honest caveat

Qwen3 nominally supports two in-message tokens that gate the visible reasoning block:

- **`/think`** — force `<think>...</think>` reasoning trace before the answer.
- **`/no_think`** — suppress it for a snappier answer.

On this specific `Qwen3-VL-8B-Thinking` Q4_K_M GGUF, `/no_think` is a no-op. The chat template baked into the GGUF has no `enable_thinking` gate and its `add_generation_prompt` block hardcodes `<|im_start|>assistant\n<think>\n` — so every assistant turn always starts a reasoning block regardless of what the user types. Empirically verified today (see ADR-0009 → "Instruct vs Thinking").

If a board member asks why we don't just turn thinking off: we can, by swapping to `Qwen3-VL-8B-Instruct` (a separate GGUF, ~5 GB, one `activate-llm.sh` swap + 30 s restart) — same vision quality, no reasoning trace, faster answers. Post-dGPU, that becomes the default and Thinking gets used only for hard multi-step questions where the extra latency is worth the accuracy. Today's demo runs Thinking because that's what's loaded; the swap is scripted and reversible.

Section D — Speed reality check (iGPU today → dGPU projection)

Verification numbers are on the **AMD Radeon 890M iGPU** sharing LPDDR5x (~130 GB/s aggregate bandwidth). Decode is memory-bandwidth-bound — physics, not tuning.

Metric	iGPU today (Qwen3-VL-8B-Thinking Q4)	Projected: dGPU ≥ 32 GB (RTX 5090 / A6000 / L40S)
Decode	**~8 tok/s**	45-70 tok/s (**6-9× faster**)
Prefill	~170 tok/s	800-1500 tok/s (**5-9× faster**)
Dense-receipt OCR (~300 out tok)	~40 s	**5-8 s**
Farsi-with-translation (~1500 out tok)	~190 s	~25 s
Room for 30B-class VL model	no (won't fit)	**yes** — `Qwen3-VL-30B-A3B-Thinking` fits in 32 GB Q4; jumps another quality tier
Room for larger context	tight with `-c 32768` + `-np 4`	comfortable; plus batched requests

The dGPU isn't a nice-to-have — it turns "wow, it can read a Farsi receipt in 40 s" into "yes, it just did that in real time while I was still finishing the sentence." Same code, same architecture, same everything else — only inference speed changes. Every prompt in Section B stays valid.

\newpage

Chapter 7 Ingestion pipeline

Ingestion pipeline — mixed docx / xlsx / pdf corpus

Author: David Kubicek (david.kubicek@eywo.cz)

This is the design reference for how `services/ingester/ingester.py` (+ `xlsx_tables.py`, `corpus_manifest.py`) turns the corpus into Qdrant points. Those files are **frozen and regression-locked** (see BUILD_STATE step 04): treat this doc as the explanation of what the parser does and why, and the **code as the source of truth where they differ**. Do not edit the parser to match this doc — raise a blocker instead. The corpus is ~10–30 files, mostly Czech + English, split between table-structured xlsx/csv (host → IP → VLAN → KVM-host assignments and contact sheets) and prose docx/pdf (descriptions in both languages).

The guiding principle: **tables and prose have different information shapes and are chunked differently.** A table row is a self-contained record; a paragraph is part of a flowing argument. One chunker for both would wreck retrieval on at least one.

Relevant ADRs: **ADR-0002** (BGE-M3 dense+sparse embedder, CPU), **ADR-0006** (content-addressed corpus sync — manifest, alias, watcher), **ADR-0007** (section-aware prose chunking; the `text`/`text_clean` split).

1. File routing

Walk the corpus root recursively (`scan_disk`). Route by extension (`iter_file`):

- `.xlsx`, `.xls` → **table path**, delegated to `xlsx_tables.iter_xlsx` (§2)
- `.csv` → **table path**, single-header CSV (`iter_csv`, §2)
- `.docx` → **docx path**: embedded tables → table path, body → prose path (§3)
- `.txt` → **prose path**, plain (`markdown=False`)
- `.md`, `.markdown` → **prose path**, Markdown headings honoured (`markdown=True`)
- `.pdf` → **prose path**, text-layer extraction (`pdfplumber`); a scanned PDF with no text layer is skipped with a warning
- anything else → skip with a logged warning (do not guess)

2. Table path (xlsx / csv / docx-tables) — one row per chunk

Tables are lookup data. The retrieval need is "give me the row for host X", "which hosts are on VLAN 203", or "the contact for the project lead" — precise, record-level. So:

- **One Qdrant point per data row.** Never embed a whole sheet or table as one chunk — that destroys row-level precision and overflows context.

- Each row is serialized to a flat, self-describing string joining each cell to its **header key** (`_serialize_kv`), sheet-prefixed for multi-sheet/table sources:

  ```
  sheet: CutOver kontakty, Řízení projektu Jméno: Marek Česal, Řízení projektu
  Firma: EPC, Tel: 739223474, E-mail: marek.cesal@epcommodities.cz
  ```

  - Empty cells skipped; whitespace normalised. **Values kept verbatim** — no lowercasing/reformatting of IPs/hostnames/VLAN IDs; exact tokens are the point.
  - This serialization is what makes the sparse vector useful: literal tokens (`prod-kvm-03`, `10.20.1.43`, `203`) land in the lexical index for exact match.
- **Payload** (per row): `source_file`, `sheet` (sheet name, `"csv"`, or `"table<n>"` for docx tables), `row_index`, `kind: "table_row"`, `lang: "n/a"` (rows are language-neutral key:value), and the serialized string as both `text` and `text_clean` (rows have no decoration to strip).
- **CSV** (`iter_csv`): first non-empty row is the header; a sheet with no sensible header is logged and skipped rather than guessed.

Messy real-world XLSX

All non-trivial XLSX structure — multi-table sheets, stacked/banded headers, multi-row title+header blocks, carry-forward and vertical-merge grouping, headerless data, trailing totals/notes trimming — lives in **`xlsx_tables.py`** and is locked by the offline regression (`tests/fixtures`, `test_xlsx_tables.py` → `ALL PASS`). This doc does not restate that logic; the code and its fixtures are authoritative. BGE-M3 handles 8192 tokens so a wide row is not a truncation risk; the parser front-loads identifying columns so the lexical signal leads.

3. Prose path (docx / pdf / txt / md) — section-aware chunks (ADR-0007)

Descriptions are flowing text where meaning spans sentences, and the primary docx uses **bold body text as headings** (not real Heading styles). The earlier paragraph-splitting chunker collapsed such a document into one page-sized slab whose embedding was a semantic average; ADR-0007 replaced it with section-aware chunking.

- **Section detection** (`_sections_from_lines`, `_docx_heading_level`): a new section starts at a Markdown `#…######` heading, a Word `Heading N`/`Title` style, **or** a short standalone fully-bold docx paragraph (≤12 words, ≤80 chars — the bold-run fallback). Top-level heading opens a section and clears the subsection; a deeper heading becomes the subsection.

- **Keep a coherent section whole; split only when forced** (`_emit_sections`). Sizes are measured in **whitespace tokens (words)**, a deterministic proxy for model tokens:
  - `CHUNK_MAX_WORDS = 480` (~600 model tokens) — a section at or under this is emitted as **one chunk**, never split just to hit a target (no partial answers).
  - `CHUNK_TARGET_WORDS = 300` — packing target when a larger section is split.
  - Splitting happens at **sentence boundaries only** (`_SENT_SPLIT`); a chunk never cuts a sentence, so retrieval can't hand the model a truncated instruction. A lone sentence over the ceiling is emitted whole.
  - `OVERLAP_FRAC = 0.15` — trailing whole sentences (~15% of target) are carried into the next chunk so a topic straddling a split appears in both.
- **Two texts per chunk** (ADR-0007): `text` / `text_raw` is **verbatim** (returned to the LLM for citation/quoting); `text_clean` is Markdown-decoration-stripped (`_strip_md`) and is the **embedding input** so syntax stops polluting the dense vector. Link/URL tokens and hostnames are kept in `text_clean` (the sparse branch needs them); only scaffolding (`**`, `#`, list/quote markers, code fences, pipes) is removed. For non-Markdown sources the two coincide.
- **Line-span citation metadata**: `chunk_start_line` / `chunk_end_line`. docx has no source lines, so the **paragraph ordinal** (1-based over all paragraphs) is the line proxy; pdf uses extracted-text line numbers.
- **Language** (`detect_lang` via `langdetect` on `text_clean`): `cs` | `en` | `other` | `unknown`. Metadata only — BGE-M3 is multilingual so retrieval does not depend on it, but it aids debugging/optional filtering.
- **Payload** (per chunk): `source_file`, `section_title`, `subsection_title`, `heading` (= section_title, used as retrieval's `location`), `chunk_index`, `chunk_start_line`, `chunk_end_line`, `kind: "doc_text"`, `lang`, `text`, `text_clean`.

4. Embedding + storage

For each chunk (either path), batched (`BATCH = 64`):

1. Call the embedder `POST /embed` with `kind="passage"` on the chunk's **`text_clean`** (table rows fall back to their verbatim `text`).
2. Receive `dense` (1024-dim) and `sparse` (token→weight map) per chunk.
3. Upsert one Qdrant point with BOTH named vectors and the §2/§3 payload:
   - `dense`: 1024-dim, cosine distance
   - `sparse`: into Qdrant's sparse index
   - the exact embedded string is mirrored back into the payload as `text_clean`.

Point IDs are opaque `uuid4`. The parser also computes a deterministic per-row hash, but it is ignored on upsert — identity and pruning live in the checksum

manifest (§5, ADR-0006), not the point ID. Old and new versions of a file never share an ID, which is what makes blind delete-by-recorded-UUID safe.

Collection schema + alias (created on first run)

```
vectors:
  dense:  { size: 1024, distance: Cosine }
sparse_vectors:
  sparse: {}                      # Qdrant sparse index
```

Physical collections are named `corpus_<timestamp>_<uuid8>`; **retrieval targets the alias `corpus`** (`QDRANT_COLLECTION`), never a physical name. `--recreate` builds a new physical collection alongside the live one and atomically repoints the alias.

5. Sync, idempotency & re-runs (ADR-0006 — content-addressed)

Identity is the **file content checksum** (`blake2b`), tracked in a SQLite manifest (`corpus_manifest.py`, `manifest.db`) mapping `checksum → {uuids, path, departed_at}`. Identical-content files collapse to one checksum (intended dedup). Three modes (`main`, mutually exclusive, each under a pass lock so a manual run and the watcher never collide):

- **default — incremental** (`incremental`, grace=None): in-place sync of the live collection against disk. Order is **embed-before-prune** so no file's data is ever missing mid-pass: (1) add new checksums, (2) reconcile still-present content — cancel stale departures, and on a pure rename just refresh `source_file` in the payload (no re-embed), (3) prune departed checksums immediately.
- **`--recreate`** (`recreate`): full rebuild into a fresh `corpus_<ts>_<uuid8>`, embed everything, then **atomically switch the alias** and drop the old physical collection — **zero retrieval blackout**. The one-time exception is migrating a legacy pre-ADR-0006 physical `corpus` collection (a single momentary switch). Use `--recreate` whenever the schema, embedder model, or chunking changes.
- **`--watch`** (`watch`): autonomous debounced watcher (the `ragfarm-ingester-watcher` service). Debounce/quiescence (`INGEST_DEBOUNCE`) so a half-written file is never read and event storms coalesce; a full re-scan every `INGEST_SCAN_INTERVAL` as the missed-event backstop; **deletes are grace-gated** (`INGEST_DELETE_GRACE`) — a vanished file is only marked, and reappearing before the grace expires cancels the delete (rides out atomic-save / rsync windows). Read-only filesystem events (the watcher's own checksum reads) are ignored to avoid a self-feeding loop.

A model change (dim or model id) mixed into an existing collection is silently wrong; the alias/`--recreate` design makes the correct action (rebuild + switch) the easy one. Keep `models/embeddings/MODEL.md` in step with what a collection was built against.

6. Why hybrid (dense + sparse) for this corpus specifically

- **Dense** carries semantics and crosses languages: a Czech query finds the relevant English description and vice versa (the half the old English-only NPU build could never do). It embeds `text_clean`, so Markdown syntax doesn't skew it.
- **Sparse** carries exact lexical matches: querying `prod-kvm-03` or `10.20.1.43` hits the precise table row even though those tokens have no "semantics" to embed. Pure dense retrieval is unreliable for identifiers; sparse fixes it — which is why `text_clean` deliberately keeps URL/host tokens.
- `search_corpus` (rag-retrieval) fuses both with RRF over a broad candidate pool, then re-ranks with a GPU cross-encoder (bge-reranker-v2-m3 via a dedicated llama.cpp/Vulkan server on `:8081`, ADR-0008), so one tool call serves both "which VLAN is host X on" (lexical) and "how do we handle host maintenance" (semantic, either language).

7. The automatic watcher (deployed autonomous sync)

This expands §5's `--watch` mode into the full operational picture — it is how the corpus stays in sync in normal running, with **no manual ingest**.

Deployment

The watcher runs as the systemd service **`ragfarm-ingester-watcher`** (host, as `dave`): `ingester.py --watch`, `Restart=on-failure`, logs to `logs/ingester-watcher.log`. It `Wants`/`After` `ragfarm-embedder.service` because every pass calls `/embed` — the embedder must be up first. It reads `CORPUS_PATH=/data/corpus`, writes through the alias `corpus` (`QDRANT_COLLECTION`), and keeps its manifest in `services/ingester/manifest.db` (`MANIFEST_DB`). Start/stop/status like any host service (see `docs/deployment.md` → Autostart).

How a pass is triggered

A background thread watches `CORPUS_PATH` recursively (watchdog **inotify** `Observer`; set `INGEST_WATCH_POLL=1` to use the `PollingObserver` on a network / overlay filesystem where inotify is unreliable). Two things schedule a pass:

- **Event + debounce.** A content-mutating event marks the state dirty; a pass runs only after `INGEST_DEBOUNCE` seconds of **quiescence**, so a half-written file is never read and an event storm coalesces into one pass. **Read-only events**

are ignored — every pass opens and reads each file to checksum it, which the observer sees as `opened`/`closed_no_write`; reacting to those would let the watcher's own hashing re-arm the debounce and spin forever (a self-feeding loop).
- **Full-scan backstop.** Regardless of events, a full re-scan runs every `INGEST_SCAN_INTERVAL` seconds, catching anything inotify missed.

What a pass does

Exactly the §5 `incremental()` sync, but with **grace-gated deletes** (`grace=INGEST_DELETE_GRACE`), embed-before-prune ordered so retrieval never sees a gap mid-pass:

1. **Add** new checksums (parse → embed → upsert, new uuid4 IDs, manifest updated).
2. **Reconcile** still-present content: cancel a pending departure if a file came back; on a pure rename just refresh `source_file` in the payload — **no re-embed**.
3. **Depart**: a vanished checksum is only marked with a timestamp, not deleted.
4. **Reap**: content absent for ≥ grace is deleted; reappearing before grace expires cancels the delete. This rides out atomic-save / rsync / editor-swap windows where a file briefly disappears and returns.

Concurrency & robustness

- Every pass takes a **non-blocking lock** on the manifest. If a manual run (`--recreate` or a manual incremental, which take the lock blocking) is in progress, the watcher **skips that tick and retries next** — the two never double-write. After a manual `--recreate` flips the alias, the watcher's next pass automatically operates on the new physical collection (it resolves the alias each pass), so the two modes compose cleanly.
- A failed pass is logged and the loop continues; a crashed process is restarted by systemd (`RestartSec=5`).
- The watcher only does **incremental** sync — it never `--recreate`s. A schema, embedder-model, or chunking change still needs a manual `--recreate` (§5); the watcher then picks up the new collection via the alias.

Deployed knobs (unit values; all env-overridable)

env	unit value	meaning
`INGEST_DEBOUNCE`	60 s	quiescence required after an event before a pass
`INGEST_DELETE_GRACE`	120 s	how long a vanished file is tolerated before its points are pruned
`INGEST_SCAN_INTERVAL`	3600 s	full re-scan backstop for missed events
`INGEST_WATCH_POLL`	unset (inotify)	`1` → polling observer for network/overlay FS

8. Verification (feeds step 04's gate)

After ingesting the corpus (or a 2–3 file subset):

- The offline parser regression passes: `FIXTURES=tests/fixtures python services/ingester/test_xlsx_tables.py` → `ALL PASS`.
- Alias `corpus` resolves to a physical collection with point count > 0 and a schema showing `dense` (1024) + `sparse`.
- A query for a known hostname returns its exact table row in the top hits (sparse path works).
- A Czech semantic query returns a relevant doc chunk (multilingual dense works). Together these prove the whole reason for the BGE-M3/CPU redo and the ADR-0006/0007 rework.

\newpage

Chapter 8a Embedder README

Embedder — BAAI/bge-m3 on CPU (`:8090/embed`)

Per ADR-0002 the embedder runs **BAAI/bge-m3 on CPU** (the NPU path was abandoned — English-only, seq-limited models don't fit this mixed Czech/English, wide-table corpus; `quark_quantize.py` here is a vestige of that abandoned NPU attempt).

One endpoint, one job

This service exposes exactly **one** endpoint: `POST /embed`. It returns BGE-M3's dense (1024-dim) + sparse vectors in a single pass, for both ingestion and query embedding. That is all it does.

Reranking is NOT here. It used to be a `/rerank` sub-endpoint on this service, but the reranker and the embedder no longer share a model family, a device,

or a purpose, so co-hosting them was a wart (ADR-0008). The cross-encoder reranker is now its own dedicated GPU service — a `llama-server --reranking` instance on **:8081** — with its own unit (`manifests/ragfarm-reranker.service`) and record (`models/reranker/MODEL.md`). This service stays embeddings-only.

Contract

```
POST http://127.0.0.1:8090/embed
  {"input": ["text1", ...], "kind": "passage"|"query"}
->{"dense": [[...1024...]], "sparse": [{"<tok_id>": weight, ...}], "dim": 1024}
```

Run

Code: `services/embedder/server.py` (FastAPI + FlagEmbedding `BGEM3FlagModel`). Unit: `manifests/ragfarm-embedder.service` (host, CPU; `EMBED_MODEL_PATH` pins the BGE-M3 snapshot). Model record + revision: `models/embeddings/MODEL.md`.

\newpage

Chapter 8b llama.cpp README

iGPU llama.cpp + Vulkan on Radeon 890M — TWO servers

Per ADR-0001 we run llama.cpp on the **iGPU via Vulkan** (not ROCm, which is unofficial on gfx1150). We now run **two** `llama-server` instances from the same build, both on the iGPU, both Vulkan:

server	model	port	endpoint	unit
LLM	Qwen2.5-7B-Instruct Q4_K_M GGUF	8080	OpenAI-compatible (`--jinja`, tool-calling)	`manifests/ragfarm-llama.service`
reranker	bge-reranker-v2-m3 f16 GGUF	8081	`--reranking` (cross-encoder scores)	`manifests/ragfarm-reranker.service`

The reranker moved here (from a CPU sub-endpoint on the embedder) because llama.cpp's `--reranking` runs the cross-encoder on the **same Vulkan iGPU** as the LLM — no ROCm, no torch — cutting rerank latency from ~36 s to ~1.7 s (ADR-0008). The two are separate processes: a single `llama-server` serves one model, and there is no shared KV cache to gain (a reranker is a single-pass encoder — no autoregressive cache — and caches are model-specific anyway). VRAM is ample: ~4.7 GB LLM + ~1.2 GB reranker in the ~48 GB UMA. Records: `models/llm/MODEL.md`, `models/reranker/MODEL.md`.

The build below is shared by both servers.

Per ADR-0001 the generative LLM runs on the **iGPU via Vulkan** (not ROCm, which is unofficial on gfx1150). This serves an OpenAI-compatible API with tool-calling.

Build (Vulkan backend)

```
sudo apt-get install -y libvulkan-dev vulkan-tools glslc cmake build-essential git
spirv-headers libshaderc-dev
vulkaninfo | grep -i "deviceName"   # expect Radeon 890M / RADV GFX1150

git clone https://github.com/ggml-org/llama.cpp
cd llama.cpp
cmake -B build -DGGML_VULKAN=ON -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release -j
```

Model

`Qwen2.5-7B-Instruct` GGUF, Q4_K_M. Place in `../../models/gguf/`. (7B Q4_K_M ~4.7 GB; fits comfortably in UMA. Bump iGPU/UMA allocation in BIOS if needed.)

Launch (OpenAI-compatible server with tools enabled)

```
./build/bin/llama-server \
  -m ../../models/gguf/Qwen2.5-7B-Instruct-Q4_K_M.gguf \
  --host 127.0.0.1 --port 8080 \
  -ngl 999 \                # offload all layers to the iGPU
  -c 16384 \                # context; raise if your docs need it
  --jinja \                 # enable chat template -> needed for tool-calling
  --alias qwen2.5-7b-instruct
```

Health check: `curl 127.0.0.1:8080/v1/models`

Launch — reranker (second server, same binary)

```
./build/bin/llama-server \
  -m ../../models/gguf/bge-reranker-v2-m3-f16.gguf \
  --reranking \           # cross-encoder scoring endpoint (raw logits)
  --host 127.0.0.1 --port 8081 -ngl 999 --mlock --mmap \
  -b 4096 -ub 4096 \      # each (query,doc) pair scored in ONE physical batch;
must exceed the longest pair
  --alias bge-reranker-v2-m3
```

Probe: `curl 127.0.0.1:8081/reranking -d '{"query":"q","documents":["a","b"]}'` → `{"results":[{"index":i,"relevance_score":<logit>}, ...]}`. rag-retrieval sigmoids the logit to [0,1]. The GGUF is generated from cached HF weights — see `models/reranker/MODEL.md`.

Notes

- `-ngl 999` pushes all layers to Vulkan; verify VRAM/UMA headroom with `llama-server` startup logs.
- If Vulkan underperforms, benchmark a CPU build as a floor, and only then evaluate experimental ROCm for gfx1150 — do not assume ROCm works.
- This server is the single LLM endpoint the MCP gateway/agent talks to.

\newpage

Chapter 9 Configuration reference (.env.example)

Repo-root .env — copy to .env and fill in. The real .env is gitignored and

lives only on the build host. Plain VAR=VALUE lines; no quoting needed.

HOW THIS FILE REACHES THE RUNNING SYSTEM:

`source scripts/proxy-env.sh` **loads EVERY line here into the shell (not just**

the proxy vars — the loader exports all of them). Any var referenced in

infra/compose.yaml as `${VAR:-default}` **then flows into containers via that**

shell env when you run `docker compose up`**. Always source the loader first:**

source scripts/proxy-env.sh && docker compose -f infra/compose.yaml up -d

Vars NOT listed as `${VAR}` **in compose.yaml (Category 2 below) are baked in**

as literals there and are NOT affected by anything you put in this file when

running via compose — they only apply if you run a service's server.py

directly on the host. This distinction is exactly what caused the confusion

this file exists to resolve — read the category headers below.

Full variable inventory generated by grepping every os.environ/getenv,

`${VAR}` compose substitution, and shell `${VAR}` read in the repo.

==========

1. LIVE KNOBS — vars that actually change running behavior. Two mechanisms:

1a. containers: infra/compose.yaml `${VAR:-default}` substitution.

1b. host systemd units (llama/embedder/reranker — NOT compose): each unit's

EnvironmentFile=-/home/dave/ragfarm/.env is loaded AFTER its own

Environment= line, so a value here overrides that default the

same way for all three. Restart the unit to apply.

==========

--- 1a. corpus (step 04 ingester)

Absolute path to the read-only corpus on the host. Also read directly by

services/ingester/ingester.py when run bare (BUILD_STATE step 04 passes

--corpus explicitly instead, which overrides this — see BUILD_STATE.md).
CORPUS_PATH=/data/corpus

--- OpenNebula MCPs: mock mode + safety allowlist (ADR-0004) ----------

ONE_MOCK / HOST_MOCK: "1" (default) serves canned data / simulates actions

with NO live OpenNebula — lets the agent path and confirmation UX be tested

with no cluster. Set to "0" only once ONE_XMLRPC/ ONE_AUTH are real (see

services/mcp-placement/.env.example) and HOST_ALLOWLIST is deployment-real.
#ONE_MOCK=1 #HOST_MOCK=1

HOST_ALLOWLIST: comma-separated hostnames mcp-host-control's reboot_host will

ever act on with confirm=True — a target-host-name allowlist, NOT a user

allowlist (there is currently no per-user gate; see ADR-0004 §4 / the

confirmation-wrapper pattern for who can trigger it). Refuses any host not

listed, even if the wrapper's confirmation modal was approved.

#HOST_ALLOWLIST=node-01,node-02,node-03

--- Open WebUI exposure ----------

0.0.0.0 (default) = reachable from the LAN, login-gated by WEBUI_AUTH — the

only service meant to be externally reachable. Set 127.0.0.1 for loopback-only

(reach via `ssh -L 3000:127.0.0.1:3000 <host>` instead). Restrict :3000 at the

host firewall to trusted networks regardless of this setting.

#OWUI_HOST=0.0.0.0

--- Prompt-round timeout ceilings (all seconds) ---

OWUI's aiohttp client uses AIOHTTP_CLIENT_TIMEOUT for /v1/chat/completions and

outbound tool-server calls; its built-in default is 300. AIOHTTP_CLIENT_TIMEOUT_TOOL_SERVER

defaults to AIOHTTP_CLIENT_TIMEOUT unless set. mcpo's CONNECTION_TIMEOUT is used as

SSE-read timeout for MCP-subprocess tool calls (default falls back to 900 in mcpo but

we set it explicitly for symmetry with OWUI). rag-retrieval's embedder and reranker

HTTP calls are hardcoded at 300 s (services/rag-retrieval/server.py). llama-server's

--timeout is 3600 s by default (already ample). Together these guarantee no aiohttp

deadline fires mid-generation for the Farsi/Chinese OCR (~190 s) or long RAG turns.

Set in infra/compose.yaml; values here override on the shell for one-off runs.

#AIOHTTP_CLIENT_TIMEOUT=600
#AIOHTTP_CLIENT_TIMEOUT_TOOL_SERVER=600
#CONNECTION_TIMEOUT=600

--- outbound proxy (optional)

Set these when the build host reaches PyPI / HuggingFace / container

registries through a corporate proxy. Leave unset/blank for direct access.
#HTTP_PROXY=http://proxy.corp.example:3128 #HTTPS_PROXY=http://proxy.corp.example:3128

Hosts that must BYPASS the proxy. The loader always prepends the internal

baseline (localhost,127.0.0.1,0.0.0.0,::1,host.docker.internal) so

list here only ADDITIONAL no-proxy targets — e.g. your LAN / OpenNebula subnet.
#NO_PROXY=.corp.example,10.0.0.0/8

--- 1b. host model units: LLM (ragfarm-llama), embedder (ragfarm-embedder),

reranker (ragfarm-reranker) — see the 1b note above for the override rule ---

LLM_GGUF_PATH: main GGUF. Written by `scripts/fetch-llm.sh` (download) or

`scripts/activate-llm.sh` (switch among models already on disk).
#LLM_GGUF_PATH=/home/dave/ragfarm/models/llm/qwen2.5-7b-instruct-gguf/qwen2.5-7b-instruct-q4_k_m-00001-of-00002.gguf

LLM_GGUF_MMPROJ: multimodal-projector GGUF, only for a vision-language model

(e.g. Qwen2.5-VL) — blank for text-only. fetch-llm.sh/activate-llm.sh detect and

set this automatically (from a repo's own mmproj.gguf); no manual flag needed.

They also CLEAR it when the active model has none, so a stale path never lingers.

#LLM_GGUF_MMPROJ=

EMBED_MODEL_PATH: embedder weights dir. Written by `scripts/fetch-encoder.sh`.

Swapping this changes the vector space — re-ingest is required (see deployment.md

→ "Activating a swapped model").

#EMBED_MODEL_PATH=/home/dave/ragfarm/models/embeddings/bge-m3

RERANK_GGUF_PATH: reranker GGUF, also written by `scripts/fetch-encoder.sh`

(fetches the matched embedder+reranker pair together).

#RERANK_GGUF_PATH=/home/dave/ragfarm/models/reranker/bge-reranker-v2-m3/bge-reranker-v2-m3-f16.gguf

==========

2. INTERNAL WIRING — read by each service's code (os.environ.get with a

default), but infra/compose.yaml sets these to a FIXED LITERAL per

service, not `${VAR}` substitution. Putting them in .env has NO EFFECT on

the containerized services. Listed here only so "what does this var do"

has one answer — they matter if you run a server.py directly on the host

(outside compose) for local debugging.

==========

#QDRANT_URL=http://localhost:6333 # ingester.py, rag-retrieval/server.py #EMBED_ENDPOINT=http://localhost:8090/embed # ingester.py, rag-retrieval/server.py #RERANK_ENDPOINT=http://localhost:8081/reranking # rag-retrieval -> dedicated GPU llama.cpp reranker (ADR-0008) #QDRANT_COLLECTION=corpus # ingester.py, rag-retrieval/server.py #RAG_HOST=0.0.0.0 # rag-retrieval/server.py bind address #RAG_PORT=8104 # rag-retrieval/server.py bind port #FS_SANDBOX_ROOT=/data/corpus # mcp-fs/server.py (experimental, unbridged)

RAG_PREFETCH: candidates pulled per branch (dense/sparse) before RRF fusion in

search_corpus. Not set anywhere in compose — code default (40) always applies

unless exported manually before running rag-retrieval/server.py directly.
#RAG_PREFETCH=40

--- rerank + window (ADR-0007/0008; rag-retrieval/server.py, code defaults shown) ---

All are unset in compose, so the code defaults below always apply unless

exported manually. Tune these when evaluating retrieval quality.

RAG_CANDIDATES: size of the fused RRF pool handed to the re-ranker (broad-in).

Wide enough that all same-template rows (e.g. every EPC contact) are in play.
#RAG_CANDIDATES=40

RAG_USE_RERANKER: 1 (default) = cross-encoder rerank via RERANK_ENDPOINT; 0 =

legacy MMR path (kept only for A/B — see ADR-0008 for why MMR mis-fires on the

small row-per-record chunks by treating N distinct records as "redundant").
#RAG_USE_RERANKER=1

RAG_MIN_SCORE: drop reranked hits whose normalized (sigmoid) relevance is below

this. 0.0 = keep all (default until calibrated on real dumps). Observed on the

corpus: real answers score ~0.13-0.95, junk ~0.002, so a floor in 0.01-0.1

cleanly removes noise once you've eyeballed enough result dumps.
#RAG_MIN_SCORE=0.0

RAG_MMR_LAMBDA: LEGACY, only used when RAG_USE_RERANKER=0. MMR relevance-vs-

diversity weight in [0,1]; 0.3 leans toward diversity. Superseded by the reranker.
#RAG_MMR_LAMBDA=0.3

RAG_EXPAND_NEIGHBORS: same-section neighbor chunks to fold into each returned hit

on EACH side (0 disables window expansion). Only widens split sections; never

crosses a section boundary.
#RAG_EXPAND_NEIGHBORS=1

RAG_EXPAND_MAX_WORDS: word cap on an expanded window; neighbors past the cap are

dropped so a hit never balloons back into a page-sized slab.
#RAG_EXPAND_MAX_WORDS=600

EMBED_MODEL_PATH and RERANK_GGUF_PATH are host-unit knobs, listed under 1b

above (NOT here — they are not compose-literal like the rest of this category).

==========

3. ONE-SHOT ADMIN SCRIPT OVERRIDES — infra/openwebui/setup_openwebui.py and

check_toolchain.py. Normally passed inline on the command line (they're

invoked ad hoc, not run as services), e.g.:

OWUI_EMAIL=admin@ragfarm.local OWUI_PASSWORD=… python3 infra/openwebui/setup_openwebui.py

Can be placed here instead for convenience since the loader exports

everything — but OWUI_PASSWORD is a credential, so weigh that before

putting it in a file, even a gitignored one.

---- setup_openwebui.py ----

#OWUI_URL=http://127.0.0.1:3000 #OWUI_TOKEN= # admin JWT, alternative to email+password #OWUI_EMAIL=admin@ragfarm.local #OWUI_PASSWORD= #MCPO_RAG_URL=http://127.0.0.1:8000/rag #MCPO_PLACEMENT_URL=http://127.0.0.1:8000/placement #BASE_MODEL_ID=qwen2.5-7b-instruct

---- check_toolchain.py (step-07 gate check) ----

#MCPO_URL=http://127.0.0.1:8000 #LLAMA_URL=http://127.0.0.1:8080 #MCPO_CONTAINER=infra-mcpo #OWUI_MODEL=rag-farm #RAG_TOOL_ID=server:0 #HOSTNAME_PROBE=hsmbvxip001ts #CZECH_PROBE=Jak se přistupuje z ŠA do hostingu?

==========

4. TEST-ONLY

==========

FIXTURES: fixture dir for services/ingester/ test_xlsx_tables.py (offline

parser regression, no services needed). Default tests/fixtures is correct for

a normal checkout; override only if running fixtures from elsewhere.

#FIXTURES=tests/fixtures

==========

Per-service secrets live in their own directory, not here:

services/mcp-placement/.env.example -> ONE_XMLRPC, ONE_AUTH (OpenNebula creds)

mcp-host-control currently has no secrets of its own (HOST_MOCK/HOST_ALLOWLIST

come from Category 1 above via compose), so it has no .env.example — per

ADR-0005 collocation rules, one is added only if/ when it takes secrets.

==========

\newpage

Chapter 10a Model record — LLM

Generative LLM Model Record (ADR-0001)

Field	Value
Model	Qwen2.5-7B-Instruct
Quant	GGUF Q4_K_M (2 shards)
Source	Qwen/Qwen2.5-7B-Instruct-GGUF (latest) — fetched by `scripts/fetch-llm.sh`; path in `.env` `LLM_GGUF_PATH`
Weights	models/llm/qwen2.5-7b-instruct-gguf/qwen2.5-7b-instruct-q4_k_m-0000{1,2}-of-00002.gguf (~4.7 GB, gitignored)
mmproj	none — text-only model. A vision-language swap (e.g. Qwen2.5-VL) needs a second GGUF here, path in `.env` `LLM_GGUF_MMPROJ`; see `scripts/fetch-llm.sh --list` and `docs/deployment.md` → "Model management".
Backend	llama.cpp `llama-server`, **Vulkan / iGPU** (Radeon 890M, RADV GFX1150)
Context	32768 (`-c 32768`), `--context-shift`, `--keep 3072`
Decoding	greedy / deterministic: temp 0, top_k 1, top_p/min_p 0, seed 42
Tools	`--jinja` (chat template) → OpenAI-compatible tool-calling
Service	OpenAI-compatible — http://127.0.0.1:8080/v1 (alias `qwen2.5-7b-instruct`)
Unit	manifests/ragfarm-llama.service (host, iGPU/Vulkan)
Build	infra/llama/README.md (Vulkan backend)
Updated	2026-07-21

The embedder is recorded in `../embeddings/MODEL.md`, the reranker in `../reranker/MODEL.md`. Both `llama-server` instances (this LLM on :8080 and the reranker on :8081) run on the iGPU via Vulkan. Prod-hardware swap (CUDA server, a ~30B model) is tracked in `docs/deployment.md` → "What changes on prod NVIDIA hardware".

\newpage

Chapter 10b Model record — embedder

Embedding Model Record

Field	Value
Model	BAAI/bge-m3
Revision	latest (not pinned; fetched by `scripts/fetch-encoder.sh`)
Backend	FlagEmbedding 1.4.0 (BGEM3FlagModel), CPU, FP32 (use_fp16=False)
Weights	models/embeddings/bge-m3/ — `model.safetensors` (~2.3 GB) + `sparse_linear.pt` head (load-bearing for sparse); path in `.env` `EMBED_MODEL_PATH`
Output	dense 1024-dim (L2-normalised) + sparse (lexical weights)
Languages	100+ incl. Czech and English
Max tokens	8192
Service	services/embedder/server.py — POST http://127.0.0.1:8090/embed (embeddings only)
Unit	manifests/ragfarm-embedder.service (host, CPU)
Updated	2026-07-21

The sibling cross-encoder reranker is recorded separately in `../reranker/MODEL.md` (it moved out of the embedder to its own GPU service, ADR-0008); the generative LLM in `../llm/MODEL.md`.

\newpage

Chapter 10c Model record — reranker

Reranker Model Record (ADR-0008)

Field	Value
Model	BAAI/bge-reranker-v2-m3
Revision	latest (not pinned; fetched by `scripts/fetch-encoder.sh`)
Backend	llama.cpp `--reranking`, **Vulkan / iGPU** (Radeon 890M, RADV GFX1150), f16 GGUF
Weights	models/reranker/bge-reranker-v2-m3/bge-reranker-v2-m3-f16.gguf (~1.15 GB, gitignored); path in `.env` `RERANK_GGUF_PATH`
HF source	BAAI/bge-reranker-v2-m3 (latest) — downloaded + converted to GGUF by `scripts/fetch-encoder.sh`
Output	one relevance score per (query, document). llama.cpp returns the **raw logit**; rag-retrieval applies `sigmoid` → [0,1] (identical to FlagReranker `normalize=True`)
Role	cross-encoder rerank of the fused RRF candidate pool in `search_corpus`
Languages	100+ incl. Czech and English (XLM-RoBERTa-large family, sibling of bge-m3)
Latency	**~1.7 s / 40 candidates on the iGPU** (was ~36 s on CPU inside the embedder)
Service	dedicated llama.cpp server — POST http://127.0.0.1:8081/reranking
Unit	manifests/ragfarm-reranker.service (host, iGPU/Vulkan)
Updated	2026-07-21

Why a second llama.cpp server (not an embedder sub-endpoint)

The reranker shares nothing with the embedder anymore — different model, different device (iGPU vs CPU), different purpose. It is a plain `llama-server --reranking` instance, so there is no first-party code and no `services/reranker/` directory: the unit file is the whole deliverable. See ADR-0008.

Regenerating the GGUF (weights are gitignored)

The download + f16 GGUF conversion live in the fetch script — one command:

```
scripts/fetch-encoder.sh            # fetches bge-m3 + converts this reranker
to GGUF
scripts/fetch-encoder.sh --force    # re-fetch + re-convert
scripts/fetch-encoder.sh --list     # other embedder+reranker pairs
```

It writes `RERANK_GGUF_PATH` into `.env`, which `ragfarm-reranker.service` reads.

\newpage

Chapter 11a Instrumentation — README

RAG-farm Full Transparency Instrumentation Toolkit

Complete visibility into every microsecond and byte of ragfarm LLM inference and chat orchestration.

What's been delivered

1. Extended Benchmark (`ragfarm_bench_extended.py`)

Purpose: Measure every single inference stage with absolute numbers (not just rates).

Metrics captured (per-prompt):

- Prompt tokens, bytes, estimated chars/token
- Prefill latency (ms) + prefill rate (tok/s)
- Decode latency (ms) + decode rate (tok/s)
- Completion tokens, bytes, estimated chars/token
- E2E latency (ms)
- Context growth (running total)
- Request/response payloads (full, for debugging)

Outputs:

- Human-readable terminal display (per-prompt + aggregate summary)
- CSV export (for regression tracking across commits/deploys)
- JSON export (full payloads + all metrics)

File: `/mnt/user-data/outputs/ragfarm_bench_extended.py`

---

2. Chat Execution Tracer (`chat_execution_tracer.py`)

Purpose: Trace complete chat sessions to reveal orchestration overhead, tool execution timing, and decision phase latency.

Captures (per chat session):

- Every request/response step (user input, LLM prefill, tool decision, tool execution, tool result, LLM decision phase, final generation)
- Absolute timing for each step (millisecond precision)
- Token counts and byte counts at each stage
- Tool invocation details (schema IN, execution time, schema OUT)
- Cumulative time tracking
- Overhead breakdown (tool execution %, decision phase %, LLM generation %)

Output:

- Real-time trace display (as session executes)
- Full trace report (structured timeline)
- JSON export (complete session data, all payloads)

Demo included: Run with `--demo` flag to see a simulated chat session with tool invocation.

File: `/mnt/user-data/outputs/chat_execution_tracer.py`

---

3. Documentation (`INSTRUMENTATION_GUIDE.md`)

Comprehensive guide including:

- What each metric means
- Expected values and interpretation
- Usage examples for all commands
- CSV/JSON output format reference
- How to integrate with Open WebUI
- Troubleshooting guide
- Performance targets for ragfarm

File: `/mnt/user-data/outputs/INSTRUMENTATION_GUIDE.md`

Quick start

Baseline benchmark (5 Czech prompts)

```
cd /mnt/user-data/outputs
chmod +x ragfarm_bench_extended.py
./ragfarm_bench_extended.py
```

Benchmark with results export

```
./ragfarm_bench_extended.py --rag 3 --max-tokens 512 \
  --csv bench_$(date +%s).csv \
  --json bench_$(date +%s).json
```

Chat tracer demo (no LLM required)

```
chmod +x chat_execution_tracer.py
./chat_execution_tracer.py --demo
```

Output saved to:

- Terminal: full formatted trace report
- JSON: `/mnt/user-data/outputs/chat_trace_demo.json`

Expected output (extended bench)

Per-prompt display

```
================================================================================
Run 1/5
================================================================================

Prompt:
┌─ Jak se přihlásím do EPC?
└─ 45 tokens | 180 bytes

INPUT METRICS:
  Prompt tokens:            45
  Prompt bytes:            180
  Est. chars/token:        4.0

STAGE-BY-STAGE BREAKDOWN:
  Stage                | Latency  | Tokens | Bytes    | Tok/ms
  Request prep         |     2.34 |      0 |      256 |  0.000
  Prefill              |   145.30 |     45 |      180 |  0.309
  Decode               |    87.20 |     87 |      348 |  0.998

OUTPUT METRICS:
  Completion tokens:           87
  Completion bytes:           348
  Est. chars/token:           4.0

TIMING SUMMARY:
  Prefill latency:           145.30 ms
```

```
  Decode latency:              87.20 ms
  E2E latency:                232.50 ms
  Prefill rate:               310.21 tok/s
  Decode rate:                998.28 tok/s

CONTEXT GROWTH:
  Before (tokens):              0
  Before (bytes):               0
  After (tokens):             132
  After (bytes):              528
  Context growth:             132 tokens

Answer:
┌─ EPC je interní portál...
└─ 87 tokens | 348 bytes

REQUEST PAYLOAD:
  prompt length:             180 chars
  max_tokens:                256
  temperature:              0.70

RESPONSE PAYLOAD (summary):
  prompt_tokens:              45
  completion_tokens:          87
  total_tokens:              132
```

Aggregate summary

```
═══════════════════════════════════════════════════════════════════════════════
AGGREGATE SUMMARY (5 runs)
═══════════════════════════════════════════════════════════════════════════════

ABSOLUTE TOTALS:
  Total prompt tokens:         225
  Total completion tokens:     435
  Total tokens:                660
  Total prompt bytes:          900
  Total completion bytes:     1740
  Total bytes:                2640

AVERAGES PER RUN:
  Avg prompt tokens:           45.0
  Avg completion tokens:       87.0
  Avg prefill latency:        145.30 ms
  Avg decode latency:          87.20 ms
```

```
  Avg E2E latency:           232.50 ms
  Avg prefill rate:          310.21 tok/s
  Avg decode rate:           998.28 tok/s

THROUGHPUT:
  Total inference time:       1.16 sec
  Overall throughput:        569.03 tok/s
```

Expected output (chat tracer demo)

Real-time trace

```
✓ Session started: chat_20250725_001 (Qwen2.5-7B-Q4_K_M)
→ [ 1] User prompt                        IN |      0.00ms (cumul:       1.3ms) |
17t       69b
← [ 2] LLM prefill request               OUT |    145.30ms (cumul:     102.2ms) |
64t      256b
→ [ 3] LLM tool decision                  IN |     87.20ms (cumul:     152.5ms) |
24t       97b
  🔧 query_fw_rules       |   150.50ms | in:    29b out:   200b
← [ 4] Tool result                       OUT |     0.00ms (cumul:    302.9ms) |     50t
200b [query_fw_rules]
→ [ 5] LLM decision phase                 IN |     78.30ms (cumul:     383.2ms) |
15t       61b
→ [ 6] LLM final generation               IN |    120.70ms (cumul:     503.4ms) |
69t      277b
```

Full report

```
================================================================================
CHAT EXECUTION TRACE REPORT
================================================================================

Session: chat_20250725_001
Model: Qwen2.5-7B-Q4_K_M
Timestamp: 2026-07-25T02:38:36.664428
Total duration: 503.46ms

EXECUTION TIMELINE:
Seq  Step                                  Dir     Latency        Cumul    Tokens
Bytes Tool
---  ----------------------------- --- ---------- ---------- -------- ----------
--------------------
  1 User prompt                IN      0.00       1.3      17       69
  2 LLM prefill request        OUT    145.30     102.2      64      256
```

```
 3 LLM tool decision           IN    87.20    152.5     24      97
  4 Tool  result                        OUT         0.00       302.9         50
200 [query_fw_rules]
 5 LLM decision phase          IN    78.30    383.2     15      61
 6 LLM final generation        IN   120.70    503.4     69     277

TOOL INVOCATIONS:

  query_fw_rules:
    Execution time:    150.50 ms
    Input bytes:           29
    Output bytes:         200
    Input schema:    {"network_name": "EPC_AZURE"}

AGGREGATE STATISTICS:
  Total tokens:                    239
  Total bytes:                     960
  Total time:                   503.46 ms
  LLM generation time:          272.72 ms
  Tool execution time:          150.50 ms
  Decision/thinking time:        80.24 ms
  Overhead (decision):            15.9%
  Overhead (tools):               29.9%

THROUGHPUT:
  Tokens/sec:                   474.72
  Bytes/sec:                   1906.82
```

Integration workflow

Phase 1: Establish baseline (iGPU, current setup)

```bash
# Run extended bench
./ragfarm_bench_extended.py --rag 5 \
  --csv baseline_igpu_$(date +%Y%m%d).csv \
  --json baseline_igpu_$(date +%Y%m%d).json

# Record typical values:
# - Prefill rate (tok/s)
# - Decode rate (tok/s)
# - E2E latency (ms)
# - Context growth per prompt
```

Phase 2: Profile a chat session

```
# Start with demo to understand output
./chat_execution_tracer.py --demo

# Then manually instrument your Open WebUI session:
# 1. Use browser DevTools to capture API calls
# 2. Parse into tracer format
# 3. Generate report showing orchestration overhead
```

Phase 3: vLLM migration

```
# After deploying vLLM on RTX 6000 Blackwell:
./ragfarm_bench_extended.py --rag 5 \
  --csv baseline_vllm_$(date +%Y%m%d).csv \
  --json baseline_vllm_$(date +%Y%m%d).json

# Compare:
# - Prefill: 300 tok/s → 500+ tok/s (1.7-2x improvement)
# - Decode: 1000 tok/s → 2500+ tok/s (2.5x improvement)
# - E2E: 240 ms → 100 ms (2.4x improvement)
```

Phase 4: Production monitoring

```
# Run daily benchmarks:
0 6 * * * cd /opt/ragfarm && /opt/ragfarm/ragfarm_bench_extended.py \
  --rag 3 --max-tokens 256 \
  --csv /var/log/ragfarm/bench_$(date +\%Y\%m\%d).csv \
  --json /var/log/ragfarm/bench_$(date +\%Y\%m\%d).json
```

What to look for

Good baseline (iGPU, Vulkan):

- Prefill: 280-350 tok/s
- Decode: 900-1100 tok/s
- E2E (256 tokens): 250-400 ms
- Context growth: 80-150 tokens/prompt

Red flags (investigate):

- Prefill <200 tok/s → GPU memory bandwidth issue
- Decode <500 tok/s → KV cache inefficiency or driver overhead
- E2E >1000 ms → Request queuing or model size mismatch
- Decision phase >25% of total time → Tool framework overhead
- Context grows >500 tokens/prompt → Conversation history not being truncated

vLLM targets (RTX 6000):

- Prefill: 500-800 tok/s (tensor ops + continuous batching)
- Decode: 2000-3500 tok/s (memory bandwidth fully utilized)
- E2E (256 tokens): 80-150 ms
- Decision phase: <10% (MCP overhead minimal)

Files

File	Purpose	Lines
`ragfarm_bench_extended.py`	Extended benchmark with absolute numbers per stage	~500
`chat_execution_tracer.py`	Chat session tracer with orchestration visibility	~550
`INSTRUMENTATION_GUIDE.md`	Complete reference documentation	~450
`README_INSTRUMENTATION.md`	This file — quick start and integration guide	~400

Next steps

1. **Run baseline** with current iGPU setup:

   ```
   ./ragfarm_bench_extended.py --rag 5 --csv baseline.csv
   ```

2. **Study the output** — note which stages consume most time
3. **Test chat tracer demo** — understand overhead accounting:

   ```
   ./chat_execution_tracer.py --demo
   ```

4. **Instrument a real Open WebUI chat** — identify tool orchestration overhead
5. **After vLLM deploy** — re-run benches and measure improvement
6. **Integrate into production monitoring** — daily regression detection

---

Troubleshooting

Bench won't connect to llama-server:

```
# Check if service is running
systemctl status ragfarm-llama.service

# Check endpoint
curl http://localhost:8001/health

# Check logs
journalctl -u ragfarm-llama.service -n 50
```

Tracer shows inconsistent token counts:

- Token estimation is ~4 chars/token (conservative)
- Actual tokens from LLM endpoint are more accurate (captured in full payloads)

Decision phase overhead >30%:

- Profile MCP server response time
- Check if tool schemas are large (inefficient serialization)
- Consider caching tool results

Performance interpretation

Prefill rates tell you about prompt processing efficiency:

- Token/s during prefill phase is ~3-4x lower than decode (prefill is memory-bound, decode is latency-bound on GPUs)
- vLLM's continuous batching should improve this by 2-3x vs llama.cpp

Decode rates tell you about generation speed (user perception):

- This is what users feel — the streaming response speed
- <100 tok/s → users notice stalls
- 500 tok/s → feels natural and responsive

E2E latency tells you total wait time:

- Includes prefill + decode + overhead
- vLLM should cut this by 50-70% on RTX 6000

Decision phase overhead reveals orchestration cost:

- Gap between tool returns and next LLM inference
- Should be <100 ms for responsive chat
- 300 ms → investigate tool framework, MCP server, LLM decision latency

---

All tools are self-contained, no external dependencies beyond `requests` (standard library).

Run them and let me know what the baseline numbers look like on your current setup.

\newpage

Chapter 11b Instrumentation — full guide

RAG-farm Instrumentation & Transparency Guide

Full visibility into every microsecond and byte of LLM inference and chat orchestration.

This toolkit provides two complementary tools:

1. **ragfarm_bench_extended.py** — Benchmark with absolute numbers for every inference stage
2. **chat_execution_tracer.py** — Chat session tracer capturing tool orchestration, decision phases, and overhead

---

Part 1: Extended Benchmark (ragfarm_bench_extended.py)

What it measures

Every **single stage** with **absolute numbers**:

Metric	Meaning	Example
Prefill latency	Time to process prompt (ms)	145.3 ms
Decode latency	Time to generate answer (ms)	87.2 ms
E2E latency	Total time (ms)	232.5 ms
Prompt tokens	Input size	45 tokens
Prompt bytes	Input size (UTF-8)	180 bytes
Completion tokens	Answer size	87 tokens
Completion bytes	Answer size (UTF-8)	348 bytes
Context growth	Running context accumulation	+132 tokens/run
Prefill rate	Tokens/sec during prompt processing	310.2 tok/s
Decode rate	Tokens/sec during generation	998.3 tok/s

Output format

Per-prompt output (human-readable):

```
Prompt 1/5: Jak se přihlásím do EPC?
┌─ EPC je interní portál...
├─ Request prep      |     2.34ms |      0 tokens |       256 bytes |   0.000 tok/ms
├─ Prefill           |   145.30ms |     45 tokens |       180 bytes | 0.309 tok/ms
├─ Decode            |    87.20ms |     87 tokens |       348 bytes | 0.998 tok/ms
└─ E2E               |   232.50ms

INPUT METRICS:
  Prompt tokens:           45
  Prompt bytes:           180
  Est. chars/token:      4.0

STAGE-BY-STAGE BREAKDOWN:
  Stage                | Latency  | Tokens | Bytes    | Tok/ms
  Request prep         |     2.34 |      0 |      256 |  0.000
  Prefill              |   145.30 |     45 |      180 |  0.309
  Decode               |    87.20 |     87 |      348 |  0.998

OUTPUT METRICS:
  Completion tokens:           87
  Completion bytes:           348
  Est. chars/token:          4.0
```

```
TIMING SUMMARY:
  Prefill latency:           145.30 ms
  Decode latency:             87.20 ms
  E2E latency:               232.50 ms
  Prefill rate:              310.21 tok/s
  Decode rate:               998.28 tok/s

CONTEXT GROWTH:
  Before (tokens):             0
  Before (bytes):              0
  After (tokens):            132
  After (bytes):             528
  Context growth:            132 tokens
```

Aggregate summary (all prompts):

```
AGGREGATE SUMMARY (5 runs)
═══════════════════════════════════════════════════════════════


ABSOLUTE TOTALS:
  Total prompt tokens:        225
  Total completion tokens:    435
  Total tokens:               660
  Total prompt bytes:         900
  Total completion bytes:    1740
  Total bytes:               2640

AVERAGES PER RUN:
  Avg prompt tokens:          45.0
  Avg completion tokens:      87.0
  Avg prefill latency:       145.30 ms
  Avg decode latency:         87.20 ms
  Avg E2E latency:           232.50 ms
  Avg prefill rate:          310.21 tok/s
  Avg decode rate:           998.28 tok/s

THROUGHPUT:
  Total inference time:        1.16 sec
  Overall throughput:        569.03 tok/s
```

Usage

```
# Basic: 5 default Czech prompts
./ragfarm_bench_extended.py

# 10 random samples from defaults (with replacement)
```

```
./ragfarm_bench_extended.py --prompt 10

# 3 RAG corpus queries
./ragfarm_bench_extended.py --rag 3

# Custom max tokens + RAG + save CSV + JSON
./ragfarm_bench_extended.py --rag 5 --max-tokens 512 --csv bench.csv --json
bench.json

# Custom prompts from file
./ragfarm_bench_extended.py --prompt my_prompts.txt

# Monitor after vLLM deploy (regression tracking)
./ragfarm_bench_extended.py  --rag  3  --csv  deploy_$(date  +%s).csv  --json
deploy_$(date +%s).json
```

CSV Output

Columns: `timestamp`, `run_idx`, `prompt`, `prompt_tokens`, `prompt_bytes`, `completion_tokens`, `completion_bytes`, `total_tokens`, `total_bytes`, `context_before_tokens`, `context_after_tokens`, `context_before_bytes`, `context_after_bytes`, `prefill_ms`, `decode_ms`, `e2e_ms`, `prefill_tok_s`, `decode_tok_s`

Example:

```
timestamp,run_idx,prompt,prompt_tokens,prompt_bytes,completion_tokens,completion_bytes,total_tokens
2026-07-25T02:38:36.664428,0,"Jak se přihlásím do EPC?",45,180,87,348,132,528,...
```

JSON Output

Full payloads from every request/response, including:

- Complete request payload (prompt, tokens, settings)
- Complete response payload (usage, completion, timing)
- Stage breakdown with millisecond precision

Part 2: Chat Execution Tracer (chat_execution_tracer.py)

What it captures

Every **phase** of a chat session with **absolute timing**:

1. **User input** — Size, tokens
2. **LLM prefill** — Time to process prompt, decision if tool needed
3. **Tool detection** — Which tool(s), schema extracted
4. **Tool execution** — Wall-clock time per tool, input/output schemas and sizes

5. **Decision phase** — Time between tool return and next LLM generation (who's thinking?)
6. **LLM generation** — Final answer, tokens, bytes, latency

Output format

Real-time trace output (as session runs):

```
✓ Session started: chat_20250725_001 (Qwen2.5-7B-Q4_K_M)
→ [ 1] User prompt                        IN |      0.00ms (cumul:       1.3ms) |
17t       69b
← [ 2] LLM prefill request               OUT |    145.30ms (cumul:     102.2ms) |
64t      256b
→ [ 3] LLM tool decision                  IN |     87.20ms (cumul:     152.5ms) |
24t       97b
  🔧 query_fw_rules       |   150.50ms | in:    29b out:   200b
← [ 4] Tool result                       OUT |     0.00ms (cumul:    302.9ms) |     50t
200b [query_fw_rules]
→ [ 5] LLM decision phase                 IN |     78.30ms (cumul:     383.2ms) |
15t       61b
→ [ 6] LLM final generation               IN |    120.70ms (cumul:     503.4ms) |
69t      277b
```

Full trace report:

```
EXECUTION TIMELINE:
Seq  Step                                  Dir     Latency        Cumul    Tokens
Bytes Tool
---  ----------------------------- --- ---------- ---------- -------- ----------
--------------------
 1 User prompt                IN     0.00       1.3     17       69
 2 LLM prefill request        OUT    145.30     102.2     64      256
 3 LLM tool decision           IN     87.20     152.5     24       97
  4  Tool  result                          OUT         0.00        302.9         50
200 [query_fw_rules]
 5 LLM decision phase          IN     78.30     383.2     15       61
 6 LLM final generation        IN    120.70     503.4     69      277

TOOL INVOCATIONS:
  query_fw_rules:
    Execution time:    150.50 ms
    Input bytes:           29
    Output bytes:         200
    Input schema:    {"network_name": "EPC_AZURE"}

AGGREGATE STATISTICS:
```

```
  Total tokens:                239
  Total bytes:                 960
  Total time:               503.46 ms
  LLM generation time:      272.72 ms
  Tool execution time:      150.50 ms
  Decision/thinking time:    80.24 ms
  Overhead (decision):       15.9%
  Overhead (tools):          29.9%

THROUGHPUT:
  Tokens/sec:               474.72
  Bytes/sec:               1906.82
```

Key metrics explained

Metric	Interpretation
LLM generation time	Time LLM spent on inference (excluding tools)
Tool execution time	Wall-clock time tools ran
Decision/thinking time	Gap between tool returns and next LLM request (orchestration overhead)
Overhead (decision)	% of total time spent in decision phase — lower is better
Overhead (tools)	% of total time spent executing tools — indicates tool efficiency

Usage

Demo with simulated chat session:

```
./chat_execution_tracer.py --demo
```

Integration with Open WebUI (proxy mode — not yet implemented, but framework ready):

```
# Start tracer as proxy between Open WebUI and vLLM
./chat_execution_tracer.py --listen 0.0.0.0:8002 --forward localhost:8001

# Configure Open WebUI to use proxy:
# Settings > API Settings > LLM Endpoint: http://localhost:8002/v1
```

JSON Output

```
{
  "session": {
    "session_id": "chat_20250725_001",
```

```
    "timestamp": "2026-07-25T02:38:36.664428",
    "model_name": "Qwen2.5-7B-Q4_K_M",
    "total_time_ms": 503.46,
    "total_tokens": 239,
    "total_bytes": 960,
    "tool_execution_time_ms": 150.5,
    "decision_time_ms": 80.24,
    "llm_generation_time_ms": 272.72
  },
  "requests": [
    {
      "step_number": 1,
      "step_name": "User prompt",
      "direction": "IN",
      "payload_bytes": 69,
      "estimated_tokens": 17,
      "latency_ms": 0,
      "cumulative_ms": 1.3,
      "full_payload": {...}
    },
    ...
  ],
  "tool_invocations": {
    "query_fw_rules": [
      {
        "timestamp_ms": 302.9,
        "execution_time_ms": 150.5,
        "input_bytes": 29,
        "output_bytes": 200,
        "input": {"network_name": "EPC_AZURE"},
        "output": {...}
      }
    ]
  }
}
```

Integration: Testing with Real Open WebUI Chat Session

Setup

1. **Start llama-server:**

   ```
   systemctl start ragfarm-llama.service
   ```

2. **Start Open WebUI** (if not already running):

   ```
   docker run -d -p 3000:8080 --name open-webui ghcr.io/open-webui/open-webui:latest
   ```

3. **Run extended bench as baseline:**

   ```
   ./ragfarm_bench_extended.py --rag 3 --csv baseline.csv --json baseline.json
   ```

4. **Test in Open WebUI:**

   - Open http://localhost:3000
   - Have a chat session (3-5 exchanges)
   - Include at least one tool call (e.g., ask for FW rules, contacts, etc.)

5. **Capture session manually** (until proxy mode implemented):

   - In Open WebUI developer console (F12 → Network tab), capture requests to `/v1/completions`
   - Save HAR file
   - Parse with tracer (custom JSON conversion tool)

Expected observations
Baseline bench (no tools):

- Prefill: 300-400 tok/s (GPU-bound)
- Decode: 900-1200 tok/s (memory-bound)
- E2E: 200-300 ms per prompt
- Context growth: ~50-150 tokens/prompt

Chat session (with tools):

- LLM generation: ~50-100 ms
- Decision phase: 10-50 ms (orchestration overhead)
- Tool execution: 50-500 ms (depends on tool)
- Total overhead: 15-40% (decision + orchestration)

Interpretation guide
Prefill rates (tok/s):

- <200 tok/s → GPU memory bottleneck (check bus bandwidth, tensor ops)
- 200-400 tok/s → Good (Q4_K_M on Vulkan iGPU typical)
- 400 tok/s → Excellent (RTX 6000 Blackwell target)

Decode rates (tok/s):

- <100 tok/s → Generation too slow, users waiting (unacceptable for chat)

- 100-300 tok/s → Acceptable (iGPU)
- 500 tok/s → Excellent (GPU)

Decision phase overhead:

- <5% → Minimal orchestration overhead
- 5-15% → Normal (acceptable tool coordination)
- 20% → High orchestration burden, investigate tool framework overhead

Tool execution time:

- Should be <500 ms for responsive chat
- If >1 sec, profile tool (DB queries, API calls, etc.)

Advanced: Extending the tracer for your MCP tools

To instrument your custom MCP tool invocations:

```
from chat_execution_tracer import ChatTracer

tracer = ChatTracer()
tracer.start_session(session_id="myapp_001", model_name="Qwen2.5-7B")

# Your LLM request
tracer.log_request(
    step_name="LLM decision",
    direction="IN",
    payload={"tool": "my_tool", "params": {...}},
    latency_ms=87.3
)

# Tool execution
import time
t0 = time.time()
result = my_tool(params)
execution_time = (time.time() - t0) * 1000

tracer.log_tool_invocation(
    tool_name="my_tool",
    schema={"name": "my_tool", "params": {...}},
    input_params=params,
    output=result,
    execution_time_ms=execution_time
)
```

```
# Generate report
session = tracer.end_session()
print(tracer.format_session_report(session))
```

Files

- **ragfarm_bench_extended.py** — Standalone benchmark (no dependencies beyond requests)
- **chat_execution_tracer.py** — Standalone tracer (demo mode included)
- **INSTRUMENTATION_GUIDE.md** — This file

Performance targets for ragfarm (energy distributor, NIS2)

Metric	iGPU (baseline)	RTX 6000 Blackwell (target)
Prefill rate	300 tok/s	500+ tok/s
Decode rate	1000 tok/s	2500+ tok/s
E2E (256 tokens)	400-600 ms	100-200 ms
Tool overhead	<20%	<10%
Context (8K window)	8000 tokens	32000 tokens

Troubleshooting

Bench shows 0 tok/s on prefill/decode

→ Check llama-server is up: `curl http://localhost:8001/health`

Tracer reports high decision phase overhead (>30%)

→ Check Open WebUI backend; may indicate slow MCP server response

Context grows too fast

→ Verify conversation history isn't being re-sent on every request; should use KV cache

Tool execution time > 1 sec

→ Profile with `perf` or `py-spy`:

```
python -m py_spy record -o flamegraph.html -- your_tool
```

Next steps

1. Run extended bench against current iGPU setup → establish baseline
2. Run demo tracer → verify overhead accounting works
3. Capture real Open WebUI session → identify orchestration bottlenecks
4. Migrate to vLLM + RTX 6000 → re-run both benchmarks → measure improvement
5. Integrate tracer into production monitoring (optional)

David, this gives you **complete visibility**:

- Every single stage timed to millisecond precision
- Every request/response payload captured
- Context growth tracked
- Tool orchestration overhead quantified
- Decision phase overhead isolated

Run the demo, then against your actual ragfarm setup. You'll see exactly where the time is going.

\newpage

Chapter 11c Tracer integration guide

RAG-farm HTTP Tracing & Engine Telemetry Integration Guide

Complete visibility into the ragfarm inference pipeline: HTTP calls + direct engine metrics.

Architecture Overview

Your ragfarm setup likely looks like:

```
┌──────────────────┐
│  Open WebUI      │
│  (orchestrator) │
└──────────┬──────┘
           │
           ├──→ /v1/completions → llama.cpp (generation, :8001)
           ├──→ /v1/embeddings  → llama.cpp (embedder, :8003) [optional]
```

```
        ├──→ /retrieval       → Qdrant vector DB (:6333) [optional]
        └──→ /v1/rerank       → llama.cpp (reranker, :8002) [optional]
```

The tracer captures:

1. **HTTP calls** between Open WebUI and inference engines (via proxy or log parsing)
2. **Engine telemetry** queried directly from each llama.cpp instance
3. **Call sequence** showing which engines are used in what order

---

What you need from docs/topology.md

Create/verify `docs/topology.md` contains:

```yaml
services:
  generation:
    type: "llama.cpp"
    endpoint: "http://localhost:8001"
    model: "Qwen2.5-7B-Q4_K_M"
    quantization: "Q4_K_M"
    settings: "GPU, Vulkan"

  reranker:
    type: "llama.cpp"
    endpoint: "http://localhost:8002"
    model: "BGE-M3-reranker"
    quantization: "FP32"
    settings: "CPU"

  embedder:
    type: "Embedding"
    engine: "FlagEmbedding"
    model: "BGE-M3"
    endpoint: "http://localhost:8003"

  vector_db:
    type: "Qdrant"
    endpoint: "http://localhost:6333"

orchestration:
  chat_flow:
    - step: "Embed user query"
      engine: "embedder"
      endpoint: "/embed"
```

```
    - step: "Retrieve documents"
      engine: "vector_db"
      endpoint: "/search"
    - step: "Generate answer"
      engine: "generation"
      endpoint: "/v1/completions"
    - step: "Rerank results"
      engine: "reranker"
      endpoint: "/v1/rerank"
    - step: "Return to user"
      target: "Open WebUI"
```

Tool 1: Simple Tracer (`ragfarm_tracer_simple.py`)

Best for: Quick profiling without running a proxy.

Query engines for live telemetry

```
chmod +x /mnt/user-data/outputs/ragfarm_tracer_simple.py

# Query your actual ragfarm setup
python3 ragfarm_tracer_simple.py query \
  --generation localhost:8001 \
  --reranker localhost:8002
```

Output:

```
Querying engines...

✓ generation  (localhost:8001)   2.34ms → Qwen2.5-7B-Q4_K_M
✓ reranker    (localhost:8002)   1.87ms → BGE-M3

═══════════════════════════════════════════════════════════════

RAGFARM ENGINE TELEMETRY

═══════════════════════════════════════════════════════════════

Engine        Endpoint              Model                 Status         Latency
──────────────────────────────────────────────────────────────────────────────
generation    localhost:8001        Qwen2.5-7B-Q4_K_M       ✓ OK          2.34ms
reranker          localhost:8002              BGE-M3                          ✓
OK              1.87ms
```

Saved to: `telemetry_snapshot.json`

Tool 2: HTTP Request Tracer

Best for: Seeing the exact sequence of HTTP calls during a chat.

Method A: Proxy approach (advanced)

```
python3 ragfarm_http_tracer.py \
  --listen 0.0.0.0:8000 \
  --generation localhost:8001 \
  --reranker localhost:8002 \
  --output trace.json
```

Then configure Open WebUI to use `http://localhost:8000/v1` as LLM endpoint.

Every request flows through the proxy, which captures:

- HTTP method + path
- Request/response sizes (bytes)
- Latency (wall-clock time per request)
- Status code
- Which engine handled it

Method B: Parse Open WebUI logs (simpler)

1. **Open Browser DevTools** (F12 → Network tab)
2. **Have a chat** in Open WebUI
3. **Export requests** as HAR (right-click → Save all as HAR with content)
4. **Parse with tracer** (parser not yet included, see below for manual approach)

Method C: Manual capture (most straightforward)

Run this during a chat:

```
# Terminal 1: Monitor HTTP traffic to ragfarm engines
watch -n 0.1 'netstat -i | grep -E "localhost:800[123]"'

# Terminal 2: Run curl to capture timing
time curl -X POST http://localhost:8001/v1/completions \
  -H "Content-Type: application/json" \
  -d '{"prompt":"Jak se přihlásím do EPC?","max_tokens":256}' \
  -w "@curl_format.txt" -o /dev/null
```

Where `curl_format.txt`:

```
    time_namelookup:  %{time_namelookup}\n
       time_connect:  %{time_connect}\n
    time_appconnect:  %{time_appconnect}\n
   time_pretransfer:  %{time_pretransfer}\n
      time_redirect:  %{time_redirect}\n
 time_starttransfer:  %{time_starttransfer}\n
```

```
                    ──────────────────────────────\n
     time_total:  %{time_total}\n
```

Interpreting HTTP traces

Call sequence example (typical RAG chat)

```
Session: user_chat_001
Timestamp: 2026-07-25T10:30:45.123456
Total requests: 5
Total time: 523.45ms

Seq Engine       Endpoint           Latency    Req      Resp  Status
--- ------------ ─────────────────── ────────── -------- ──── ────────
  1 embedder     localhost:8003       45.30    256      512  200  → Embed query
  2 vector_db    localhost:6333       35.10   1024     2048  200  → Search docs
  3 generation   localhost:8001      187.30    512     1024  200  → Generate answer
  4 reranker     localhost:8002       67.80   2048      256  200  → Rerank results
  5 generation   localhost:8001      145.20   1024      512  200  → Final synthesis

PER-ENGINE STATISTICS:
Engine        Requests  Total time  Avg latency
──────────────────────── ──────────── ──────────────
embedder             1       45.30      45.30ms
vector_db            1       35.10      35.10ms
generation           2      332.50     166.25ms
reranker             1       67.80      67.80ms

Total latency: 523.45ms
  Embeddings:    45.30ms   (8.7%)
  Retrieval:     35.10ms   (6.7%)
  Generation:   332.50ms   (63.5%) ← Most time here
  Reranking:     67.80ms   (12.9%)
  Overhead:      42.75ms   (8.2%)
```

What to look for

Metric	Meaning	Good	Bad
Generation latency	Time for LLM inference	<200ms	>500ms
Reranker latency	Time to score docs	<100ms	>300ms
Retrieval latency	Time to search vectors	<50ms	>200ms
Request count	HTTP calls per prompt	3-5	>10 (orchestration overhead)
Total time	End-to-end	<600ms	>1000ms

Combined workflow: Bench + Trace

Step 1: Baseline (no RAG)

```
# Extended bench (measures LLM inference alone)
python3 ragfarm_bench_extended.py --prompt 5 \
  --csv bench_baseline.csv

# Note:
# - Prefill rate (tok/s)
# - Decode rate (tok/s)
# - E2E latency (ms)
```

Step 2: Query engine telemetry

```
# Direct engine queries
python3 ragfarm_tracer_simple.py query \
  --generation localhost:8001 \
  --reranker localhost:8002
```

Step 3: Trace a chat session

```
# Method 1: Proxy (capture all traffic)
python3 ragfarm_http_tracer.py \
  --listen 0.0.0.0:8000 \
  --generation localhost:8001 \
  --reranker localhost:8002

# Method 2: Manual curl
for i in 1 2 3; do
  time curl -X POST http://localhost:8001/v1/completions \
```

```
    -H "Content-Type: application/json" \
    -d '{"prompt":"test","max_tokens":256}' \
    2>&1 | grep real
done
```

Step 4: Analyze results

Compare:

- **Bench results** → pure LLM throughput
- **Tracer results** → real chat with orchestration overhead
- **Difference** → tool orchestration cost

Example:

```
Bench (isolated LLM):
  Generation: 1000 tok/s (decode rate)
  E2E per prompt: 250ms

Chat (with tools):
  Generation request latency: 187ms (in HTTP trace)
  Tool orchestration: +100ms (decision phases, context building)
  Total E2E: ~400ms

Overhead: 250ms → 400ms = +60% from tools/orchestration
```

Integration with docs/topology.md

Step 1: Define your topology

```
cat > docs/topology.md << 'EOF'
# RAG-farm Service Topology

## Inference Pipeline
```

User Query ↓ [Embedder] ← BGE-M3 (CPU) ↓ (query embedding) [Vector DB] ← Qdrant (:6333) ↓ (top-k documents) [Generation] ← Qwen2.5-7B (GPU, Vulkan, :8001) ↓ (answer with context) [Reranker] ← BGE-M3 reranker (CPU, :8002) ↓ User Response

```
## Service Endpoints

| Service | Type | Endpoint | Model | Status |
|---------|------|----------|-------|--------|
| generation | llama.cpp | :8001 | Qwen2.5-7B-Q4_K_M | ✓ |
```

```
| reranker | llama.cpp | :8002 | BGE-M3 | ✓ |
| embedder | Embedding | :8003 | BGE-M3 | ✓ |
| vector_db | Qdrant | :6333 | — | ✓ |

EOF
```

Step 2: Map to tracer

```
# Update tracer config to match topology
python3 ragfarm_tracer_simple.py query \
  --generation localhost:8001 \
  --reranker localhost:8002 \
  --embedder localhost:8003
```

Step 3: Track changes

As you optimize:

- Update `topology.md` with performance metrics
- Re-run tracer before/after changes
- Compare HTTP traces to measure improvement

Metrics to track over time

CSV for regression testing

```
# Baseline
python3 ragfarm_bench_extended.py --rag 5 \
  --csv bench_$(date +%Y%m%d_%H%M%S).csv

# Track these:
# - prefill_tok_s (should stay stable)
# - decode_tok_s (should stay stable)
# - e2e_ms (should stay stable)
```

Engine telemetry for changes

```
# Track model/endpoint changes
python3 ragfarm_tracer_simple.py query \
  --generation localhost:8001 > telemetry_$(date +%s).txt
```

HTTP traces for orchestration

```
# Track tool overhead
python3 ragfarm_http_tracer.py ... \
  --output trace_$(date +%s).json
```

```
# Analyze: grep generation trace_*.json | wc -l
#          (more calls = more overhead)
```

Troubleshooting

"Connection refused" errors

```
# Check services are running
systemctl status ragfarm-llama.service
systemctl status ragfarm-llama-reranker.service

# Check endpoints respond
curl -s http://localhost:8001/health | jq .
curl -s http://localhost:8002/health | jq .
```

Tracer shows 0 requests

```
# Check if Open WebUI is configured correctly
# Edit Open WebUI settings → API Settings → LLM Endpoint
# Should be: http://localhost:8000/v1 (if using proxy)
# Or: http://localhost:8001/v1 (if direct)

# Test manually:
curl -X POST http://localhost:8000/v1/completions \
  -H "Content-Type: application/json" \
  -d '{"prompt":"test","max_tokens":10}'
```

Inconsistent latencies

```
# Check system load
top -b -n 1 | grep Cpu

# If high:
# - Background tasks stealing resources
# - Other services contending for GPU
# - vLLM needs optimization

# Profile:
python -m py_spy record -o flamegraph.html -- <command>
```

Files you need

File	Purpose
`ragfarm_bench_extended.py`	Baseline LLM throughput
`ragfarm_tracer_simple.py`	Query engines + parse traces
`ragfarm_http_tracer.py`	HTTP proxy for detailed tracing
`ragfarm_integrated_tracer.py`	Combined HTTP + engine metrics (requires aiohttp)
`docs/topology.md`	Your service configuration
`TRACER_INTEGRATION_GUIDE.md`	This file

Quick start (TLDR)

```
# 1. Check engines are responding
python3 ragfarm_tracer_simple.py query --generation localhost:8001

# 2. Benchmark baseline
python3 ragfarm_bench_extended.py --rag 3 --csv baseline.csv

# 3. Have a chat in Open WebUI, then:
#    Open DevTools (F12) → Network tab
#    Filter to "localhost:8001", "localhost:8002", etc.
#    Note the latencies

# 4. For detailed tracing, run HTTP tracer as proxy
python3 ragfarm_http_tracer.py \
  --listen 0.0.0.0:8000 \
  --generation localhost:8001 \
  --reranker localhost:8002

# 5. Configure Open WebUI to use http://localhost:8000/v1
# 6. Have another chat — tracer captures everything
# 7. Check output: http_trace.json
```

All tools work with your current llama.cpp setup. No vLLM changes needed yet.

When you do migrate to vLLM:

1. Update `topology.md` (new endpoint)
2. Re-run benches (should see 2-3x improvement)
3. Re-run tracer (should see less orchestration overhead)
4. Compare before/after JSONs

\newpage

Chapter 11d RAG pipeline setup notes

RAG Pipeline Tracing Setup - Questions for David

To build the RAG pipeline tracer that shows context blowup at each stage, I need to know your exact flow:

1. Qdrant Retrieval

- **Endpoint:** (default localhost:6333?)
- **Top-K retrieved initially:** How many candidates do you get from Qdrant? (e.g., top-100? top-500?)
- **What's the score type:** similarity_score? cosine? bge_score?

2. RRF (Reciprocal Rank Fusion)

- **Who implements it:** Open WebUI? mcpo? Custom script in ragfarm?
- **Where does it run:** Before/after MMR?
- **Input:** All Qdrant results?
- **Output:** Reranked list, how many survive? (e.g., top-100 → top-50?)

3. MMR (Max Marginal Relevance)

- **Who implements it:** Same as RRF?
- **Diversity threshold:** What value? (higher = more filtering)
- **Input:** RRF output?
- **Output:** Further filtered, how many? (e.g., top-50 → top-20?)

4. Reranker (llama.cpp instance on :8002)

- **Model:** BGE-M3? Something else?
- **Input:** MMR output (top-20? top-10?)
- **Output:** Final scored list, how many to LLM? (e.g., top-5?)
- **Score range:** 0-1? or raw logits?

5. LLM Context

- **Generation llama.cpp:** localhost:8001?
- **Max context window:** 4K? 8K? 32K?
- **How context built:** (Qdrant docs + user query + system prompt)?

- **Current problem:** Does context overflow (grow >4K)? When? After how many turns?

Example output I'll build:

```
RETRIEVAL CANDIDATE POOL EVOLUTION
═══════════════════════════════════════════════════════════

[1] Qdrant Initial Retrieval: 100 candidates
──────────────────────────────────────────────
  rank score text[:200]
  1    0.95  "EPC is internal portal for management..."
  2    0.93  "Login requires domain credentials..."
  3    0.88  "FAQ: common authentication errors..."
  ...
  100  0.45  "Old archived docs, not relevant..."

[2] After RRF: 50 candidates
──────────────────────────────────────────────
  (scores adjusted by reciprocal rank fusion)
  1    0.97  "Login requires domain credentials..."
  2    0.95  "EPC is internal portal..."
  ...
  50   0.52  "..."

[3] After MMR: 20 candidates
──────────────────────────────────────────────
  (diversity filter applied, some removed)
  1    0.97  "Login requires domain credentials..."
  3    0.95  "EPC is internal portal..."
  (rank 2 removed - too similar to rank 1)
  ...
  20   0.67  "..."

[4] After Reranker: 5 candidates
──────────────────────────────────────────────
  (BGE-M3 final scoring)
  1    0.98  "Login requires domain credentials..."
  2    0.96  "EPC is internal portal..."
  ...
  5    0.78  "..."

[5] Context for LLM: 4 docs selected
──────────────────────────────────────────────
  Documents: 2048 tokens
```

```
  System prompt: 512 tokens
  User query: 45 tokens
  ──────────────────────
  Total context: 2605 tokens (65% of 4K window)

[6] LLM Generation
──────────────────────────────────────────

  Generated: 256 tokens
  Documents cited: [1, 2, 4]
  Lost citation: [3, 5]
```

Once you answer these, I'll build:

1. **Qdrant query** → get initial results with scores
2. **RRF tracer** → show before/after scores
3. **MMR tracker** → show which docs filtered
4. **Reranker input/output** → query your :8002 instance, show final scores
5. **LLM context builder** → show what actually goes to LLM
6. **Citation extraction** → which docs the LLM actually cited/used

This will show you exactly where context blowup happens.

\newpage

Chapter 11e Quick reference — instrumentation

RAG-farm Instrumentation Quick Reference

Commands

```bash
# 1. Extended Benchmark
cd /mnt/user-data/outputs
chmod +x ragfarm_bench_extended.py

# Default (5 Czech prompts)
./ragfarm_bench_extended.py

# 10 random prompts
./ragfarm_bench_extended.py --prompt 10

# 3 RAG queries (expects knowledge base)
./ragfarm_bench_extended.py --rag 3

# With CSV/JSON export
./ragfarm_bench_extended.py --rag 5 --csv bench.csv --json bench.json
```

```
# Custom tokens + timeout
./ragfarm_bench_extended.py --rag 3 --max-tokens 512 --timeout 60

# 2. Chat Execution Tracer
chmod +x chat_execution_tracer.py

# Demo (no LLM required)
./chat_execution_tracer.py --demo

# Result saved to chat_trace_demo.json
```

Metric Interpretation Cheat Sheet

Prefill latency (ms)

```
What:    Time to process the entire prompt
Typical: 100-200 ms (Vulkan iGPU), 30-50 ms (RTX 6000)
Bad:     >500 ms (GPU memory bottleneck)
```

Decode rate (tok/s)

```
What:    Tokens generated per second (streaming speed)
Typical: 800-1200 tok/s (Vulkan iGPU), 2000-3500 tok/s (RTX 6000)
Bad:     <300 tok/s (unacceptable for chat)
Target:  >1000 tok/s for responsive chat
```

Prefill rate (tok/s)

```
What:    Prompt tokens processed per second
Typical: 250-400 tok/s (Vulkan iGPU), 500-800 tok/s (RTX 6000)
Bad:     <150 tok/s (GPU memory bandwidth issue)
```

E2E latency (ms)

```
What:    Total time from prompt to final answer
Formula: prefill_latency + decode_latency
Typical: 200-500 ms (for 256-token generation)
Target:  <300 ms for responsive chat
```

Context growth (tokens/prompt)

```
What:    How many tokens are retained after each prompt
Typical: 80-150 tokens/prompt
Bad:     >500 tokens/prompt (context window filling too fast)
Check:   Is conversation history being truncated?
```

Decision phase overhead (%)

```
What:    % of total time spent between tool return and next LLM generation
Formula: (decision_time_ms / total_time_ms) * 100
Typical: 10-20% (with tools)
Bad:     >30% (tool orchestration overhead too high)
```

Tool execution time (ms)

```
What:    Wall-clock time one tool takes to run
Typical: 50-200 ms (DB query, API call)
Bad:     >1000 ms (profiling needed)
Track:   Accumulates quickly with multiple tools
```

Output files

CSV columns

```
timestamp
run_idx
prompt
prompt_tokens, prompt_bytes
completion_tokens, completion_bytes
total_tokens, total_bytes
context_before_tokens, context_after_tokens
context_before_bytes, context_after_bytes
prefill_ms, decode_ms, e2e_ms
prefill_tok_s, decode_tok_s
```

JSON structure


```
{
  "session": {
    "session_id": "string",
    "timestamp": "ISO8601",
    "model_name": "string",
    "total_time_ms": 503.46,
    "total_tokens": 239,
    "total_bytes": 960,
    "tool_execution_time_ms": 150.5,
    "decision_time_ms": 80.24,
    "llm_generation_time_ms": 272.72
  },
  "requests": [
    {
      "step_number": 1,
      "step_name": "User prompt",
```

```
      "direction": "IN",
      "payload_bytes": 69,
      "estimated_tokens": 17,
      "latency_ms": 0,
      "cumulative_ms": 1.3,
      "full_payload": {...}
    }
  ],
  "tool_invocations": {
    "tool_name": [
      {
        "timestamp_ms": 302.9,
        "execution_time_ms": 150.5,
        "input": {...},
        "output": {...}
      }
    ]
  }
}
```

Baseline targets

Current setup (iGPU, Vulkan, Qwen2.5-7B Q4_K_M)

```
Prefill rate:          280-350 tok/s
Decode rate:           900-1200 tok/s
E2E (256 tokens):      250-400 ms
Context/prompt:        100-150 tokens
Overhead (tools):      15-25%
Overhead (decision):   8-12%
```

After vLLM + RTX 6000 Blackwell

```
Prefill rate:          500-800 tok/s     (+1.8-2.5x)
Decode rate:           2000-3500 tok/s   (+2.2-3.0x)
E2E (256 tokens):      80-150 ms         (+3-5x faster)
Context/prompt:        same
Overhead (tools):      10-15%            (reduced)
Overhead (decision):   5-8%              (reduced)
```

Troubleshooting matrix

Symptom	Cause	Fix
Prefill <150 tok/s	GPU memory bottleneck	Check bandwidth with profiler
Decode <300 tok/s	Model quantization loss, driver overhead	Switch model, update drivers
E2E >1000 ms	Request queuing, slow llama-server	Check CPU load, increase workers
Context >500 tokens/prompt	Not truncating history	Verify conversation truncation logic
Decision >30%	Slow tool execution or MCP overhead	Profile tools, check MCP response time
Tool >1 sec	Slow DB/API	Use profiler (py-spy), add caching
Inconsistent numbers	Token estimation vs actual	Use actual tokens from response.usage

---

Quick comparison: Before/After vLLM

```
METRIC                  iGPU          vLLM          IMPROVEMENT
──────────────────────────────────────────────────────────────
Prefill (tok/s)         300           650           +2.2x
Decode (tok/s)          1000          2500          +2.5x
E2E 256-token (ms)      350           100           +3.5x
Decision overhead       15%           8%            -46%
Tool exec overhead      25%           12%           -52%
──────────────────────────────────────────────────────────────
User perception:        "Noticeable   → "Instant
                         response"      responses"
```

Real-world example

```
User asks: "What are the EPC_AZURE FW rules?"

Timeline with tracer:
──────────────────────────────────────────────────────────
User sends prompt:      1.3 ms    (17 tokens)
```

```
LLM prefill:              102.2 ms  (decision: "need tool")
LLM decision:             152.5 ms  (24 tokens decision)
Tool execution:           302.9 ms  (query_fw_rules: 150 ms)
LLM decision phase:       383.2 ms  (thinking about result: 78 ms)
LLM generation:           503.4 ms  (final answer: 120 ms)
──────────────────────────────────────────────────────────────────

Breakdown:
- LLM work:         272.7 ms (54%)
- Tool work:        150.5 ms (30%)
- Orchestration:     80.2 ms (16%)
Total:              503.4 ms

User experience: "Feels like 500ms response time"
```

Files you need

1. **ragfarm_bench_extended.py** — Run this to measure inference
2. **chat_execution_tracer.py** — Run `--demo` to understand output
3. **INSTRUMENTATION_GUIDE.md** — Full reference
4. **README_INSTRUMENTATION.md** — Getting started
5. **QUICK_REFERENCE.md** — This file

All in: `/mnt/user-data/outputs/`

One-liner tests

```bash
# Test if llama-server is up
curl -s http://localhost:8001/health | jq .

# Get current model
curl -s http://localhost:8001/v1/models | jq '.data[0].id'

# Run quick 3-prompt benchmark with export
./ragfarm_bench_extended.py --rag 3 --max-tokens 256 --csv bench_$(date +%s).csv

# See what tracer captures
./chat_execution_tracer.py --demo | head -50

# Check baseline numbers exist
ls -lh baseline*.csv
```

Performance debugging workflow

```
1. Run baseline benchmark
   ./ragfarm_bench_extended.py --rag 5 --csv baseline.csv

2. Check if prefill OK
   grep prefill baseline.csv | awk '{print $NF}'

3. Check if decode OK
   grep decode baseline.csv | awk '{print $NF}'

4. If slow, profile LLM endpoint
   curl -X POST http://localhost:8001/v1/completions \
     -H "Content-Type: application/json" \
     -d '{"prompt":"test","max_tokens":10}' | jq '.usage'

5. If tools slow, trace one call
   ./chat_execution_tracer.py --demo
   grep "execution_time" chat_trace_demo.json

6. Identify bottleneck, fix, re-run baseline
   (go to step 1)
```

Print this page. Keep it next to your terminal when profiling ragfarm.

\newpage

Chapter 11f Quick reference — context diagnosis

Context Blowup Diagnosis - Quick Reference

One-minute setup

```
cd /mnt/user-data/outputs
chmod +x ragfarm_bench_chatid.py ragfarm_rag_tracer.py
```

Diagnose in 3 commands

1. Benchmark (pure LLM + context growth)

```
./ragfarm_bench_chatid.py --chat-id diag_001 --rag 5 --csv bench.csv

# Output: CSV shows context_before, context_after, context_growth per turn
# If context_growth > 150 tokens/turn → context blowup happening
```

2. RAG trace (show candidate pool)

```
python ragfarm_rag_tracer.py trace \
  --chat-id diag_001 \
  --query "leadb229p.lea.piz?" \
  --rag-endpoint http://127.0.0.1:8000 \
  --k 50

# Output: Shows Qdrant → RRF → Reranker pipeline
# Check: final_context_tokens (if >3000, you're near overflow)
```

3. Pool evolution (see shrinkage across different k)

```
python ragfarm_rag_tracer.py evolve \
  --query "jak zálohovat hostitele" \
  --rag-endpoint http://127.0.0.1:8000 \
  --k-values 50,100,200,500

# Output: Shows how many tokens at each pool size
# Look for: Score cliff (where scores drop sharply)
```

Workflow

Detect problem

```
./ragfarm_bench_chatid.py --chat-id test_001 --rag 10 --csv bench.csv
grep context_growth bench.csv  # >150? Problem detected.
```

Identify cause

```
# Check what RAG is returning
python ragfarm_rag_tracer.py evolve \
  --query "test query" \
  --k-values 100,200,500

# Read output:
# - k=500, Candidates=500, Total Tokens=216,000  ← HUGE
# - Suggests RAG_PREFETCH=500 is too large
```

Fix it

```
# Edit ragfarm/.env
export RAG_PREFETCH=100      # Reduce initial pool
export RAG_CANDIDATES=5      # Reduce final candidates
export RAG_MMR_LAMBDA=0.8    # More aggressive filtering

# Restart RAG service
systemctl restart ragfarm-rag  # (or however you start it)
```

```
# Verify improvement
./ragfarm_bench_chatid.py --chat-id test_002 --rag 10 --csv bench_after.csv
diff bench.csv bench_after.csv  # Should see smaller context_growth
```

Interpreting output

Bench CSV columns

```
chat_id         → Correlation ID (same across tools)
run_idx         → Which prompt (0, 1, 2, ...)
prompt_tokens   → Tokens in user query
completion_tokens → Tokens in LLM response
total_tokens    → Sum
context_before  → Cumulative tokens BEFORE this request
context_after   → Cumulative tokens AFTER this request
context_growth  → How much context grew (after - before)
```

RAG trace output

```
embed_ms        → Qdrant prefetch time
fuse_ms         → RRF fusion time
expand_ms       → MMR diversity filtering time
rerank_ms       → Reranker scoring time (usually dominates)
final_context_tokens → Total tokens if all candidates kept
```

Key metrics

Metric	Good	Bad	Action
context_growth/ turn	<100	>200	Reduce RAG_CANDIDATES
final_context_tokens (single query)	<1500	>3000	Reduce RAG_PREFETCH
rerank_ms	<2000	>5000	Increase RAG_MMR_LAMBDA (filter before reranker)
Score cliff	Yes (sharp drop after k=100)	No (gradual)	RAG_PREFETCH too large

---

Troubleshooting

"Connection refused" to RAG endpoint

```
curl -s http://127.0.0.1:8000/rag/search_corpus \
  -H 'Content-Type: application/json' \
  -d '{"query":"test","k":3}'

# If fails: RAG service not running
systemctl start ragfarm-rag-retrieval
```

Tracer shows 0 context tokens

```
# Check endpoint is correct
python ragfarm_rag_tracer.py trace \
  --query "test" \
  --rag-endpoint http://127.0.0.1:8000 \
  --k 10

# If response is empty: RAG corpus might be empty
```

Bench shows huge context_growth

```
# Expected: 80-150 tokens/turn
# If >300: Either RAG candidates huge or LLM generating too much
# Check:
python ragfarm_rag_tracer.py evolve --k-values 50,100,200
# Look at token counts → if they grow fast, RAG candidates too large
```

Files

```
ragfarm_bench_chatid.py          Benchmark with context tracking
ragfarm_rag_tracer.py            RAG pipeline visibility
CONTEXT_DIAGNOSIS_GUIDE.md       Full guide (this file's parent)
QUICK_REFERENCE.md               This file
```

Example session

```
# 1. Detect
./ragfarm_bench_chatid.py --chat-id diag_001 --rag 5 --csv bench.csv
# Output: context_growth = 180 tokens/turn (HIGH)

# 2. Investigate
python ragfarm_rag_tracer.py evolve \
  --query "leadb229p" \
  --k-values 50,100,200,500
```

```
# Output: k=500, Total Tokens=216,000 (HUGE)

# 3. Hypothesis
# RAG_PREFETCH=500 is too large. Reduce to 100.

# 4. Fix
export RAG_PREFETCH=100
# Restart service...

# 5. Verify
./ragfarm_bench_chatid.py --chat-id diag_002 --rag 5 --csv bench_after.csv
# Output: context_growth = 95 tokens/turn (GOOD)
```

Start with the 3 commands above. Share the output.

\newpage

Chapter 11g Context blowup diagnosis guide

RAG-farm Context Blowup Diagnosis Guide

Complete visibility into context growth through the full RAG pipeline.

The Problem

Your ragfarm RAG service starts with a **big candidate pool** from Qdrant and progressively filters:

```
Qdrant (RAG_PREFETCH)  →  RRF (fuse)  →  MMR (evict)  →  Reranker (score)  →  LLM
  N candidates          N (rescored)    M filtered    K final          Context grows
```

The issue: Context grows rapidly turn-by-turn because:

1. Each LLM response adds tokens
2. RAG candidates are added to context for next turn
3. Window fills up (4K, 8K limit)

You need to see:

- How many candidates per retrieval? → `RAG_CANDIDATES` / `RAG_PREFETCH` env vars
- How much do scores change through pipeline? → Qdrant → RRF → Reranker
- Which candidates actually get used by LLM? → Citation tracking
- When does context overflow? → Turn-by-turn accumulation

Tool 1: Benchmark with Context Tracking (`ragfarm_bench_chatid.py`)

Measures: Pure LLM inference + context accumulation

```
./ragfarm_bench_chatid.py --chat-id my_chat_001 --rag 5 --csv bench.csv
```

Output shows:

```
CONTEXT GROWTH TRACKING (Chat ID: my_chat_001)
═══════════════════════════════════════════════


Run  Prompt  Completion  Total  Running  Growth
1      45        87       132      132     132
2      48        92       140      272     140
3      51        95       146      418     146
4      49        88       137      555     137
5      52        91       143      698     143

CONTEXT BLOWUP ANALYSIS:
  Initial context:    0 tokens
  Final context:    698 tokens
  Total growth:     698 tokens
  Avg growth/prompt: 139.6 tokens
  Growth rate:      inf%

⚠ WARNING: Context growing rapidly!
  Current: 698 tokens
  Action: Review retrieval (Qdrant candidates), RRF/MMR filters, reranker output
```

CSV has these columns for correlation:

```
chat_id, timestamp, run_idx, prompt,
prompt_tokens, completion_tokens, total_tokens,
context_before, context_after, context_growth
```

Use this to: Detect when context starts overflowing

Tool 2: RAG Pipeline Tracer (`ragfarm_rag_tracer.py`)

Measures: Candidate pool evolution through Qdrant → RRF → MMR → Reranker

Single query trace

```
python ragfarm_rag_tracer.py trace \
  --chat-id my_chat_001 \
```

```
  --query "leadb229p.lea.piz?" \
  --rag-endpoint http://127.0.0.1:8000 \
  --output rag_trace_my_chat_001.json
```

Output shows:

```
RAG PIPELINE TRACE
═══════════════════════════════════════════════════


Chat ID: my_chat_001
Query: leadb229p.lea.piz?
Total time: 2685.0ms

TIMING BREAKDOWN:
Stage            Time (ms)  % Total
──────────────────────────────────────
embed_ms             258.3    9.6%
fuse_ms               23.6    0.9%
expand_ms              0.0    0.0%
rerank_ms           2369.1   88.2%

CANDIDATE POOL EVOLUTION:

Qdrant Prefetch (258.3ms)
  Candidates: 10 | Initial retrieval from Qdrant
  [ 1] score=0.9659 | EPC25-FW-rules-EPC-20260713-DB_ENDUR.xlsx | table_row
       sheet: Firewall Rules DB ENDUR, Date Request: 13/7/2026...
  [ 2] score=0.9540 | EPC25-FW-rules-EPC-20260713-DB_ENDUR.xlsx | table_row
       sheet: Firewall Rules DB ENDUR, Date Request: 13/7/2026...
  ... (8 more)

RRF Fusion (23.6ms)
  Candidates: 10 | Sparse+dense fusion, scores may change
  [ 1] score=0.9659 | ... (same, slightly reranked)
  ... (9 more)

Reranker Score (2369.1ms)
  Candidates: 10 | LLM semantic relevance scoring
  [ 1] score=0.9800 | ... (score adjusted by reranker)
  ... (9 more)

FINAL CONTEXT:
  Candidates: 10
  Total tokens (est): 4,320
  Avg tokens/doc: 432
```

```
⚠ WARNING: Large context window
  4,320 tokens is 1x a 4K window
  This will cause context overflow after a few turns
```

Pool evolution (multiple k values)

```
python ragfarm_rag_tracer.py evolve \
  --query "jak zálohovat hostitele" \
  --rag-endpoint http://127.0.0.1:8000 \
  --k-values 50,100,200,500
```

Shows:

```
CANDIDATE POOL EVOLUTION
═══════════════════════════════════════════════════════════════


Query: jak zálohovat hostitele

k Value  Candidates  Avg Score  Min Score  Total Tokens
──────────────────────────────────────────────────────
     50          50     0.7234     0.4521       21,600
    100         100     0.6845     0.3210       43,200
    200         200     0.6234     0.2156       86,400
    500         500     0.5123     0.1034      216,000
```

Use this to:

1. See how many candidates RAG returns for each query
2. Identify score cliff (where scores drop sharply)
3. Calculate context blowup: `Total Tokens × avg_turns_per_session`

Workflow: Diagnose Context Overflow

Step 1: Run bench to see context growth

```
# 5 RAG prompts, track context
./ragfarm_bench_chatid.py --chat-id diag_001 --rag 5 --csv bench_diag.csv

# Check the CSV
cat bench_diag.csv | awk -F, '{print $1, $10, $11, $12}' \
  # Shows: chat_id context_before context_after context_growth
```

If context_growth is >150 tokens/prompt: → Go to Step 2

Step 2: Trace actual RAG queries

```
# Use same chat_id to correlate
for query in "leadb229p.lea.piz?" "jak zálohovat" "Vypiš FW pravidla"; do
  python ragfarm_rag_tracer.py trace \
    --chat-id diag_001 \
    --query "$query" \
    --output rag_trace_diag_001_${i}.json
done

# Check final_context_tokens in each JSON
jq '.final_context_tokens' rag_trace_diag_001_*.json
```

If final_context_tokens > 2000 for each query: → Go to Step 3

Step 3: Identify the bottleneck

Check which RAG service env vars are set:

```
# Show RAG service config (from ragfarm/.env or container logs)
echo "RAG_PREFETCH (initial pool size):"
echo "RAG_CANDIDATES (final candidates to LLM):"
echo "RAG_MMR_LAMBDA (diversity threshold):"
echo "RAG_USE_RERANKER (true/false):"
```

Then run pool evolution to see shrinking:

```
python ragfarm_rag_tracer.py evolve \
  --query "jak zálohovat hostitele" \
  --k-values 100,200,500,1000

# Look for:
# - k=1000, Candidates=1000, Total Tokens=500,000  ← TOO MANY
# - Score drops sharply after k=200 ← candidates are low quality below this
# - Reranker time dominates ← LLM bottleneck
```

Step 4: Adjust RAG configuration

Based on findings:

```
# CASE 1: Too many initial candidates
# Solution: Reduce RAG_PREFETCH in .env
export RAG_PREFETCH=200  # was 500

# CASE 2: Too many final candidates to LLM
# Solution: Reduce RAG_CANDIDATES in .env
export RAG_CANDIDATES=5  # was 10

# CASE 3: Reranker is slow
# Solution: Increase RAG_MMR_LAMBDA (filter more before reranker)
```

```
export RAG_MMR_LAMBDA=0.8  # was 0.6

# CASE 4: Reranker is disabled
# Solution: Enable it to filter low-quality candidates
export RAG_USE_RERANKER=true

# Restart RAG service
systemctl restart ragfarm-rag-retrieval

# Re-run bench to verify improvement
./ragfarm_bench_chatid.py --chat-id diag_002 --rag 5 --csv bench_diag_after.csv
```

Interpreting Results

Good baseline (before optimization):

```
Bench:
  context_growth per prompt: 120-150 tokens
  final context after 5 turns: 600-750 tokens

RAG trace:
  Qdrant prefetch: 500 candidates
  After RRF: 500 (rescored)
  After reranker: 10 candidates to LLM
  Final context: 2048 tokens
```

Warning signs (context blowup):

```
Bench:
  context_growth per prompt: >200 tokens
  final context after 5 turns: >1000 tokens (overflow at 4-5 turns)

RAG trace:
  Qdrant prefetch: 1000+ candidates (too many)
  After reranker: 20+ candidates to LLM (should be <10)
  Final context: >3000 tokens (approaching window limit)
```

Optimized (after tuning):

```
Bench:
  context_growth per prompt: 80-100 tokens
  final context after 10 turns: 800-1000 tokens

RAG trace:
  Qdrant prefetch: 100-200 candidates (controlled)
```

```
  After reranker: 5 candidates to LLM
  Final context: 1200-1500 tokens (comfortable margin)
```

CSV Correlation

Both tools use `chat_id` for correlation:

```
# Bench CSV
chat_id,run_idx,prompt,context_before,context_after,context_growth
diag_001,0,lead...,0,132,132
diag_001,1,jak...,132,272,140
diag_001,2,Vypiš...,272,418,146

# RAG JSON (from tracer)
{"chat_id": "diag_001", "query": "leadb229p.lea.piz?",
 "final_candidate_count": 10, "final_context_tokens": 4320}
{"chat_id": "diag_001", "query": "jak zálohovat",
 "final_candidate_count": 10, "final_context_tokens": 4850}

# Cross-reference:
# Row 1 of bench → RAG query 1 (leadb229p.lea.piz?)
# Row 2 of bench → RAG query 2 (jak zálohovat)
# Check: context_growth (132) vs final_context_tokens (4320)
#        The RAG candidates are NOT all in context (they're scored, top-k selected)
```

Files

```
ragfarm_bench_chatid.py         LLM + context tracking (chat_id support)
ragfarm_rag_tracer.py           RAG pipeline visibility (pool evolution)
CONTEXT_DIAGNOSIS_GUIDE.md      This file
```

Summary

To diagnose context blowup:

1. Run `ragfarm_bench_chatid.py` → see context accumulation
2. Run `ragfarm_rag_tracer.py trace` → see candidates per query
3. Run `ragfarm_rag_tracer.py evolve` → see pool shrinking
4. Compare timings + token counts → identify bottleneck
5. Adjust RAG env vars → retest

Chat ID correlation: All results tagged with same `chat_id` for cross-referencing across tools.

Next: Run this workflow and share the results. I'll help interpret.

\newpage

Chapter 12 Build progress (PROGRESS.md)

PROGRESS — blocker channel between the agent and Dave

This is the only file Dave writes into to steer the build. The agent appends `BLOCKED:` entries when it needs something only Dave can supply; Dave flips them to `UNBLOCKED:` once supplied. Linear build progress lives in `BUILD_STATE.md`, not here — this file carries only blockers and their resolution.

See `CLAUDE.md` Chapter 2 for exactly when and how this file is read and written. All timestamps are UTC. Newest entries are appended at the end. Resolved entries are kept as historical record, never deleted.

Format

```
BLOCKED: <NN-stepname> — <UTC timestamp>
  need:   <exactly what Dave must supply>
  where:  <exact path / command / .env key / BIOS field involved>
  detail: <one or two lines of context>
```

Dave clears a blocker by editing that block's first line in place:

```
UNBLOCKED: <NN-stepname> — <UTC timestamp Dave cleared it>
  supplied: <what he did — file in place, creds in .env, toggle set, etc.>
  need:   <original need, left for the record>
  where:  <original where, left for the record>
  detail: <original detail, left for the record>
```

Entries

UNBLOCKED: 01-npu-bringup — 2026-06-15T12:55Z need: Reboot the machine so the staged kernel parameter takes effect. where: /etc/default/grub — GRUB_CMDLINE_LINUX_DEFAULT now contains `amd_iommu=force_isolation`; update-grub has already been run; GRUB config is current. detail: The NPU PCI device (0000:c6:00.1) is in an IOMMU identity domain because the BIOS ACPI IVRS table has a unity-mapping entry for it. AMD IOMMU SVA (required by the amdxdna driver for PASID-based DMA) only works when the device is in a DMA-translated domain. The in-tree amdxdna driver logs "SVA bind device failed, ret

−95" on every open(). The fix is to add `amd_iommu=force_isolation` which overrides IVRS unity-mapping entries and forces all devices into isolated (translated) domains. After reboot: re-run `xrt-smi examine` and `python infra/npu/quicktest.py` to verify the gate (NPU Strix in examine output, "Test Finished" from quicktest).

BLOCKED: 05-mcp-placement — 2026-06-18T11:30Z need: live OpenNebula access — ONE_XMLRPC endpoint + ONE_AUTH credentials, and network reachability to the ON frontend where: .env keys ONE_XMLRPC, ONE_AUTH (shape in .env.example) detail: PoC MiniPC is not yet placed on the live network; ON access lands at deployment. The placement MCP code + XML parsing are unit-tested; only the live one.vm.info/one.vmpool.info round-trip is unverified.

BLOCKED: 06-mcp-fs-host-control — 2026-06-18T11:30Z need: live OpenNebula access (same as 05) before host-control real actions where: .env keys ONE_XMLRPC, ONE_AUTH detail: fs-agent (sandboxed read) can be implemented and tested now, but host-control's drain-then-reboot is via ON and cannot be verified without a live cluster. Keep host-control dry-run/confirm-gated; do not enable real actions until ON is reachable at deployment.

NOTE: ADR-0006 activated — 2026-07-14 UTC owner: Dave Kubicek (owner-directed capability change, not a build-step) summary: Content-addressed corpus sync with SQLite manifest + alias switch + autonomous watcher deployed and validated end-to-end. changes: - Step-04 "ingester frozen" rule superseded by ADR-0006 for owner-driven work; xlsx_tables.py parser remains off-limits to build agents. - Project venv moved: .venv on python3.12 replaces /opt/ryzenai/venv for both ingester and embedder. NPU env stripped from ragfarm-embedder.service. - ragfarm-ingester-watcher.service installed, enabled, wired into ragfarm-stack. - Legacy physical corpus collection migrated to alias corpus -> corpus_20260714-025915_901856e8 via --recreate (zero blackout after migration). - infra/compose.yaml ingester: block removed (superseded by host watcher). gate: All §9 gates passed; see logs/ingester-adr0006.log.

NOTE: OWUI serving tuning (context handling + tool-calling) — 2026-07-14 UTC owner: Dave Kubicek (owner-directed debugging, not a build-step) problem: Multi-turn RAG chats in Open WebUI hit a slow CPU-busy "before-tool" delay, context overflow (hard 400 errors), stalls, and degraded tool-calling (model narrated instead of calling reboot_host). Root cause: unbounded accumulation of REPLAYED tool-result blobs in OWUI conversation history — each turn resends the full prior tool outputs (a single docx "How-Tos" chunk is ~4k tokens), so prompts reached 10-14k+ tokens. A warm llama prompt-cache had hidden the cost for days; a llama restart exposed it (cold full re-prefill of the whole history). NOT caused by the .venv/embedder move

or the watcher. changes: - manifests/ragfarm-llama.service: -c 16384 -> -c 32768; added --context-shift --keep 3072. NOTE: --context-shift only rescues GENERATION overflow, NOT an oversized prompt (verified: 28k-token prompt still returns 400 exceed_context_size_error). So 32k only raises the ceiling; it does not make overflow impossible. --keep 3072 preserves system prompt + tool schemas across shifts. (-parallel 1 was tried then reverted to default 4 slots; prompt-cache restore makes single-slot safe but 4 slots matched the proven baseline.) - OWUI model "ragfarm": function_calling native -> default. THIS was the effective fix — restored immediate/correct tool calls, faster generation, and cut per-turn accumulation. Stored in the openwebui_data docker volume (webui.db), NOT version-controlled; revert with a one-field DB update. tradeoff / PROD caveat: - In "default" mode the model answers a REPEATED question from context instead of re-calling the tool (observed: 2nd "Kde bezi sftp-gw?" and 2nd "Jak se prihlasim?" made zero tool calls). Harmless in test, but STALE ANSWERS are unacceptable for production. - PROD PLAN: return to function_calling = "native" for the real deployment, paired with a larger/newer model + bigger context window (which is what lets native mode carry the accumulated trace without overflowing). - A reboot sentence that prefaced one login answer was verified as a pure context echo (that turn called zero tools) — no host-control execution. deferred lever #2 (accumulation suppression via retrieval; NOT applied): - In services/rag-retrieval/server.py, cap each result's "text" (~800 chars) and lower default k (5 -> 4) so each tool result is ~1k instead of ~4k tokens, slowing history growth. Parked because it degrades answer grounding to patch a plumbing issue; the clean fix is finer docx chunking in the (frozen) ingester parser. Keep #2 as a fallback if context buildup returns.